ABSTRACT

Title of Dissertation: Visual Content Synthesis at Scale

Songwei Ge
Doctor of Philosophy, 2025

Dissertation Directed by: Professor David Jacobs
Department of Computer Science

Humans love to create visual content. Every day, we take photos with smartphones, edit videos using intuitive apps, and create artworks through increasingly accessible digital tools. These widespread practices have led to an explosion of visual data shared continuously on the internet, building massive collections of images and videos that capture diverse human experiences. This enormous accumulation of visual data, together with rapid advancements in GPU computing, has become the foundation for training large-scale generative models, the key to automatically synthesizing top-tier visual content. By learning directly from the rich online visual repositories, these models internalize intricate patterns, styles, and concepts, enabling recompose these elements to novel samples based on the user's inputs. In this thesis, we study and design scalable generative models that digest and improve with visual data, evaluation metrics that can precisely monitor the progress, and develop applications based on these pre-trained models. This thesis begins by designing frameworks for scalable video generation models. This includes both autoregressive models trained on the discrete tokens obtained through a discrete to-

kenizer and diffusion models trained directly on the pixels. In addition, we develop a novel video tokenization schema, enabling more compact video representations for larger generative models to train on. Next, we perform a careful analysis of the mainstream automatic evaluation metric. In the last chapter of the thesis, we study several practical scenarios to apply the pre-trained large-scale generative models, with tasks not only generation and beyond the original image and video domains.

VISUAL CONTENT SYNTHESIS AT SCALE

by

Songwei Ge

Dissertation submitted to the Faculty of the Graduate School of the
University of Maryland, College Park in partial fulfillment
of the requirements for the degree of
Doctor of Philosophy
2025

Advisory Committee:
Professor David W. Jacobs, Chair/Advisor
Professor Jia-Bin Huang, Co-Advisor
Professor Tom Goldstein
Professor Angjoo Kanazawa
Professor Shuvra S. Bhattacharyya

© Copyright by
Songwei Ge
2025

To my parents and Qianhe, who believed before I did.

Acknowledgments

This thesis is the culmination of a rewarding journey made possible through the generous assistance, patience, and encouragement of numerous individuals.

I would like to begin by thanking my advisors, David Jacobs and Jia-Bin Huang, whose guidance, support, and care have profoundly shaped my PhD journey. David's broad knowledge and acute insight make him one of the best people in the world to discuss research ideas with. He always comprehends my immature ideas in a surprisingly short time and shares his insights about any tricky problem in a rather intuitive manner. I was lucky to learn from David and grow as a researcher. But David is a mentor to me more than in research. Our countless outdoor Sweetgreen meetings—snow or shine—became treasured memories, not only helping me to stay in shape but also providing delightful chats that enriched my research and life. I'm especially grateful that David always puts my personal interests first, providing me with immense intellectual freedom and enthusiastically supporting my pursuit of generative modeling, which forms the main body of this thesis.

I am equally thankful to Jia-Bin, whose vibrant personality and boundless creativity made our collaboration on generative models immensely enjoyable. Jia-Bin's enthusiasm for teaching, creating educational videos, and exploring new skills greatly influences my own professional growth. I especially enjoyed learning numerous tricks and skills in many aspects, from making paper figures to career planning. Most importantly, I'm highly grateful that he stood firmly behind me throughout several difficult times along this journey, offering reliable advocacy and emotional support. I'm indebted to David and Jia-Bin's support, who generously facilitated my opportunities to collaborate with research groups outside the school.

I feel fortunate to have had the opportunity to undertake several internships and visits, experiences that have greatly helped this thesis. First, my thanks go to Devi Parikh, Larry Zitnick, Chun-Liang Li, Haohan Wang, and Ming-Yu Liu, from the early stages of my PhD, whose invaluable advice and support significantly shaped my research practices and life habits, enabling me to survive the rigors of PhD life. My sincere thanks go to Jun-Yan Zhu for his creative and inspiring research discussions and meticulous, word-by-word feedback to improve my writing skills. Additionally, I extend my profound appreciation to Angjoo Kanazawa for warmly hosting me for quite some time at Berkeley, during which I have acquired essential skills for both life and research and made lasting friendships that I will cherish. Her emphasis on quality over quantity, along with many other philosophies on research and career, fundamentally changed me, shaping not only how I approach my work but also how I see myself as a researcher.

During my PhD, I had the great pleasure of working alongside many brilliant researchers on diverse topics. I am deeply grateful to the following extraordinary individuals (listed in no particular order): Shlok Mishra, David McAllister, Alyosha Efros, Vasu Singla, Thomas Hayes, Xi Yin, Vedanuj Goswami, Ting-Hsuan Liao, Yiran Xu, Yao-Chih Lee, Badour AlBahar, Alex Holynki, Yue Feng, Aniruddha Mahapatra, Gaurav Parmar, Jiaye Wu, Fitsum Reda, Jinwei Gu, Xian Liu, Taesung Park, Andrew Wang, Vedanuj Goswami, Ronen Basri, Gengshan Yang, Qunyh Phung, Yutong Bai, Yogesh Balaji, Seungjun Nah, Haowen Liu, Haoxiang Wang. Many of the works I published with them became part of this thesis, and I have learned so much from each of them — not only about conducting research but also about being a mentee, a collaborator, and a mentor. My special thank goes to Yogesh Balaji, who has been a role model in life and an inspiration in my journey to becoming a qualified mentor. Shlok Mishra was my research brother at Maryland, sharing everything from biryani to daily arxiv papers. I am glad to have had

numerous delightful research discussions with David McAllister at Berkeley, who also played a key role in helping me revive my basketball skills. Seungjun Nah taught me the best way to survive even the toughest deadlines.

Warm thanks go to my wonderful friends outside of research, Zhengbao Jiang, Enzo Shao, Juncheng Yang, Frank Xu, Zhiqing Sun, Hang Gao, Yiding Jiang, and Yiwei Dai, whose pleasant support, laughter, and friendship provided the balance and emotional strength I needed to complete this journey. I give my gratitude to my roommates, Haoan Feng, Xue Bai, Donglai Xiang, Kezhi Kong, and Hongwei Ye. I would like to thank Sheng-Yu Wang, Chonghyuk Song, Muyang Li, Or Patshinik, Nupur Kumari, Max Jones from Jun-Yan's lab, and Brent Yi, Ethan Weber, Georgios Pavlakos, Justin Kerr, Ruilong Li, Chung Min Kim, Haven Feng, Vickie Ye, Hongsuk Choi, and Tyler Bonnen from KAIR. Special thanks go to Shuyan Zhou, Xianyi Cheng, Yufei Ye, Mingxuan Wu, Riley Peterlinz, Gengshan Yang, Sha Yi, Yifei Zhang, Juncheng Liu, and Qianqian Wang, who helped take care of my naughty cat, Piggy, when I was not around.

Finally, I owe endless gratitude to my family. I thank my mom and dad for their unconditional support and love. Last and most importantly, I dedicate this thesis to my favorite human, partner in crime, light of life, and beloved wife, Qianhe Ji. Every piece of this thesis was completed under her loving supervision. Without her unwavering support, endless patience, and joyful presence, I could not have done this.

Table of Contents

Dedication ii

Acknowledgements iii

Table of Contents vi

List of Tables viii

List of Figures ix

Chapter 1: Introduction 1

Chapter 2: Background 3
- 2.1 Generative Model for Visual Synthesis . . . 3
  - 2.1.1 Generative Adversarial Network . . . 3
  - 2.1.2 Autoregressive models . . . 4
  - 2.1.3 Implicit Neural Representations . . . 4
  - 2.1.4 Diffusion models . . . 5
- 2.2 Large-scale Generative Model Training and Evaluation . . . 5
  - 2.2.1 Latent Generative Models . . . 6
  - 2.2.2 Image and Video Tokenizers . . . 6
  - 2.2.3 Leverage Knowledge from Images for Video Generation . . . 7
  - 2.2.4 Evaluation Metrics and Benchmarks . . . 7
- 2.3 Exploiting Pretrained Diffusion Models on Tasks Beyond Generation . . . 8
  - 2.3.1 Controllable Visual Synthesis and Editing . . . 9
  - 2.3.2 Controllable Visual Synthesis with Diffusion Models . . . 10
  - 2.3.3 Score Distillation Sampling . . . 10
  - 2.3.4 Generalized Visual Content Generation with SDS . . . 11

Chapter 3: Scalable Generative Model Architectures and Training 12
- 3.1 Overview . . . 12
- 3.2 Video Generation with Vector Quantized Variational AutoEncoder . . . 13
  - 3.2.1 Introduction . . . 13
  - 3.2.2 Methodology . . . 17
  - 3.2.3 Experiments . . . 24
  - 3.2.4 Conclusion . . . 31
- 3.3 Video Generation with Diffusion Models . . . 32

3.3.1 Introduction . . . 32
3.3.2 Preliminaries . . . 35
3.3.3 Method . . . 36
3.3.4 Experiments . . . 40
3.3.5 Ablation Study . . . 46
3.3.6 Conclusion . . . 48
3.4 Scaling Video Generation with Autoregressive Tokenizers . . . 49
3.4.1 Introduction . . . 49
3.4.2 Approach . . . 52
3.4.3 Experiments . . . 57
3.4.4 Conclusion . . . 64

Chapter 4: Evaluation 66
4.1 Overview . . . 66
4.2 Analyzing the Bias in Existing Evaluation Metric . . . 67
4.2.1 Introduction . . . 67
4.2.2 Quantifying the Temporal Sensitivity of FVD . . . 70
4.2.3 Probing the Perceptual Null Space in FVD . . . 76
4.2.4 Case Study: Long Video Generation . . . 80
4.2.5 Discussion . . . 83

Chapter 5: Applications 85
5.1 Overview . . . 85
5.2 Enriching Text-Conditioning with Rich Text . . . 86
5.2.1 Introduction . . . 86
5.2.2 Rich Text to Image Generation . . . 89
5.2.3 Experimental Results . . . 96
5.2.4 Visual Comparison . . . 103
5.2.5 Conclusion . . . 108
5.3 Score Distillation as a General Image Prior . . . 109
5.3.1 Introduction . . . 109
5.3.2 Method . . . 111
5.3.3 Experiments . . . 121
5.3.4 Discussion on Solving the Linear Approximation Error . . . 126
5.3.5 Conclusion . . . 128

Bibliography 129

List of Tables

3.1 Quantitative evaluation of TATS. . . . 25
3.2 Visual correlation between the first and the last frames. . . . 30
3.3 Cosine similarity of the reversed noise from image diffusion models. . . . 36
3.4 Zero-shot text to video generation evaluation on UCF-101. . . . 41
3.5 Zero-shot text to video generation evaluation on MSRVTT. . . . 41
3.6 Unconditional UCF-101 generation results. . . . 44
3.7 Quantitative results of different training strategies. . . . 45
3.8 Model configs for different size variants. . . . 57
3.9 Quantitative evaluation of discrete video tokenizers. . . . 60
3.10 Quantitative evaluation of video prediction. . . . 61
3.11 Pre-bottleneck feature ablation. . . . 62
3.12 Ablation study on downsampling strategy. . . . 62

4.1 Analyzing FVD temporal sensitivity with video distortions. . . . 72
4.2 Results of probing the temporal perceptual null space of FVD. . . . 79
4.3 FVD case study I. . . . 81
4.4 FVD case study II. . . . 82

5.1 Comparison of the text prompts statistics in existing benchmarks. . . . 100
5.2 Quantitative evaluation of long prompt generation. . . . 102
5.3 Zero-shot FID comparison with different score distillation methods. . . . 122
5.4 Quantitative comparisons of NeRF optimization . . . 124
5.5 Reducing first-order approximation error improves generation quality. . . . 126

List of Figures

3.1 Long videos generation results by our model TATS. . . . 13
3.2 Video generation results with a vanilla video VQGAN and a time-agnostic VQGAN. 15
3.3 Overview of the proposed TATS framework. . . . 21
3.4 Illustration of the temporal effects induced by paddings . . . 22
3.5 Quality degradation of long video generation. . . . 27
3.6 Video coherency evaluation of long video generation. . . . 28
3.7 Video generation results with recurrent events. . . . 30
3.8 Video generation with homomorphisms. . . . 31
3.9 Video manipulation results by modifying the texts. . . . 31
3.10 Visualizing the noise map correlations . . . 32
3.11 Model architecture of PYoCo. . . . 37
3.12 Video generation results by PYoCo. . . . 42
3.13 Qualitative comparison of PYoCo with baseline approaches. . . . 45
3.14 Visual ablation on the hyperparameter α. . . . 47
3.15 Quantitative ablation on hyperparameter α. . . . 47
3.16 Ablation on model size on video diffusion models. . . . 48
3.17 Redundancy of tokens from existing tokenizers. . . . 49
3.18 Tokenizer bottlenecks. . . . 52
3.19 Latent tokenizer framework. . . . 54
3.20 Autoregressive video tokenization. . . . 55
3.21 Transformer tokenizer scalability. . . . 58
3.22 Discrete AR tokenizer reconstruction results at different iterations and model sizes. 59
3.23 Visual comparison of image-to-video. . . . 61
3.24 Zero-shot editing propagation with AR tokenizer. . . . 63
3.25 Motion-content decomposition. . . . 64

4.1 A simple example showing that FVD is biased towards per-frame quality than temporal consistency. . . . 67
4.2 Visualization of the spatial and spatiotemporal corruptions. . . . 70
4.3 Analyzing the FVD's sensitivity to temporal consistency. . . . 71
4.4 FVD sensitivity with different video feature extractors. . . . 73
4.5 The origin of FVD sensitivity. . . . 75
4.6 Probing the perceptual null space in FVD. . . . 76
4.7 FVD decrease induced by resampling across different models and datasets. . . . 77
4.9 Candidate videos with the largest and smallest weights. . . . 78
4.11 Videos generated by StyelGAN-v and its LSTM variant. . . . 81
4.13 Video frames generated by DIGAN at different extrapolated time steps. . . . 82

5.1 Images generated with plain text vs. rich text. . . . 87
5.2 Rich-text-to-image framework. . . . 89
5.3 Region-based diffusion. . . . 92
5.4 Qualitative comparison on style control. . . . 96
5.5 Qualitative comparison on precise color generation. . . . 97
5.6 Qualitative comparison on detailed description generation. . . . 98
5.7 Qualitative comparison on token reweighting. . . . 99
5.8 Qualitative results on customized concept generation. . . . 99
5.9 Ablation of token maps. . . . 101
5.10 Quantitative evaluation of local style control. . . . 102
5.11 Quantitaive evaluation on precise color generation. . . . 103
5.12 Our rich-text-to-image workflow. . . . 104
5.13 Image editing with rich text. . . . 105
5.14 Ablation of injection method. . . . 106
5.15 Optimization with diffusion models as approximation of a Schrödinger Bridge Problem. . . . 114
5.16 Comparision of SDS variants under our analysis 116
5.17 Text-to-image generation results with COCO Captions. . . . 121
5.18 Text-guided NeRF optimization with different score distillation methods. . . . 123
5.19 Painting-to-Real comparison. . . . 125
5.20 Painting-to-Real results. . . . 126

Chapter 1: Introduction

The true sign of intelligence is not knowledge but imagination. *– Albert Einstein*

Creating visual arts has been a joyful process that people love to engage in. The visual content humans create has always shared a dynamic, symbiotic relationship with technological innovation. Impressionism flourished alongside the invention of synthetic pigments, filmmaking emerged with the advent of the zoopraxiscope, and video games blossomed through advances in computer graphics. Throughout this journey of technological progress, we've accumulated an extraordinary amount of visual data—from classical paintings to vast collections of images and videos online. Today, The availability of such data is unprecedented in human history.

Generative models are one of the effective ways to benefit from these data. By training scalable models on a large volume of visual data, the models can synthesize top-tier visual content with unseen compositionality. Further combined with various controllability tools, it empowers individuals to create their desired artistic content by instructing with natural language or operating with intuitive user interfaces, even without any skill training like it was needed before. Experts can also explore the inexhaustible creation space more efficiently and seek inspiration complementing their expertise.

To achieve this, we first need powerful synthesis methods for content generation. We study and develop scalable generation frameworks in Chapter 3 in the scenario of video generation.

The scalable models demand high-quality data and need evaluation to monitor the progress. We analyze the existing primary evaluation metric for video generation in Chapter 4. Chapter 5 explores improving the controllability of the pre-trained text-conditioned generative models by using rich text, as well as study the way to utilize a pre-trained generative model as a general prior for other tasks and domain.

Chapter 2: Background

2.1 Generative Model for Visual Synthesis

Various types of machine learning models have been developed to generate images and videos. The models are trained to model the distribution of the natural image and video, optionally conditioning on the additional information, such as texts, layout, and so on. In this section, we review several mainstream models developed over time that have reached milestones in visual synthesis, including Generative Adversarial Networks (GAN), Autoregressive models (AR), implicit neural representations (INR), and diffusion models.

2.1.1 Generative Adversarial Network

GANs incorporate two networks: a generator and a discriminator. The discriminators are trained to distinguish the real and fake data samples, while the generators are trained to fool the discriminator. Stabilizing the GAN training has long been the key to its success [1]. Image generation has contained remarkable progress [2–6] driven by GANs. Adapting GANs to video synthesis requires modeling the temporal dimension. The earliest work [7] generates entire videos using 3D deconvolution. Subsequent works separate the temporal branch from 2D frame generation and adopt RNN or LSTM [8] models to predict the latent vectors as the input to image

generators [9, 10, 10–14]. For instance, MoCoGAN [10] utilizes a RNN to sample motion vectors to synthesize different frames. MoCoGAN-HD [14] leverages an LSTM to predict a trajectory in the latent space of a trained image generator.

2.1.2 Autoregressive models

Autoregressive models generate images or videos pixel-by-pixel and are popular choices for video synthesis due to the video's causal nature. Earlier works first utilize RNN and LSTM [15, 16] or pixel CNNs [17, 18]. More recently, transformers [19, 20] have become the common choice. The challenge in AR models is their slow inference speed. Owing to the development of VQVAE [21], this problem is mitigated by training AR models on the compressed tokens [22]. Image generation has been greatly advanced by these approaches [23–26]. Furthermore, NÜWA [27] unifies the training of generative models for images and videos using a frame-based VQGAN and adopts 3D nearby self-attention for better efficiency. A frame-based compression often leads to temporal artifacts and does not fully utilize temporal redundancy. To this end, CCVS [28] utilizes a flow module to improve temporal consistency. VideoGPT [29] further leverages 3D convolution and axial self-attention. Our model falls into this line of video generators. Developing and scaling AR-based video models has thus become an active research direction [30–32].

2.1.3 Implicit Neural Representations

Implicit Neural Representations [33, 34] represent the continuous image and video data using a neural network by mapping the coordinate space to RGB space. This motivates works

that apply it to image synthesis [35, 36] by training generators to generate the weights of the neural networks. DIGAN [37] first generalized this idea to video generation by decomposing the network weights for the spatial and temporal coordinates. StyleGAN-v [38] maps a continuous temporal positional embedding to the input feature map of the StyleGAN. The biggest advantage of these models is their ability to generate arbitrarily long videos non-autoregressively. However, the videos they generate often suffer from quality degradation or contain periodic artifacts due to the positional encoding and struggle at synthesizing new content.

2.1.4 Diffusion models

Diffusion models [39, 40] learn a denoiser that iteratively transforms the noise into realistic samples, greatly simplifying the task of learning the data distribution and significantly advancing the progress of text-based photorealistic, compositional image generation [41–48]. Driven by the tremendous success in image synthesis, multiple works have proposed to explore diffusion models for conditional and unconditional video synthesis [49, 49–60]. For example, Singer et al. extend the unCLIP framework [46] to text-to-video generation, which allows training without video captions [61]. Ho et al. [62] extend the Imagen framework [47] by repeatedly up-scaling low-resolution small-fps videos in both spatial and temporal directions with multiple models [63].

2.2 Large-scale Generative Model Training and Evaluation

Fueled by the large-scale image and video datasets [64–66], various training and inference techniques [39, 40, 67, 68], and the development in just compute and infrastructure, scaling generative models have been the most effective way to achieve better generation quality. A key to

scaling is to develop more compact visual representations for the generative models to train on. In addition, making progress requires a precise estimate of the progress via proper evaluation. In this section, we review several tricks to enable large-scale generative model training, as well as the evaluation of these models.

2.2.1 Latent Generative Models

Two common approaches have been used to generate high-resolution images and videos. The first approach applies cascaded super-resolution models in the RGB space [46, 47, 61, 63, 67, 69, 70], while the second approach leverages a decoder to exploit latent space [48, 51, 58–60, 71–73, 73–80]. Training generative models in a latent space have become the most popular paradigm [42, 81–84] due to its compute-friendly property. Latent diffusion models [39, 85, 86] use a continuous tokenizer, often the Variational Autoencoder (VAE) [87], to transform visual input into a continuous latent space to reduce the computational cost. There are a growing number of video [59, 72, 73, 77, 88–92] diffusion models recently that produce remarkable progress. The other line of the latent generative model is based on transformer models [20], motivated by their success in LLMs [93, 94]. A quantizer [95] discretizes the continuous tokens into discrete indices. Similarly, there is increasing interest in using transformer and discrete tokenizers for visual understanding [96–100] and video generation [29–32, 101, 102].

2.2.2 Image and Video Tokenizers

Tokenizer design draws increasing attention due to its essential role in latent generative models. Existing works study the model architecture and training [30, 103–108], quantizer de-

sign [95, 109–114], and tokenization framework [115–118]. Several works have looked into using a transformer as the backbone for tokenization [30, 103–105, 109], but all focus on low-resolution videos due to the computational cost. Instead, we focus on high-resolution videos. A recent work [119] also looks into tokenizer scalability and finds that the bottleneck hinders scalability. ElasticTok [115] adaptively tokenizes video frames with a variable number of tokens to reduce temporal redundancy, sharing a similar motivation with our approach. However, training a generative model with ElasticTok is impractical due to the adaptive token number.

2.2.3 Leverage Knowledge from Images for Video Generation

Training large-scale visual models requires massive amounts of data to learn caption-relatedness, frame photorealism, and temporal dynamics. However, in contrast to the abundant image data resources, video data are more limited in style, volume, and quality. To resolve such scarcity issue of text-video data, previous works have resorted to different strategies to leverage knowledge from image data for text-to-video generation, including joint training on the text-image data from scratch [62, 63, 101, 102], first training a text-to-image model and then finetuning partially [52, 53, 120, 121] or entirely [61, 122] on the video dataset, and using CLIP image features as the conditional information [51, 61].

2.2.4 Evaluation Metrics and Benchmarks

Reliably evaluating generative models remains a challenge. While a user study can reflect human preference more accurately, automatic metrics serve as a more scalable evaluation protocol. Many studies have focused on understanding and improving the evaluation metrics for

image generation, such as Inception Score [123], FID [124–127], Perceptual Path Length [2], and precision and recall [128, 129]. Among them, FID is the most commonly adopted one, using the Inception-V3 feature extractor [130] trained on the ImageNet dataset [131]. However, it can sometimes diverge from human judgment, especially on the out-of-domain datasets like human faces [127, 132].

To address the above issue, researchers have introduced several variants [128, 129, 133, 134] and performed analysis to understand FID [124, 125, 135, 136]. For instance, Kynkäänniemi et al. [124] study the role of training data classes in the FID metric and advocate the use of the CLIP model as the feature extractor instead. KID [133] is proposed to improve FID using the squared Maximum Mean Discrepancy (MMD) with a polynomial kernel. KID relaxes the Gaussian assumption in FID and requires fewer samples to compute. Clean-FID [125] shows that the aliasing issue caused by the preprocessing steps could significantly affect the FID scores. Similarly, Skorokhodov et al. [38] studies the "low-level" preprocessing operations in FVD, such as resizing and frame sampling strategies. Another key to monitoring the progress is the evaluation benchmark. As text-to-image models develop rapidly, many works have paid attention to the evaluation of these models [137–142]. These benchmarks focus on different aspects of the visual generation such as the text-visual alignment [137, 138] and concept learning [140, 143].

2.3 Exploiting Pretrained Diffusion Models on Tasks Beyond Generation

We start with a brief discussion of the versatile, controllable synthesis tasks enabled by various conditioning signals. We then review how pretrained text-conditioned diffusion models can be applied to these tasks, even beyond the visual modality.

2.3.1 Controllable Visual Synthesis and Editing

Spatial layouts serve as a meaningful intermediate representation, bridging the gap between abstract scene descriptions and pixel-level image synthesis. Generating images and videos according to layout has been a long-standing and compelling problem [144–150]. [148] proposed Gaussian-categorical diffusion process to extend these frameworks by jointly generating images and semantic layouts, further enriching the generative model landscape. [147] proposed Zest-Guide, a zero-shot method that uses cross-attention-derived segmentation maps for layout-guided image generation. Similarly, [150] introduced LayoutLLM-T2I to use Large Language Models (LLMs) for layout planning. They propose feedback-based sampler and relation-aware object interaction modules to enhance the quality of complex-scene image generation. [149] presented LayoutGPT, leveraging in-context learning with structured CSS-style prompts to generate layouts for both 2D images and 3D scenes. Style transfer [5, 151, 152] and Colorization [153–158] for *editing real images* have also been extensively studied. To accurately generate the image the users expect, one option is to provide more detailed prompts. Several studies have thus focused on generating images based on complex prompts [159–161]. DALL-E 3 [159] finds that training on highly descriptive synthetic captions reliably improves the alignment between text prompts and generation results. ParaDiffusion [161] utilizes pretrained LLM with a larger context window to process complex prompts. InstanceDiffusion [160] also studies detailed prompts for individual regions, while ours does *not* require layout as part of the user input.

2.3.2 Controllable Visual Synthesis with Diffusion Models

A wide range of image generation and editing applications are achieved through either fine-tuning pre-trained diffusion models [52, 121, 143, 162–166] or modifying the denoising process [41, 167–181]. For example, SDEdit [167] first adds a small amount of noise to the input and then subsequently denoises it conditioned on the target prompt. Prompt-to-prompt [169] uses attention maps from the original prompt to guide the spatial structure of the target prompt. InstructPix2Pix [182] trains an image-conditional diffusion model to follow user instructions. Although these methods can be applied to some rich-text-to-image applications, the results often fall short, as shown in Section **??**. Concurrent with our work, Mixture-of-diffusion [174] and MultiDiffusion [175] propose merging multiple diffusion-denoising processes in different image regions through linear blending. Among these methods, the attention mechanism has been used in various diffusion-based applications such as view synthesis [183–185], image editing [143, 169, 170, 172, 186], and video editing [187–190]. For instance, Attend-and-Excite [172] alleviates the missing object issue by refining corresponding cross-attention maps. Pix2pix-zero [170] minimizes the difference between the original and target image's cross-attention maps to preserve the spatial structure.

2.3.3 Score Distillation Sampling

Modalities like 3D, 4D, sketch, and vector graphics (SVGs) lack the large-scale, diverse, and high-quality datasets needed to train a domain-specific diffusion model. In these domains, previous works explore exploiting image or video as a proxy modality [191, 192]. By computing the gradient on a proxy representation with a pretrained model, optimization in the target

modality is viable with differentiable mappings, e.g. differentiable rasterization [193] for SVGs or differentiable rendering [194] for 3D objects and scenes. The seminal method, Score Distillation Sampling (SDS) [195], first proposed to apply a pretrained text-to-image diffusion model for text-to-3D generation. However, it requires a high classifier-free guidance weight and, therefore, suffers from artifacts such as over-saturation and over-smoothing. Recent works have built upon SDS to adapt it for editing tasks [196–199] or more broadly improve over the original SDS formulation [200–205]. NFSD [200] and LMC-SDS [201] inspect the individual components of the SDS gradient and propose methods to rectify the high guidance weights. However, the over-saturation problem is mitigated but not fully resolved. VSD [202] formulates the problem as particle-based variational inference and proposes to train a LoRA [206] on the fly to estimate the score of proxy distribution.

2.3.4 Generalized Visual Content Generation with SDS

Since SDS was developed for text-to-3D generation, it has also been adopted to generate various other visual content such as SVGs [71, 207], sketches [208], texture [209–213], typography [214], 3D bodies [215], dynamic 4D scenes [216–218] and illusions [219]. Among these applications, text-to-3D has been the most active research direction. In addition to designing better distillation sampling methods [200, 202, 203], prior work has also studied the underlying 3D neural representations [220–223] and leveraging multiview data to improve the 3D consistency [205, 224–227].

Chapter 3: Scalable Generative Model Architectures and Training

3.1 Overview

Earlier studies on generative models have explored various architectures, including Generative Adversarial Networks (GANs) [1, 2, 123, 228] and Variational Autoencoders (VAEs) [87, 229–231]. Despite great efforts being made to improve the stability and scalability of training these models, it is still non-trivial to perform large-scale training [4]. Training GANs involves simultaneously optimizing two networks, a generator and a discriminator, which makes it difficult to balance and easy to break. VAEs use an ELBO as the objective, which is not guaranteed to be tight.

More recently, two types of architectures stand out for their great potential of scalability, Vector-Quantized Varitional AutoEncoders (VQVAEs) [21, 81] and Diffusion Models [39, 40, 232]. VQVAEs first utilize a VAE model to compress the data into the latent space and quantize each latent by looking up the nearest neighbor in a cookbook. Then, a transformer model [20] is trained to model the discrete tokens from quantization. Diffusion models learn to remove the noise from perturbed data and perform generation through iterative denoising from Gaussian noise. Interestingly, although both models have been proven to scale up well on large-scale data, DALLE2 [42] (a diffusion-based model) is better yet smaller than DALLE [23] (a VQVAE-based model).

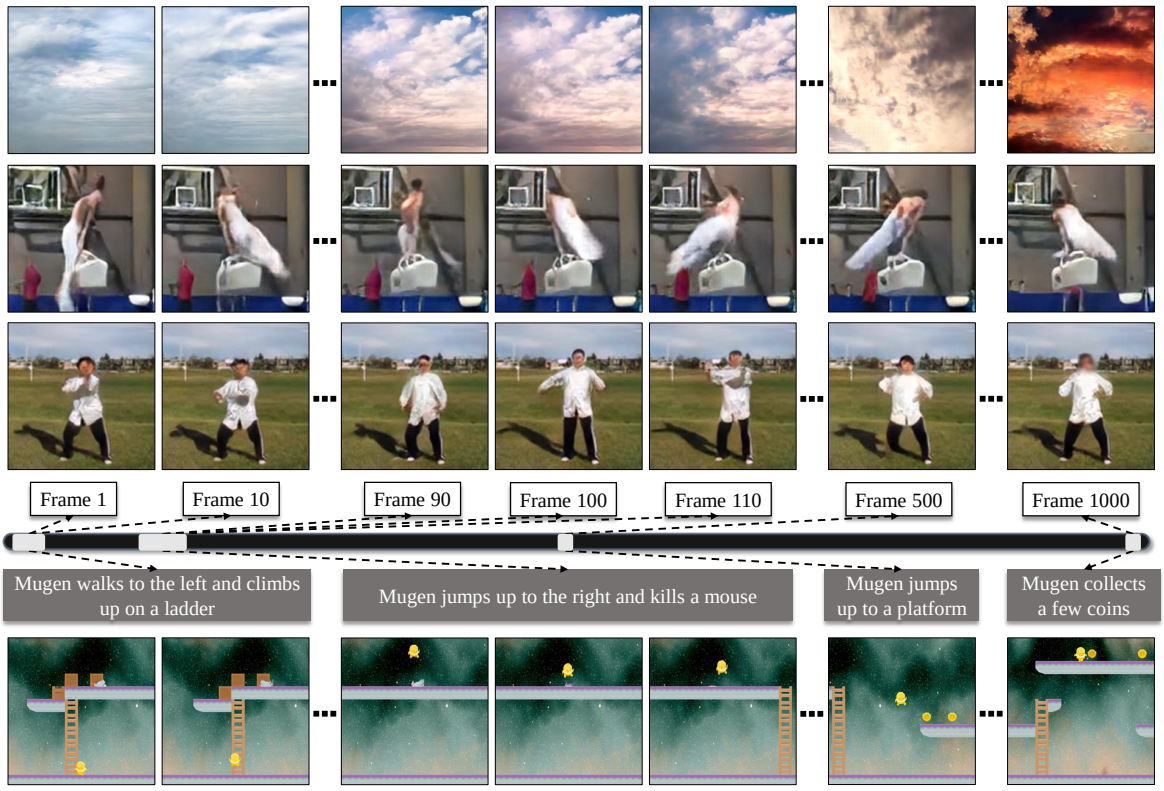


Figure 3.1: **Long videos generated by our model TATS with** 1024 **frames.**

In this Chapter, we explore applying two kinds of models to video generation. In Section 3.2, we adapt VQGAN [81] to video generation and leverage the autoregressive nature of the transformer model to synthesize longer videos. In Section 3.3, we tackle large-scale video generation by fine-tuning pre-trained image diffusion models with our proposed novel noise prior.

3.2 Video Generation with Vector Quantized Variational AutoEncoder

3.2.1 Introduction

From conveying emotions that break language and cultural barriers to being the most popular medium on social platforms, videos are arguably the most informative, expressive, diverse,

and entertaining visual form. Video synthesis has been an exciting yet long-standing problem. The challenges include not only achieving high visual quality in each frame and a natural transitions between frames, but a consistent theme, and even a meaningful storyline throughout the video. The former has been the focus of many existing studies [7, 9, 11, 14, 29] working with tens of frames. The latter is largely unexplored and requires the ability to model *long-range temporal dependence* in videos with many more frames.

Prior works and their limitations. *GAN-based methods* [7, 9, 11, 233] can generate plausible short videos, but extending them to longer videos requires prohibitively high memory and time cost of training and inference.[1] *Autoregressive methods* alleviate the training cost constraints through *sequential prediction*. For example, RNNs and LSTMs generate temporal noise vectors for an image generator [9–14]; transformers either directly generate pixel values [17, 19] or indirectly predict latent tokens [21, 22, 27–29, 81]. These approaches circumvent training on the long videos directly by unrolling the RNN states or using a sliding window during inference. Recent works use *implicit neural representations* to reduce cost by directly mapping temporal positions to either pixels [37] or StyleGAN [3] feature map values [38]. However, the visual quality of the generated frames deteriorates quickly when performing generation beyond the training video length for all such methods as shown in Figure 3.2.

Our work. We tackle the problem of long video generation. Building upon the recent advances of VQGAN [81] for high-resolution *image generation*, we first develop a baseline by extending the 2D-VQGAN to 3D (2D space and 1D time) for modeling videos. This naively extended method, however, fails to produce high-quality, coherent long videos. Our work investigates the

[1] Training DVD-GAN [11] or DVD-GAN-FP [233] on 16-frame videos requires 32-512 TPU replicas and 12-96 hours.

Figure 3.2: Video generation results with a vanilla video VQGAN and a time-agnostic VQGAN, within and beyond the training length using sliding window attention.

model design and identifies simple changes that significantly improve the capability to generate long videos of thousands of frames without quality degradation when conditioning on no or weak information. Our core insights lie in 1) removing the undesired dependence on time from VQGAN and 2) enabling the transformer to capture long-range temporal dependence. Below we outline these two key ideas.

Time-agnostic VQGAN. Our model is trained on short video clips, e.g., 16 frames, like the previous methods [7, 9–12, 14]. At inference time, we use a sliding window approach [234] on the transformer to sample tokens for a longer length. The sliding window repeatedly appends the most recently generated tokens to the partial sequence and drops the earliest tokens to maintain a fixed sequence length. However, applying a sliding attention window to 3D-VQVAE (e.g. VideoGPT [29]) or 3D-VQGAN fails to preserve the video quality *beyond the training length*, as shown in Figure 3.2. The reason turns out to be that the zero paddings used in these models *corrupts* the latent tokens and results in token sequences at the inference time that are drastically different from those observed during training when using sliding window. The amount of

corruption depends on the temporal position of the token. [2] We address this issue by using *replicate padding* that mitigates the corruption by better approximating the real frames and brings no computational overhead. As a result, the transformer trained on the tokens encoded by our time-agnostic VQGAN effectively preserves the visual quality *beyond the training video length*.

Time-sensitive transformer. While removing the temporal dependence in VQGAN is desirable, long video generation certainly needs temporal information! This is necessary to model long-range dependence through the video and follow a sequence of events a storyline might suggest. While transformers can generate arbitrarily long sequences, errors tend to accumulate, leading to quality degradation for long video generation. To mitigate this, we introduce a hierarchical architecture where an *autoregressive transformer* first generates a set of sparse latent frames, providing a more global structure. Then an *interpolation transformer* fills in the skipped frames autoregressively while attending to the generated sparse frames on both ends. With these modifications, our transformer models long videos more effectively and efficiently. Together with our proposed 3D-VQGAN, we call our model Time-Agnostic VQGAN and Time-Sensitive Transformer (TATS). We highlight the capability of TATS by showing generated video samples of 1024 frames in Figure 3.1.

Our results. We evaluate our model on several video generation benchmarks. We first consider a standard short video generation setting. Then, we carefully analyze its effectiveness for long video generation, comparing it against several recent models. Given that the evaluation of long video generation has not been well studied, we generalize several popular metrics for this task considering important evaluation axes for video generation models, including long term quality and coherence. Our model achieves state-of-the-art short and long video generation results on

[2] The large spatial span in image synthesis disguises this issue in [81].

the UCF-101 [235], Sky Time-lapse [236], Taichi-HD [237], and AudioSet-Drum [238] datasets. We further demonstrate the effectiveness of our model by conditioning on temporal information such as text and audio.

Our contributions.

- We identify the undesired temporal dependence introduced by the zero paddings in VQ-GAN as a cause of the ineffectiveness of applying a sliding window for long video generation. We propose a simple yet effective fix.
- We propose a hierarchical transformer that can model longer time dependence and delay the quality degradation, and show that our model can generate meaningful videos according to the story flow provided by text or audio.
- To our knowledge, we are the first to generate long videos and analyze their quality. We do so by generalizing several popular metrics to a longer video span and showing that our model can generate more diverse, coherent, and higher-quality long videos.

The implicit bias of zero padding. The positional inductive bias introduced by zero padding has drawn increasing attention recently [239–242]. In most cases, such inductive bias is helpful in classification [239], generation [241], or object detection [242]. Our paper observes a case where this inductive bias is harmful.

3.2.2 Methodology

In this section, we first briefly recap the VQGAN framework and describe its extension to video generation. Next, we present our time-agnostic VQGAN and time-sensitive transformer

models for long video generation (Figure 3.3).

3.2.2.1 Extending the VQGAN framework for video generation

Vector Quantised Variational AutoEncoder (VQVAE) [21] uses a discrete bottleneck as the latent space for reconstruction. An autoregressive model such as a transformer is then used to model the prior distribution of the latent space. VQGAN [81] is a variant of VQVAE that uses perceptual and GAN losses to achieve better reconstruction quality when increasing the bottleneck compression rates.

Vanilla video VQGAN. We adapt the VQGAN architecture for video generation by replacing its 2D convolution operations with 3D convolutions. Given a video $\mathbf{x} \in \mathbb{R}^{T \times H \times W \times 3}$, the VQVAE consists of an encoder $f_{\mathcal{E}}$ and a decoder $f_{\mathcal{G}}$. The discrete latent tokens $\mathbf{z} = \mathbf{q}(f_{\mathcal{E}}(\mathbf{x})) \in \mathbb{Z}^{t \times h \times w}$ with embeddings $\mathbf{c}_z \in \mathbb{R}^{t \times h \times w \times c}$ are computed using a quantization operation $\mathbf{q}$ which applies nearest neighbor search using a trainable codebook $\mathcal{C} = \{\mathbf{c}_i\}_{i=1}^{K}$. The embeddings of the tokens are then fed into the decoder to reconstruct the input $\hat{\mathbf{x}} = f_{\mathcal{G}}(\mathbf{c}_z)$. The VQVAE is trained with the following loss:

$$\mathcal{L}_{\text{vqvae}} = \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}\|_1}_{\mathcal{L}_{\text{rec}}} + \underbrace{\|\text{sg}[f_{\mathcal{E}}(\mathbf{x})] - \mathbf{c}_z\|_2^2}_{\mathcal{L}_{\text{codebook}}} + \underbrace{\beta\|\text{sg}\left[\mathbf{c}_z\right] - f_{\mathcal{E}}(\mathbf{x})\|_2^2}_{\mathcal{L}_{\text{commit}}},$$

where sg is a stop-gradient operation and we use $\beta = 0.25$ following the VQGAN paper [81]. We optimize $\mathcal{L}_{\text{codebook}}$ using an EMA update and circumvent the non-differentiable quantization step $\mathbf{q}$ with a straight-through gradient estimator [21]. VQGAN additionally adopts a perceptual loss [243, 244] and a discriminator $f_{\mathcal{D}}$ to improve the reconstruction quality. Similar to other GAN-based video generation models [10, 11], we use two types of discriminators in our model

- a spatial discriminator $f_{\mathcal{D}_s}$ that takes in random reconstructed frames $\hat{\mathbf{x}}_i \in \mathbb{R}^{H \times W \times 3}$ to encourage frame quality and a temporal discriminator $f_{\mathcal{D}_t}$ that takes in the entire reconstruction $\hat{\mathbf{x}} \in \mathbb{R}^{T \times H \times W \times 3}$ to penalize implausible motions:

$$\mathcal{L}_{\text{disc}} = \log f_{\mathcal{D}_{s/t}}(\mathbf{x}) + \log(1 - f_{\mathcal{D}_{s/t}}(\hat{\mathbf{x}}))$$

We also use feature matching losses [245, 246] to stabilize the GAN training:

$$\mathcal{L}_{\text{match}} = \sum_i p_i \left\| f^{(i)}_{\mathcal{D}_{s/t}/\text{VGG}}(\hat{\mathbf{x}}) - f^{(i)}_{\mathcal{D}_{s/t}/\text{VGG}}(\mathbf{x}) \right\|_1,$$

where $f^{(i)}_{\mathcal{D}_{s/t}/\text{VGG}}$ denotes the i^{th} layer of either a trained VGG network [247] or discriminators with a scaling factor p_i. When using a VGG network, this loss is known as the perceptual loss [244]. p_i is a learned constant for VGG and the reciprocal of the number of elements in the layer for discriminators. Our overall VQGAN training objective is as follows:

$$\min_{f_{\mathcal{E}}, f_{\mathcal{G}}, \mathcal{C}} \left(\max_{f_{\mathcal{D}_s}, f_{\mathcal{D}_t}} (\lambda_{\text{disc}} \mathcal{L}_{\text{disc}}) \right) +$$

$$\min_{f_{\mathcal{E}}, f_{\mathcal{G}}, \mathcal{C}} (\lambda_{\text{match}} \mathcal{L}_{\text{match}} + \lambda_{\text{rec}} \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{codebook}} + \beta \mathcal{L}_{\text{commit}})$$

Directly applying GAN losses to VideoGPT [29] or 3D VQGAN [81] leads to training stability issues.

Autoregressive prior model. After training the video VQGAN, each video can be encoded into its discrete representation $\mathbf{z} = \mathbf{q}(f_{\mathcal{E}}(\mathbf{x}))$. Following VQGAN [81], we unroll these tokens into a 1D sequence using the row-major order frame by frame. We then train a transformer $f_{\mathcal{T}}$ to model

the prior categorical distribution of $\mathbf{z}$ in the dataset autoregressively:

$$p(\mathbf{z}) = p(\mathbf{z}_0) \prod_{i=0}^{t \times h \times w - 1} p(\mathbf{z}_{i+1} | \mathbf{z}_{0:i}),$$

where $p(\mathbf{z}_{i+1}|\mathbf{z}_{0:i}) = f_{\mathcal{T}}(\mathbf{z}_{0:i})$ and $\mathbf{z}_0$ is given as the start of sequence token. We train the transformer to minimize the negative log-likelihood over training samples:

$$\mathcal{L}_{\text{transformer}} = \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z}_{\text{data}})}[-\log p(\mathbf{z})]$$

At inference time, we randomly sample video tokens from the predicted categorical distribution $p(\mathbf{z}_{i+1}|\mathbf{z}_{0:i})$ in sequence and feed them into the decoder to generate the videos $\hat{\mathbf{x}} = f_{\mathcal{G}}(\hat{\mathbf{c}}_z)$. To synthesize videos longer than the training length, we generalize sliding attention window for our use [234] . A similar idea has been used in 2D to generate images of higher resolution [81]. We denote the j^{th} temporal slice of $\mathbf{z}$ to be $\mathbf{z}^{(j)} \in \mathbb{Z}^{h \times w}$, where $0 \leq j \leq t-1$. For instance, to generate $\mathbf{z}^{(t)}$ that is beyond the training length, we condition on the $t-1$ slices before it to match the transformer sequence length $p(\mathbf{z}^{(t)}|\mathbf{z}^{(1:t-1)}) = f_{\mathcal{T}}(\mathbf{z}^{(1:t-1)})$. However, as shown in Figure 3.2, when paired with sliding window attention, the vanilla video VQGAN and transformer cannot generate longer videos without quality degradation. Next, we discuss the reason and a simple yet effective fix.

3.2.2.2 Time-Agnostic VQGAN

When the Markov property holds, a transformer with a sliding window can generate arbitrarily long sequences as demonstrated in long article generation [234]. However, a crucial

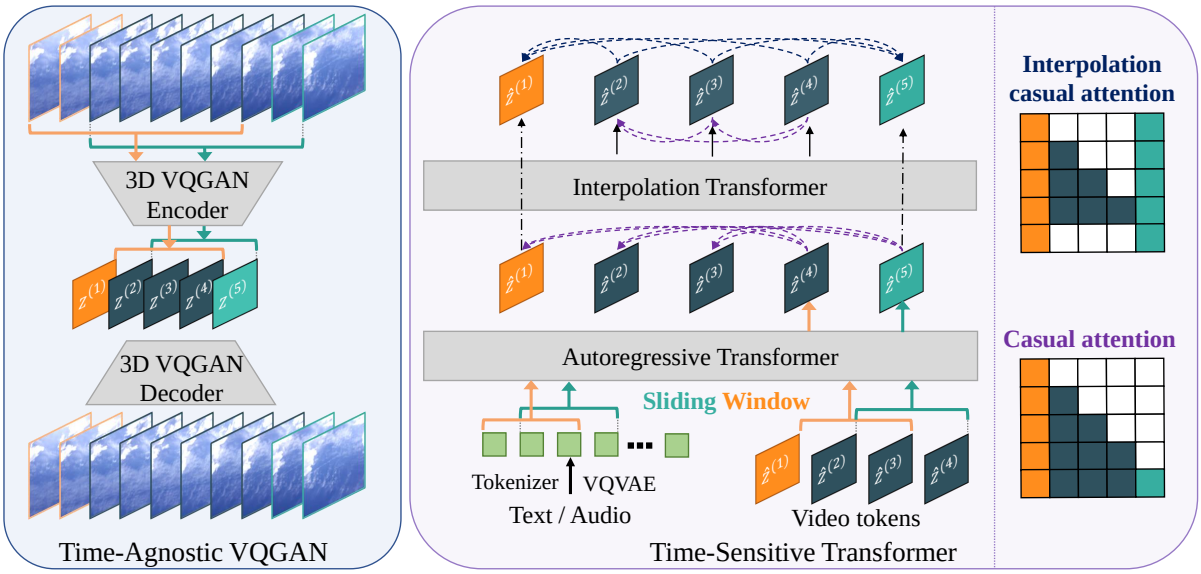


Figure 3.3: **Overview of the proposed framework.** Our model contains two modules: time-agnostic VQGAN and time-sensitive transformer. The former compresses the videos both temporally and spatially into discrete tokens without injecting any dependence on the relative temporal position, which allows the usage of a sliding window during inference for longer video generation. The latter uses a hierarchical transformer for capturing longer temporal dependence.

premise that has been overlooked is that the transformer needs to see sequences that start with tokens similar to $\mathbf{z}^{(1:t-1)}$ during its training to predict token $\mathbf{z}^t$. This premise breaks down in VQGAN.

Different from natural language modeling where the tokens come from realistic data, VQGAN tokens are produced by an encoder $f_{\mathcal{E}}$ which by default adopts zero paddings for the desired output shape. When a short video clip is encoded, the zero paddings in the temporal dimension also get encoded and affect the output tokens, causing an unbalanced effects to tokens at different temporal position [239–242]. The tokens closer to the temporal boundary will be affected more significantly. As a result, for real data to match $\mathbf{z}^{(1:t-1)}$, they have to contain these zero-frames, which is not the case in practice. Therefore, removing those paddings in the temporal dimension is crucial for making the encoder *time-agnostic* and enabling sliding window.

After removing all the padding, one needs to pad real frames to both ends of the input

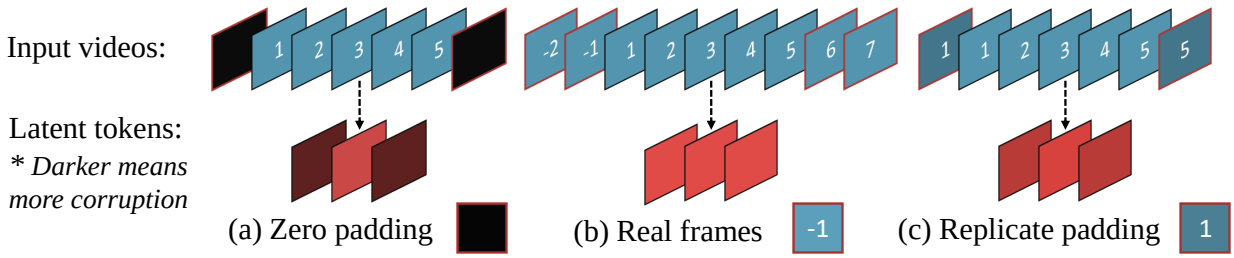


Figure 3.4: **Illustration of the temporal effects induced by paddings**. Real frame padding makes the encoder temporally shift-equivariant[3] but introduces extra computations. Replicate padding makes decent approximation to the real frames while bringing no computational overhead.

videos to obtain the desired output size [38, 248]. The number of needed real frames can be as large as $\mathcal{O}(Ld)$, where L is the number of layers and d is the compression rate in the temporal direction of the video. Note that the zeros are also padded in the intermediate layers when applied, and importantly, they need not be computed. But, if we want to pad with realistic values, the input needs to be padded long enough to cover the entire receptive field, and all these extra values in the intermediate layers need to be computed. Although padding with real frames makes the encoder fully time agnostic, it can be expensive with a large compression rate or a deep network. In addition, frames near both ends of the videos need to be discarded for not having enough real frames to pad, and consequently some short videos may be completely dropped.

Instead of padding with real values, we propose to approximate real frames with the values close to them. An assumption, which is often referred to as the "boring videos" assumption in the previous literature [249], is to assume that the videos are frozen beyond the given length. Following the assumption, the last boundary slices are always used for padding without computation. More importantly, this can be readily implemented by the replicate padding mode using a standard machine learning library. An illustration of the effects induced by different paddings can be found in Figure 3.4. Other reasonable assumptions can also be adopted and may corre-

[3]The expressions *temporally shift-equivariant* and *time-agnostic* will be used interchangeably hereinafter.

spond to different padding modes. For instance, the reflected padding mode can be used if videos are assumed to play in reverse beyond their length, which has more realistic motions than the frozen frames. We find that the replicate padding alone resolves the time-dependence issue well in practice and inherits the merit of zero paddings in terms of the computational cost. Therefore, in the following experiments, we use the replicate paddings and no real frames padded.

3.2.2.3 Time-Sensitive Transformer

The time-agnostic property of the VQGAN makes it feasible to generate long videos using a transformer with sliding window attention. However, long video generation does need temporal information! To maintain a consistent theme running from the beginning of the video to the end requires the capacity to model long-range dependence. And besides, the spirit of a long video is in its underlying story, which requires both predicting the motions in the next few frames and the ability to plan on how the events in the video proceed. This section discusses the time-sensitive transformer for improved long video generation.

Due to the probabilistic nature of transformers, errors can be introduced when sampling from a categorical distribution, which then accumulate over time. One common strategy to improve the long-term capacity is to use a hierarchical model [250, 251] to reduce the chance of drifting away from the target theme. Specifically, we propose to condition one interpolation transformer on the tokens generated by the other autoregressive transformer that outputs more sparsely sampled tokens. The autoregressive transformer is trained in a standard way but on the sampled frames with larger intervals. For the interpolation transformer, it fills in the missing frames between any two adjacent frames generated by the autoregressive transformer. To do so,

we propose interpolation attention as shown in Figure 3.3, where the predicted tokens attend to both the previously generated tokens in a causal way and the tokens from the sparsely sampled frames at both ends. We consider an autoregressive transformer that generates $4\times$ more sparse video tokens. Generalizing this model to even more extreme cases such as video generation based on key-frames would be an interesting future work.

Another simple yet effective way to improve the long period coherence is to provide the underlying "storyline" of the desired video directly. The VQVAE framework has been shown to be an effective way to do conditional image generation [81,252]. We hence consider several kinds of conditional information that provide additional temporal information such as audio [253], text [254], and so on. To utilize conditional information for long video generation, we use either a tokenizer or an additional VQVAE to discretize the audio or text and prepend the obtained tokens to the video tokens and remove the start of the sequence token $\mathbf{z}_0$. At inference time, we extend the sliding window by simultaneously applying it to the conditioned tokens and video tokens.

3.2.3 Experiments

In this section, we evaluate the proposed method on several benchmark datasets for video generation with an emphasis on long video generation.

3.2.3.1 Experimental Setups

Datasets and evaluation. We show results on UCF-101 [235], Sky Time-lapse [236], and Taichi-HD [237] for unconditional or class-conditioned video generation with 128×128 resolution

Table 3.1: Quantitative results of standard video generation on different datasets. We report FVD and KVD on the Taichi-HD and Sky Time-lapse datasets, IS and FVD on the UCF-101 dataset, SSIM and PSNR at the 45^{th} frame on the AudioSet-Drum dataset. * denotes training on the entire UCF-101 dataset instead of the train split. The class column indicates whether the class labels are used as conditional information.

(a) Sky Time-lapse

Method	FVD (↓)	KVD (↓)
MoCoGAN-HD	$183.6_{\pm 5.2}$	$13.9_{\pm 0.7}$
DIGAN	$\mathbf{114.6}_{\pm 4.9}$	$6.8_{\pm 0.5}$
TATS-base	$132.56_{\pm 2.6}$	$\mathbf{5.7}_{\pm 0.3}$

(b) TaiChi-HD

Method	FVD (↓)	KVD (↓)
MoCoGAN-HD	$144.7_{\pm 6.0}$	$25.4_{\pm 1.9}$
DIGAN	$128.1_{\pm 4.9}$	$20.6_{\pm 1.1}$
TATS-base	$\mathbf{94.60}_{\pm 2.7}$	$\mathbf{9.8}_{\pm 1.0}$

(c) AudioSet-Drum

Method	SSIM (↑)	PSNR (↑)
SVG-LP	$0.510_{\pm 0.008}$	$13.5_{\pm 0.1}$
Vougioukas *et al.*	$0.896_{\pm 0.015}$	$23.3_{\pm 0.3}$
Sound2Sight	$0.947_{\pm 0.007}$	$27.0_{\pm 0.3}$
CCVS	$0.945_{\pm 0.008}$	$27.3_{\pm 0.5}$
TATS-base	$\mathbf{0.964}_{\pm 0.005}$	$\mathbf{27.7}_{\pm 0.4}$

(d) UCF-101

Method	Class	IS (↑)	FVD (↓)
VGAN	✓	$8.31_{\pm .09}$	-
TGAN	✗	$11.85_{\pm .07}$	-
TGAN	✓	$15.83_{\pm .18}$	-
MoCoGAN	✓	$12.42_{\pm .07}$	-
ProgressiveVGAN	✓	$14.56_{\pm .05}$	-
LDVD-GAN	✗	$22.91_{\pm .19}$	-
VideoGPT	✗	$24.69_{\pm .30}$	-
TGANv2	✓	$28.87_{\pm .67}$	$1209_{\pm 28}$
DVD-GAN*	✓	$27.38_{\pm .53}$	-
MoCoGAN-HD*	✗	32.36	838
DIGAN	✗	$29.71_{\pm .53}$	$655_{\pm 22}$
DIGAN*	✗	$32.70_{\pm .35}$	$577_{\pm 21}$
CCVS*+Real frame	✗	$41.37_{\pm .39}$	$389_{\pm 14}$
CCVS*+StyleGAN	✗	$24.47_{\pm .13}$	$386_{\pm 15}$
Real data	-	90.52	-
TATS-base	✗	$57.63_{\pm .24}$	$420_{\pm 18}$
TATS-base	✓	$\mathbf{79.28}_{\pm .38}$	$\mathbf{332}_{\pm 18}$

following [37], AudioSet-Drum [238] for audio-conditioned video generation with 64×64 resolution following [28], and MUGEN [255] with 256×256 resolution for text-conditioned video generation. We follow the previous methods [14, 37] to use Fréchet Video Distance (FVD) [256] and Kernel Video Distance (KVD) [256] as the evaluation metrics on UCF-101, Sky Time-lapse, and Taichi-HD datasets. In addition, we follow the methods evaluated on UCF-101 [11, 28] to report the Inception Score (IS) [12] calculated by a trained C3D model [257]. For audio-conditioned generation evaluation, we measure the SSIM and PSNR at the 45^{th} frame which is the longest videos considered in the previous methods [28, 253].

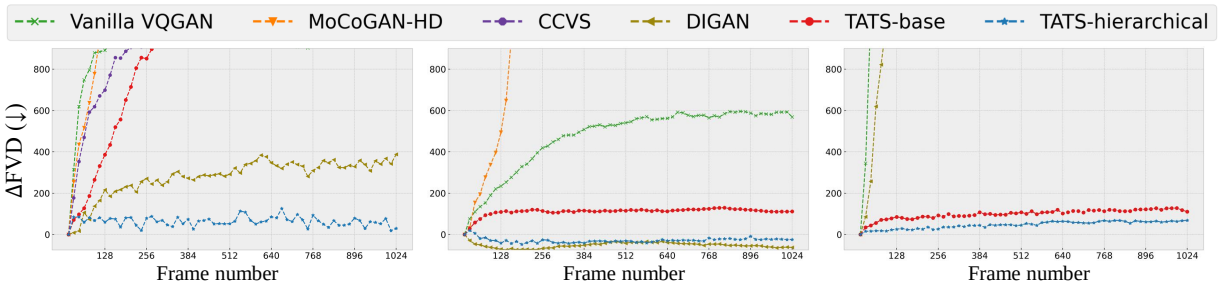
Training details. To compare our methods with previous ones, we train our VQGAN on videos with 16 frames. We adopt a compression rate $d = 4$ in temporal dimension and $d = 8$ in spatial

dimensions. For transformer models, we train a decoder-only transformer with size between GPT-1 [258] and GPT-2 [259] for a consideration of computational cost. We refer to our model with a single autoregressive transformer as **TATS-base** and the proposed hierarchical transformer as **TATS-hierarchical**. For audio-conditioned model, we train another VQGAN to compress the Short-Time Fourier Transform (STFT) data into a discrete space. For text-conditioned model, we use a BPE tokenizer pretrained by CLIP [260].

3.2.3.2 Quantitative Evaluation on Short Video Generation

In this section, we demonstrate the effectiveness of our TATS-base model under a standard short video generation setting, where only 16 frames are generated for each video. The quantitative results are shown in Table 3.1.

Our model achieves state-of-art FVD and KVD on the UCF-101 and Taichi-HD datasets, and state-of-art KVD on the Sky Time-lapse dataset for unconditional video generation, and improved the quality on the AudioSet-Drum for audio-conditioned video generation. For instance, TATS-base improves on IS by 39.3% over the prior methods and 76.2% over the method that does not rely on real frames. On the UCF-101 dataset, we also explore generation with class labels as conditional information. Following the previous method [9], during inference, we sample labels from the prior distribution as the input to the generation. We find that conditioning on the labels significantly eases the transformer modeling task and boosts the generation quality, improving IS from 57.63 to 79.28, which is close to the upper bound shown in the "Real data" row [261, 262]. This has been extensively observed in image generation [81, 263] but not quite yet revealed in the video generation. TATS-base significantly advances the IS over the previous



(a) UCF-101 (b) Sky Time-lapse (c) Taichi-HD

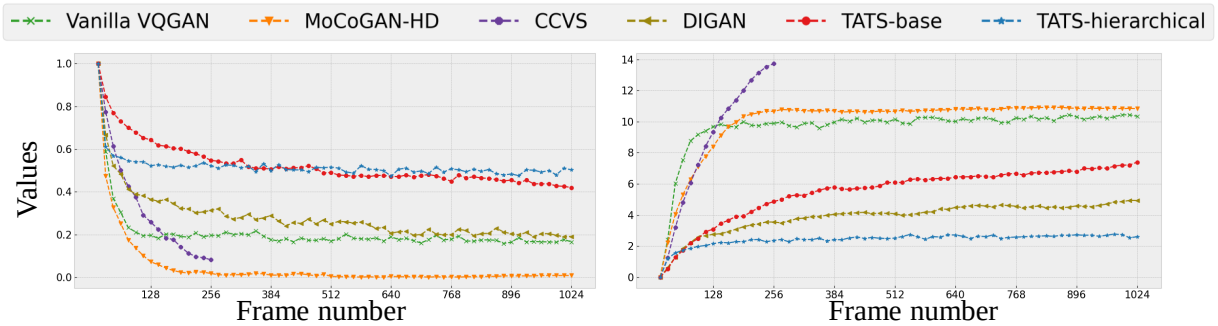
Figure 3.5: **Quality degradation.** FVD changes of non-overlapping 16-frame clips from the long videos generated by models trained on the UCF-101, Sky Time-lapse, and Taichi-HD datasets. The lower value indicates slower degradation.

methods, demonstrating its power in modeling diverse video datasets.

3.2.3.3 Quantitative Evaluation on Long Video Generation

Quantitatively evaluating long video generation results has been under explored. In this section, we propose several metrics by generalizing existing metrics to evaluate the crucial aspects of the long videos. We generate 512 videos with 1024 frames on each of the Sky Time-lapse, Taichi-HD, and UCF-101 datasets. We compare our proposed methods, TATS-base and TATS-hierarchical, with the baseline Vanilla VQGAN and the state-of-the-art models including MoCoGAN-HD [14], DIGAN [37], and CCVS [28] by unrolling RNN states, directly sampling, and sliding attention window using their official model checkpoints.

Quality. We measure video quality with respect to the duration by evaluating every 16 frames extracted side-by-side from the generated videos. Ideally, every set of 16-frame videos should come from the same distribution as the training set. Therefore, we report the FVD changes of these generated clips compared with the first generated 16 frames in Figure 3.5, to measure the degradation of the video quality. The figure shows that our TATS-base model successfully delays the quality degradation compared with the vanilla VQGAN baseline, MoCoGAN-HD, and CCVS



(a) Class Coherence Score (↑) (b) Inception Coherence Score (↓)

Figure 3.6: **Video coherency.** CCS and ICS values of every 16-frame clip extracted from long videos generated by different models trained on the UCF-101 dataset.

models. In addition, the TATS-hierarchical model further improves the long-term quality of the TATS-base model. The concurrent work DIGAN [37] also claims the ability of extrapolation, while we show that the generation still degrades severely after certain number frames on the UCF-101 and Taichi-HD datasets. We conjecture the unusual decrease of the FVD w.r.t. the duration of DIGAN and TATS-hierarchical on Sky Time-lapse can be explained by that the I3D model [249] used to calculate FVD is trained on Kinetics-400 dataset, and the sky videos can be outliers of the training data and lead to weak activation in the logit layers and therefore such unusual behaviors.

Coherence. The generated long videos should follow a consistent topical theme. We evaluate the coherence of the videos on the UCF-101 dataset since it has multiple themes (classes). We expect the generated long videos to be classified as the same class all the time. We adopt the same trained C3D model for IS calculated [12], and propose two metrics, Class Coherence Score (CCS) and Inception Coherence Score (ICS) at time step t, measuring the theme similarity between the

non-overlapped 16 frames w.r.t. the first 16 frames, defined as below:

$$\text{CCS}_t = \sum_i \frac{\mathbf{1}(\arg\max p_{C3D}(y|x_i^{(0:15)}), \arg\max p_{C3D}(y|x_i^{(t:t+15)})}{N}$$

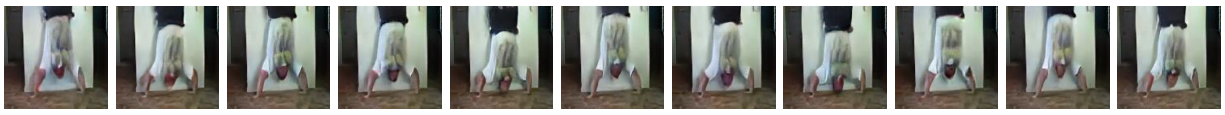
$$\text{ICS}_t = \sum_i p_{C3D}(y|x_i^{(t:t+15)}) \log \frac{p_{C3D}(y|x_i^{(t:t+15)})}{p_{C3D}(y|x_i^{(0:15)})}$$

The ICS captures class shift more accurately than the CCS, which only looks at the most probable class. On the other hand, CCS is more intuitive and allows us to define single metrics such as the area under the curve. CCS also doesn't have the asymmetry issue (unlike KL divergence used in ICS). We show the scores of TATS models and several baselines in Figure 3.6. TATS-base achieves decent coherence as opposite to its quality degradation shown previously. Such difference can be explained by its failure on a small portion of generated videos, which dominates the FVD. This also shows that CCS and ICS measure coherence on individual videos that are complementary to FVD changes. Furthermore, TATS-hierarchical outperforms the baselines on both metrics. For example, more than half of the videos are still classified consistently in the last 16 frames of the entire 1024 frames.

3.2.3.4 Qualitative Evaluation on Long Video Generation

This section shows qualitative results of videos with 1024 frames and discusses their common properties. 1024 frames per video approach the upper bound in the training data, which means the generated videos are probably different from the training videos, at least in duration.

Recurrent actions. We find that some video themes contain repeated events such as the *Handstand Push-Up* classes in the UCF-101 dataset and videos in the Taichi-HD dataset. As shown in



(a) UCF-101: Handstand Push-Up

(b) Taichi-HD

Figure 3.7: **Videos with recurrent events.** Every 100^{th} frame is extracted from the generated videos with 1024 frames on the UCF-101 and Taichi-HD datasets.

Figure 3.7, our model generalizes to long video generation by producing realistic and recurrent actions. However, we find that all the 1024 frames in these videos are unique, which shows that our model is not simply copying the short loops.

Videos	LPIPS metric	Color Similarity
Real	0.1839 ± 0.0683	0.7330 ± 0.2401
Fake	0.3461 ± 0.1184	0.0797 ± 0.1445

Table 3.2: LPIPS and color histogram correlation between the first and the last frames on the sky videos.

Smooth transitions. Videos with the same theme often share visual features such as scenes and motions. We show that with enough training data available for a single theme, our model learns to "stitch" the long videos through generating smooth transitions between different videos. For example in Figure 3.8, we show that our model generates a long sky video containing different weather and timing while the transitions between these conditions are still natural and realistic. To show that this is not the case in the training data, we compute the LPIPS score [244] and color histrogram correlation between the 1^{st} and the 1024^{th} frames and report the mean and standard deviation on the 500 generated and 216 real long videos in Table 3.2. It shows that such transitions are much more prominent on the generated videos than the training videos. A separation of content and motion latent features [10, 14, 37] may better leverage such transitions, which we leave as future work.

Meaningful synthesis. By conditioning on the temporal information, we can achieve more controllable generation, which allows us to directly create or modify videos based our own will. For

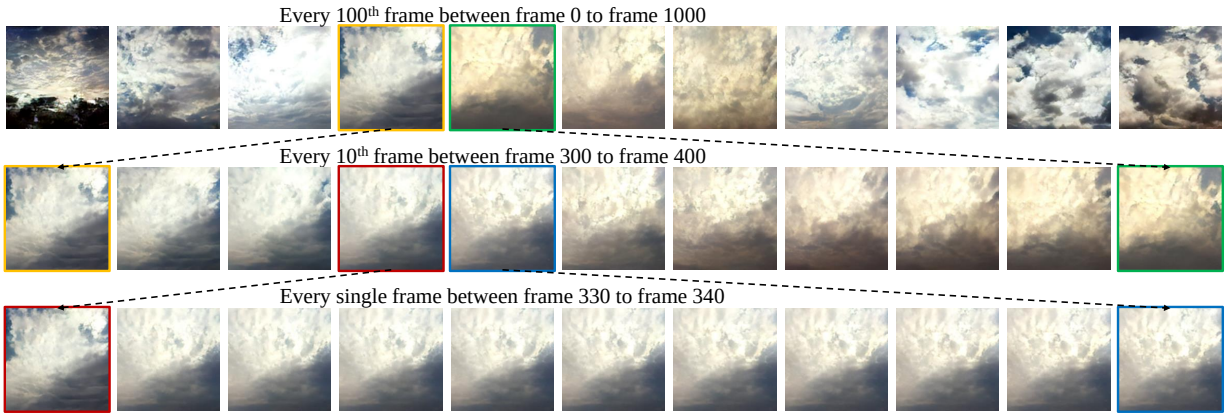
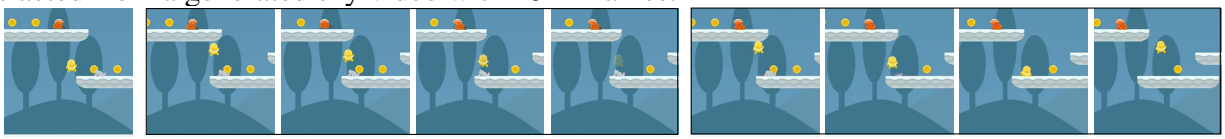


Figure 3.8: **Videos with homomorphisms.** Every 100^{th}, 10^{th}, and consecutive frames are extracted from a generated sky video with 1024 frames.



Initial frame

(a) Ground truth video: Mugen jumps to the right over a mouse and collects a coin and **killed by** a mouse.

(b) Modified video: Mugen jumps to the right over a mouse and collects a coin and **kills** a mouse.

Figure 3.9: **Videos with meanings.** Video manipulation by modifying the texts.

example, in Figure 3.9, we show that it is possible to manipulate the videos by changing the underlying storyline - by replacing "killed by" with "kills" we completely change the destiny of Mugen!

3.2.4 Conclusion

We propose TATS, a time-agnostic VQGAN and time-sensitive transformer model, that is only trained on clips with tens of frames and can generate thousands of frames using a sliding window during inference time. Our model generates meaningful videos when conditioned on text and audio. our paper is a small step but we hope it can encourage future works on more interesting forms of video synthesis with a realistic number of frames, perhaps even movies.

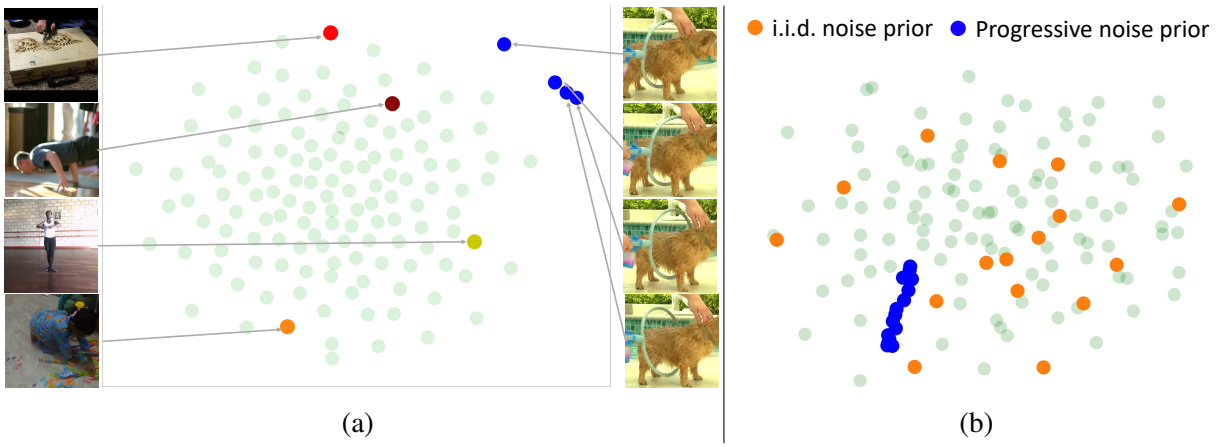


Figure 3.10: **Visualizing the noise map correlations**. (a) visualizes the t-SNE plot of the noise maps corresponding to input frames randomly sampled from videos. These noise maps are obtained by running a diffusion ODE [40, 264] on the input frames using a trained text-to-image model, but in the opposite direction of image synthesis ($\sigma : 0 \rightarrow \sigma_{\text{max}}$). The green dots in the background denote the reference noise maps sampled from an i.i.d. Gaussian distribution. The red dots and yellow dots are noise maps corresponding to input frames coming from different videos. We found they are spread out and share no correlation. On the other hand, the noise maps corresponding to the frames coming from the same video (shown in blue dots) are clustered together. (b) Using an i.i.d. noise model (orange dots) for finetuning text-to-image models for video synthesis is not ideal since temporal correlations between frames are not modeled. To remedy this, we propose a progressive noise model in which the correlation between different noise maps is injected along the temporal axis. Our progressive noise model (blue dots) aptly models the correlations present in the video noise maps.

3.3 Video Generation with Diffusion Models

3.3.1 Introduction

Large-scale diffusion-based text-to-image models [41, 46, 47] have demonstrated impressive capabilities in turning complex text descriptions into photorealistic images. They can generate images with novel concepts unseen during training. Sophisticated image editing and processing tasks can easily be accomplished through guidance control and embedding techniques. Due to the immense success in several applications [163, 182, 265], these models are established as powerful image synthesis tools for content generation. As image synthesis is largely democ-

ratized with the success of these text-to-image models, it is natural to ask whether we can repeat the same success in video synthesis with large-scale diffusion-based text-to-video models.

Multiple attempts have been made to build large-scale video diffusion models. Ho et al. [62] proposed a UNet-based architecture for the video synthesis task that is trained using joint image-video denoising losses. Imagen video [63] extends the cascaded text-to-image generation architecture of Imagen [47] for video generation. In both works, the authors directly train a video generation model from scratch. While these approaches achieve great success and produce high-quality videos, they are inherently expensive to train, requiring hundreds of high-end GPUs or TPUs and several weeks of training. After all, video generators not only need to learn to form individual images but should also learn to synthesize coherent temporal dynamics, which makes the video generation task much more challenging. While the formation of individual frames is a shared component in an image and video synthesis, these works disregard the existence of powerful pretrained text-to-image diffusion models and train their video generators from scratch.

We explore a different avenue for building large-scale text-to-video diffusion models by starting with a pretrained text-to-image diffusion model. Our motivation is that most of the components learned for the image synthesis task can effectively be reused for video generation, leading to knowledge transfer and efficient training. A similar idea is adopted by several recent works [51, 53, 61]. Without exception, when finetuning, they naively extend the image diffusion noise prior (i.i.d. noise) used in the text-to-image model to a video diffusion noise prior by adding an extra dimension to the 2D noise map. We argue that this approach is not ideal as it does not utilize the natural correlations in videos that are already learned by the image models. This is illustrated in Figure 3.10, where we visualize the t-SNE plot of noise maps corresponding to different input frames as obtained from a pretrained text-to-image diffusion model. The noise

maps corresponding to different frames coming from the same video (blue dots in Figure 3.10a are clustered together, exhibiting a high degree of correlation. The use of i.i.d. noise prior does not model this correlation, which would impede the finetuning process. Our careful analysis of the video diffusion noise prior leads us to a noise prior that is better tailored for finetuning an image synthesis model to the video generation task. As illustrated in Figure 3.10b, our proposed noise prior (shown in blue dots) aptly captures the correlations in noise maps corresponding to video frames.

We then proceed to build a large-scale diffusion-based text-to-video model. We leverage several design choices from the prior works, including the use of temporal attention [62], joint image-video finetuning [62], a cascaded generation architecture [63], and an ensemble of expert denoisers [41]. Together with these techniques and the proposed video noise prior, our model establishes a new state-of-the-art for video generation outperforming competing methods on several benchmark datasets. Our model can achieve high-quality zero-shot video synthesis capability with SOTA photorealism and temporal consistency.

In short, our work makes the following key contributions.

1. We propose a video diffusion noise tailored for finetuning text-to-image diffusion models for text-to-video.

2. We conduct extensive experimental validation and verify the effectiveness of the proposed noise prior.

3. We build a large-scale text-to-video diffusion model by finetuning a pretrained eDiff-I model with our noise prior and achieve state-of-the-art results on several benchmarks.

3.3.2 Preliminaries

Diffusion models generate data by iteratively denoising samples drawn from a noise distribution. In the case of text-to-video models, text embeddings obtained from a pre-trained text encoder are used as additional inputs in the denoising process. Formally, let $D(\mathbf{x}, \mathbf{e}, \sigma)$ denote a denoising network that operates on the noisy input video $\mathbf{x} \in \mathbb{R}^{b \times n_s \times 3 \times h \times w}$ where $\mathbf{e}$ is the text embedding, and σ is the noise level. Here n_s is the sequence length of the input video, b is the batch size, and $h \times w$ is the spatial resolution. The model D is trained to denoise the input $\mathbf{x}$.

Training We follow the EDM formulation of Karras et al. [266] to optimize the denoiser D using the following objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}_{\text{clean}}, \mathbf{e}), p(\epsilon), p(\sigma)} \left[\lambda(\sigma) \| D(\mathbf{x}_{\text{noise}}; \mathbf{e}, \sigma) - \mathbf{x}_{\text{clean}} \|_2^2 \right] \tag{3.1}$$

$$\text{where} \quad \mathbf{x}_{\text{noise}} = \mathbf{x}_{\text{clean}} + \sigma\epsilon$$

Here, $\mathbf{x}_{\text{noise}}$ is the noisy sample obtained by corrupting the clean video $\mathbf{x}$ with noise $\sigma\epsilon$, where $p(\epsilon) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and σ is a scalar for the noise level drawn from $p(\sigma)$. The loss weight, $\lambda(\sigma)$, is a function of σ given by $\lambda(\sigma) = (\sigma^2 + \sigma_{\text{data}}^2)/(\sigma \cdot \sigma_{\text{data}})^2$. Eq. (3.1) is a simple denoising objective in which the denoiser D is trained to estimate the clean video $\mathbf{x}_{\text{clean}}$ from the noisy input $\mathbf{x}_{\text{noise}}$. Following EDM, we use a log-normal distribution for σ i.e., $\ln(p(\sigma)) = \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$ with $P_{\text{mean}} = -1.2$ and $P_{\text{std}} = 1.2$.

To train the denoising model, EDM uses preconditioning terms in its objective function to properly scale the inputs and output of the denoiser model D. More specifically, the denoising model D is written as

Table 3.3: **Cosine similarity of the reversed noise.** The noise maps corresponding to the frames sampled from the same videos have a higher similarity than those sampled from different videos.

	Cosine Similarity		Cosine Similarity
(a) Same video noise	0.206 ± 0.156	(b) Different video noise	0.001 ± 0.009

$$D(\mathbf{x};\mathbf{e},\sigma) := \left(\frac{\sigma_{\text{data}}}{\sigma^*}\right)^2 \mathbf{x} + \frac{\sigma \cdot \sigma_{\text{data}}}{\sigma^*} F_\theta\left(\frac{\mathbf{x}}{\sigma^*}; \mathbf{e}, \frac{\ln(\sigma)}{4}\right)$$

Here, F_θ is a neural network with parameters θ and $\sigma^* = \sqrt{\sigma^2 + \sigma^2_{\text{data}}}$. We use $\sigma_{\text{data}} = 0.5$.

Sampling Once the denoising model is trained, sampling can be performed by solving the following ODE [266]

$$\frac{d\mathbf{x}}{d\sigma} = -\sigma \nabla_{\mathbf{x}} \log p(\mathbf{x}|\mathbf{e},\sigma) = \frac{\mathbf{x} - D(\mathbf{x};\mathbf{e},\sigma)}{\sigma} \tag{3.2}$$

for σ flowing backwards from $\sigma = \sigma_{\text{max}}$ to $\sigma = 0$. The initial value for $\mathbf{x}$ is obtained by sampling from the prior distribution $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \sigma^2_{\text{max}}\mathbf{I})$. Over the recent years, several samplers have been proposed for sampling from the trained diffusion models [39, 40, 267–269]. In this paper, we use DEIS [267] and its stochastic variant [266] for synthesizing samples from our model.

3.3.3 Method

Training text-to-video models is much more challenging than training text-to-image diffusion models due to practical difficulties in collecting billion-scale video datasets and securing enough computational resources. Additionally, generating videos is much more challenging since individual frames need to be both photorealistic and temporally coherent. Prior works

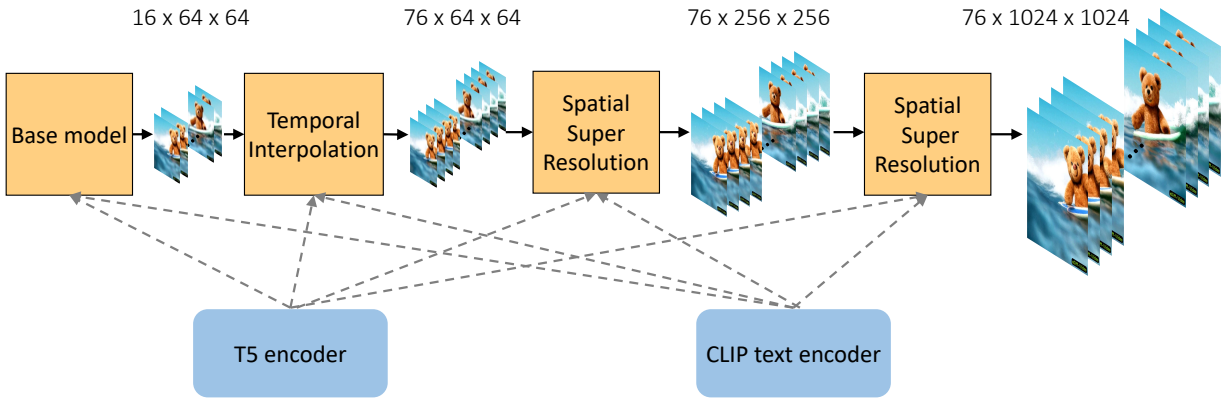


Figure 3.11: **Model architecture.** Our pipeline consists of a cascade of four networks — a base model and three upsampling models. All four models take inputs as the text embeddings obtained from the T5 encoder and the CLIP text encoder. The base model produces 16 video frames of spatial resolution 64×64 with a frameskip of 5. The first upsampling model performs a temporal interpolation, resulting in videos of size $76 \times 64 \times 64$ while the subsequent two super-resolution models perform spatial super-resolution to produce videos of sizes $76 \times 256 \times 256$ and $76 \times 1024 \times 1024$.

leverage large-scale image datasets to mitigate these difficulties by either joint training on the image datasets [62, 63, 102] or finetuning a text-to-image model on the video datasets [61, 120]. Here, we are interested in finetuning text-to-image diffusion models jointly on image and video datasets. We postulate that naively extending the image noise prior to video diffusion is not ideal. We carefully explore the design space of noise priors and propose one that is well suited for our video finetuning task, which leads to significant performance gains.

Correlated noise model An image diffusion model is trained to denoise independent noise from a perturbed image. The noise vector ϵ in the denoising objective Eq.3.1 is sampled from an i.i.d. Gaussian distribution $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. However, after training the image diffusion model and applying it to reverse real frames from a video into the noise space in a per-frame manner, we find that the noise maps corresponding to different frames are highly correlated. This is illustrated in Figure 3.10, where the t-SNE plot of noise maps corresponding to different video frames are plotted. When the input frames come from the same video (shown in blue dots in Figure 3.10a,

noise maps are clustered. The use of i.i.d. sampling (shown in orange dots in Figure 3.10b does not capture these correlations. This is also depicted quantitatively in Table 3.3 where we compute the average pairwise cosine similarity between noise corresponding to (a) same video and (b) different video. (a) is much higher than (b). As a result, the video diffusion model trained with i.i.d. noise is coerced to forget such correlation among the noise between different frames, making it difficult to preserve knowledge from the image diffusion model. Motivated by this observation, we propose to modify the noise process to preserve the correlation between different frames. To this end, we investigate two noising strategies - mixed and progressive noising.

Mixed noise model: Let $\epsilon^1, \epsilon^2, \ldots \epsilon^{n_s}$ denote the noise corresponding to individual video frames i.e., ϵ^i corresponds to the i^{th} element of the noise tensor ϵ. In the mixed noise model, we generate two noise vectors ϵ_{shared} and ϵ_{ind}. ϵ_{shared} is a common noise vector shared among all video frames, while ϵ_{ind} is the individual noise per frame. The linear combination of both these vectors is used as the final noise.

$$\epsilon_{\text{shared}} \sim \mathcal{N}\left(\mathbf{0}, \frac{\alpha^2}{1+\alpha^2}\mathbf{I}\right), \epsilon^i_{\text{ind}} \sim \mathcal{N}\left(\mathbf{0}, \frac{1}{1+\alpha^2}\mathbf{I}\right) \tag{3.3}$$

$$\epsilon^i = \epsilon_{\text{shared}} + \epsilon^i_{\text{ind}}$$

Progressive noise model: In the progressive noise model, the noise for each frame is generated in an autoregressive fashion in which the noise at frame i is generated by perturbing the noise at frame $i-1$. Let ϵ^i_{ind} denote the independent noise generated for frame i. Then,

progressive noising can be formulated as

$$\epsilon^0 \sim \mathcal{N}(\mathbf{0},\mathbf{I}) \quad \epsilon^i_{\text{ind}} \sim \mathcal{N}(\mathbf{0}, \frac{1}{1+\alpha^2}\mathbf{I}) \tag{3.4}$$
$$\epsilon^i = \frac{\alpha}{\sqrt{1+\alpha^2}}\epsilon^{i-1} + \epsilon^i_{\text{ind}}$$

In both these models, α controls how much noise is shared among different video frames. The higher the value of α, the more correlation exists among the noise maps corresponding to different frames. As $\alpha \to \infty$, all frames would have the same noise which results in generating a frozen video. On the other hand, $\alpha = 0$ corresponds to i.i.d. noise.

As shown in Figure 3.10b, the use of progressive noise sampling (blue dots) better models the correlations between different noise maps by obtaining similar clustering patterns to the noise maps of real video frames embedded by a pretrained text-to-image model in Figure 3.10a (blue dots).

Model architecture As visualized in Figure 3.11, our model consists of a cascade of four networks — a base network and three upsampling stacks. The base network generates an output video of dimension $16 \times 64 \times 64$ with a frameskip of 5. It generates the frames $\{1, 6, 11, \ldots 76\}$. The first upsampling network performs a temporal interpolation to produce a video of size $76 \times 64 \times 64$. The second and the third super-resolution network performs spatial upsampling to produce the outputs of sizes $76 \times 256 \times 256$ and $76 \times 1024 \times 1024$. We utilize eDiff-I [41], a state-of-the-art text-to-image diffusion model, to initialize our base and spatial super-resolution models. Similar to prior works [61, 62], we adapt the image-based U-Net model for the video synthesis task by making the following changes: (1) Transforming 2D convolutions to 3D by adding a dimension 1 to temporal axis and (2) Adding temporal attention layers.

Similar to Ho et al. [62], we jointly finetune the model on video and image datasets by concatenating videos and images in the temporal axis and applying our temporal modules only on the video part. Similarly to eDiff-I, our model uses both T5 text embeddings [270] and CLIP text embeddings [260]. We drop each of the embeddings independently at random during training, as in eDiff-I.

3.3.4 Experiments

In this section, we evaluate our proposed strategy of training diffusion models for video synthesis on two sets of experiments. We first comprehensively analyze our proposed noise model on the small-scale UCF-101 dataset. We then scale up our experiments to the challenging large-scale text-to-video synthesis task.

3.3.4.1 Experimental Setups

We conduct ablation experiments in a small-scale unconditional video generation setting and pick the best configuration for our large-scale text-to-video generation run.

Datasets We train our model on the UCF-101 dataset [235] for the small-scale experiments, where we follow the protocol defined in Ho et al. [62] to generate videos of size $16 \times 64 \times 64$. UCF-101 dataset contains $13,320$ videos. We randomly sample frames from these videos to train our image synthesis model. For our large-scale experiments, we use a combination of public and proprietary datasets for text-to-image and text-to-video finetuning. Most of the videos are of 2K resolution with 16:9 aspect ratio. All data was filtered using a preset CLIP and aesthetic scores[4] to ensure high quality. Our final image dataset contains around 1.2 billion text-image pairs and

[4] https://github.com/christophschuhmann/improved-aesthetic-predictor

Table 3.4: Zero-shot text to video generation on UCF-101. Our approach gives significant performance gains compared to the prior baselines both in inception score and FVD metrics.

Method	IS (↑)	FVD (↓)
CogVideo [120] (Chinese)	23.55	751.34
CogVideo [120] (English)	25.27	701.59
Make-A-Video [61]	33.00	367.23
MagicVideo [51]	-	655.00
Video LDM [53]	33.45	550.61
VideoFactory [60]	-	410.00
PYoCo	**47.76**	**355.19**

Table 3.5: Text conditional zero-shot generation on MSRVTT. Our approach with the base config achieves the best results, and using an ensemble further improves the FIDs.

Method	CLIP-FID (↓)	FID (↓)
NUWA [102] (Chinese)	47.68	-
CogVideo [120] (Chinese)	24.78	-
CogVideo [120] (English)	23.59	-
Make-A-Video [61]	13.17	-
MagicVideo [51]	-	36.50
Latent-Shift [59]	15.23	-
PYoCo (Config-A)	10.21	25.39
PYoCo (Config-B)	9.95	24.28
PYoCo (Config-C)	9.91	24.54
PYoCo (Config-D)	**9.73**	**22.14**

22.5 million text-video pairs.

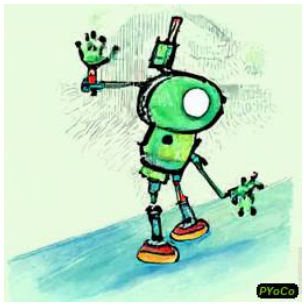
Training details In the unconditional generation experiment on the UCF-101 dataset, to do an ablation study on the model size, we design 3 models where each model has 69M, 112M, and 253M parameters, respectively. As a comparison, the baseline Video Diffusion Model (VDM) [62] contains 1.2B parameters. In the large-scale text-to-video experiment, our base and temporal interpolation models contain 1.08B parameters. Our super-resolution model adapted from the efficient U-Net [47] architecture with temporal convolution layers [61,63] contains 313M parameters.

A cute corgi wearing a red robe holding a sign that says "Merry Christmas". There is a Christmas tree in the background.

An epic tornado attacking above a glowing city at night, the tornado is made of smoke, highly detailed.

Small boat sailing in the ocean, giant Cthulhu monster coming out a dense mist in the background, giant waves attacking.

A golden retriever puppy holding a green sign that says "NVIDIA ROCKS". Background is a classroom.



A cute funny robot dancing, centered, award winning watercolor pen illustration.

A cartoon white wolf is giving puppy-dog eyes, detailed fur, very cute kid's film character.

A lightning striking atop of eiffel tower, dark clouds in the sky, slow motion.

An anime girl looks at the beautiful nature through the window of a moving train, well rendered.

A skull burning while being held up by a skeletal hand.

A huge dinosaur skeleton walking in a golden wheat field on a bright sunny day.

A cute rabbit is eating grass, wildlife photography.

Tomato sauce pouring over fries.

Figure 3.12: Sample generations. *Please check our project website to view the videos.*

Evaluation For the small-scale experiments on UCF-101 dataset, we follow the protocol defined in the prior approaches [14, 38, 62] and report the Inception Score (IS) [12] calculated by a trained C3D model [257] and Fréchet Video Distance (FVD) [256] by a trained I3D model [249]. For the large-scale text-to-video experiments, we perform the zero-shot evaluation of the video generation quality on the UCF-101 and MSR-VTT datasets following Make-A-Video [61]. We carefully discuss the evaluation process below.

UCF-101 experiment We use IS and FVD for evaluation in our small-scale experiments. UCF-101 is a categorical video dataset designed for action recognition. When sampling from the text-to-video model, we devise a set of prompts for each class name to be used as the conditional input. This is necessary as some class names (such as *jump rope*) are not descriptive. We sample 20 videos for each prompt to compute the IS metric. For FVD, we follow the prior work [14, 28] and sample $2,048$ videos for evaluation.

MSR-VTT experiment MSR-VTT [271] test set contains $2,990$ videos as well as $59,794$ captions. All the videos have the same resolution of 320×240. We generate a $76 \times 256 \times 256$ video for each $59,794$ caption and save the videos in an *mp4* format with a high bit rate. To compare with Make-A-Video, we compute FID using a ViT-B/32 model [272]. We also report a more common FID metric computed by an Inception-V3 model. We also examine the idea of ensemble denoiser [41] by finetuning the level-1 experts of each model. We denote Config-A as the configuration of using only baseline models and Config-B to Config-D as incrementally changing super-resolution model, temporal interpolation model, and base model with the corresponding ensemble models.

Table 3.6: Unconditional UCF-101 generation results. Our approach achieves the state-of-the-art inception score and FVD, while having considerably smaller parameter count compared to other diffusion-based approaches such as VDM (1B parameters).

Method	IS (↑)	FVD (↓)
TGAN [9]	$15.83_{\pm.18}$	-
LDVD-GAN [262]	$22.91_{\pm.19}$	-
VideoGPT [29]	$24.69_{\pm.30}$	-
MoCoGAN-HD [14]	32.36	838
DIGAN [37]	$29.71_{\pm.53}$	$655_{\pm22}$
CCVS [28]	$24.47_{\pm.13}$	$386_{\pm15}$
StyleGAN-V [38]	$23.94_{\pm.73}$	-
VDM [62]	$57.00_{\pm.62}$	-
TATS [106]	$57.63_{\pm.73}$	$430_{\pm18}$
PYoCo (112M)	$57.93_{\pm.24}$	$332_{\pm13}$
PYoCo (253M)	$\mathbf{60.01}_{\pm.51}$	$\mathbf{310}_{\pm13}$

3.3.4.2 Main Results

Large-scale text-to-video synthesis We quantitatively compare our method against Make-A-Video [61], NUWA [102], CogVideo [120], and several concurrent works [51, 53, 53, 59, 60]. Table 3.4 shows that our method outperforms all the baselines on the UCF-101 dataset and improves the zero-shot Inception Score from 33.45 to 47.76. In Table 3.5, we show that our baseline model achieves a new state-of-the-art CLIP-FID score [272] of 10.21, while using ensemble models further improves both CLIP-FID and FID scores. In Figure 3.12, we qualitatively visualize the synthesis capability of our approach. Our model achieves high-quality zero-shot video synthesis capability with good photorealism and temporal coherency. We also provide a qualitative comparison with Make-A-Video [61] and Imagen Video [63] in Figure 3.13. We observe that our model is able to produce videos with better details than both approaches, as shown in the animal videos. We also produce better-stylized videos than Imagen Video.

Small-scale unconditional video synthesis

Table 3.7: Quantitative results of different training strategies on UCF-101 dataset.

	IS(↑)	FVD (↓)	FID (↓)
Image Diffusion (ID)	-	-	30.05
Training from scratch	28.25	903.37	124.75
Finetuning from ID	41.25	566.67	56.43
+ Mixed Noise	52.71	**337.40**	**31.57**
+ Progressive Noise	**53.52**	339.67	31.88

We report IS and FVD scores on UCF-101 dataset in Table 3.6 and compare our model with multiple unconditional video generation baselines. Note that using class labels as conditional information could lead to sizeable improvement in IS and FVD scores [106], which we do not consider as the comparison. Our method attains state-of-the-art unconditional video generation quality. Compared with previous diffusion-based unconditional generation model [62], our model is $\sim 10\times$ smaller and has $\sim 14\times$ less training time (75 GPU-days vs. 925 GPU-days).

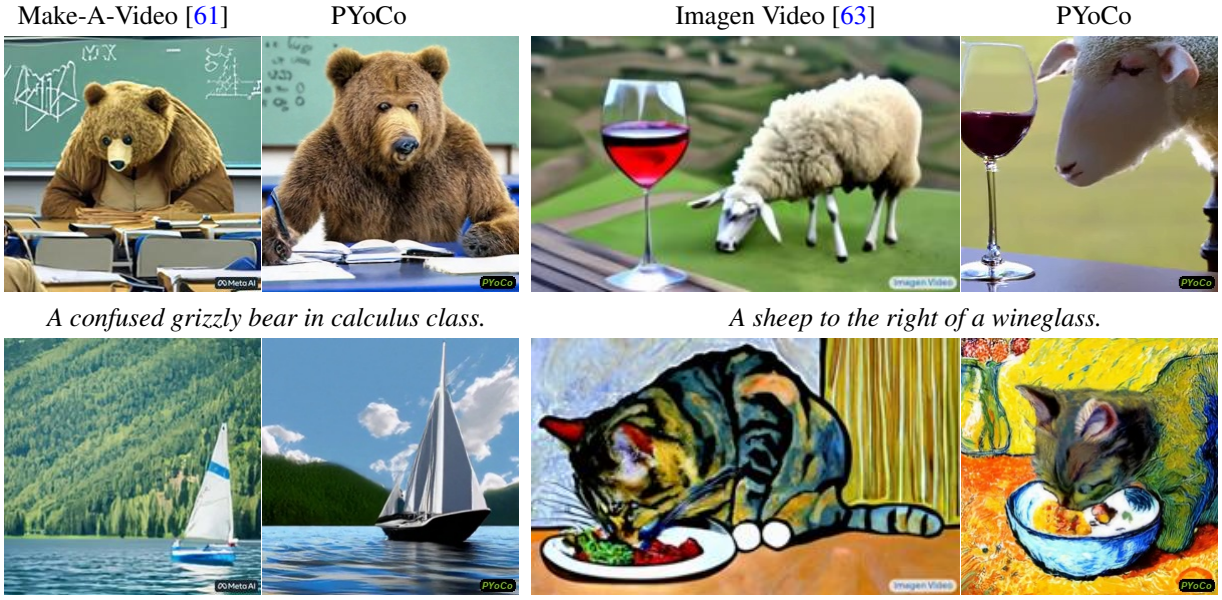


Figure 3.13: Qualitative comparison with baseline approaches. The two panels on the left show the comparison of our approach with Make-A-Video [61], while those on the right show the comparison with Imagen Video [63]. PYoCo achieves better photorealism compared to the two approaches.

3.3.5 Ablation Study

We quantitatively compare several training strategies for video diffusion models. Then, we perform ablation on the correlation ratio in the Equations 3.3 and 3.4, a key hyper-parameter in our approach.

Training strategies We compare training from scratch, a simple finetuning baseline, finetuning with mixed noising, and progressive noising using IS, FVD, and averaged frame FID metrics on the UCF-101 dataset in Table 3.7. We first find that finetuning from an image diffusion model is much more effective than training from scratch. For finetuning from the image model, the correlated noise model produces better video generation quality than the independent noise model. In addition, we notice that the correlated noise better preserves the image quality learned by the pretrained image model and produces a lower frame FID. This is particularly desired in large-scale text-to-video training to fulfill the goal of inheriting the knowledge from the image model missing in the video datasets. Specifically, most videos contain realistic scenes captured by cameras and have infrequent media types like paintings, illustrations, sketches, etc. Moreover, the video data is much smaller in volume, and the scenes are less diverse than image datasets. As shown in Figure 3.12, our model can preserve properties learned from image datasets that are not presented in our video dataset, such as the artistic styles, and generate faithful motion on them.

Correlation ratio The hyperparameter α in the Equations 3.3 and 3.4 controls the correlation between the noise of different frames. A larger α injects more correlation into the noise. The correlation disappears when $\alpha \to 0$, and the mixed and progressive noise models reproduce the vanilla noise model. To find optimal α, we train our UCF-small model (69M parameters) using $\alpha \in \{0, 0.1, 0.2, 0.5, 1, 1, 2, 5, 10, \infty\}$ and report FVD in Figure 3.15. For each α value, we

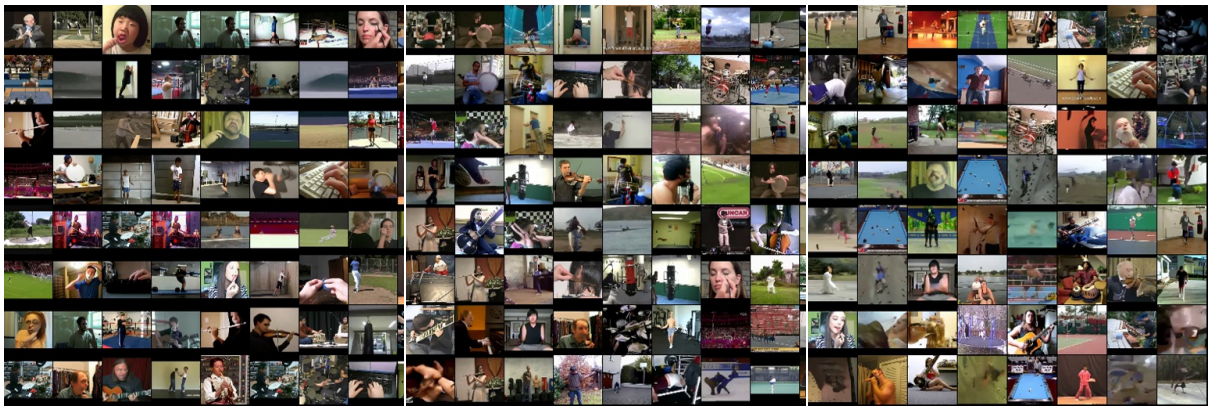


Figure 3.14: **Visual ablation on α.** Small $\alpha = 0$ reduces video quality and diversity and large $\alpha = 10$ yields motion artifacts.

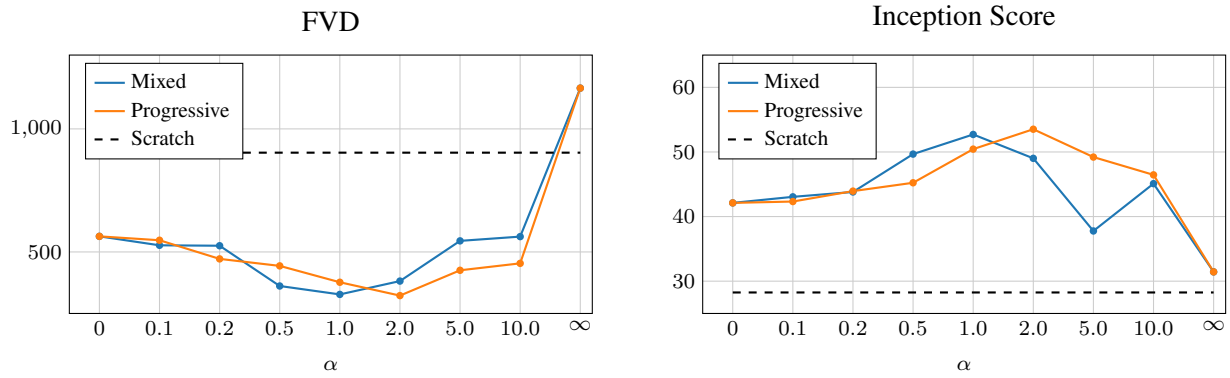


Figure 3.15: **Ablation on hyperparameter α.** Finetuning with temporally correlated prior improves over training from scratch. Using too large or too small value for α leads to inferior results. $\alpha = 1$, $\alpha = 2$ each works the best for mixed and progressive noising, respectively.

repeat the experiment 3 times and report the mean. Note that $\alpha = 0$ indicates finetuning with the independent frame noise, and $\alpha = \infty$ indicates using identical noise maps for all the frames, which produces frozen videos during the inference time. Finetuning an image diffusion model almost consistently outperforms the training-from-scratch baseline with different αs. Using $\alpha = 1$ for mixed noising and $\alpha = 2$ for progressive noising produces similar best results. We also show qualitiative results for models trained with $\alpha = 0, 1, 10$ in Figure 3.14. When α is too small, we notice a degradation in visual quality in the generated video frames and a reduced video diversity. For example, we notice many repeated samples and black borders in almost every video generated

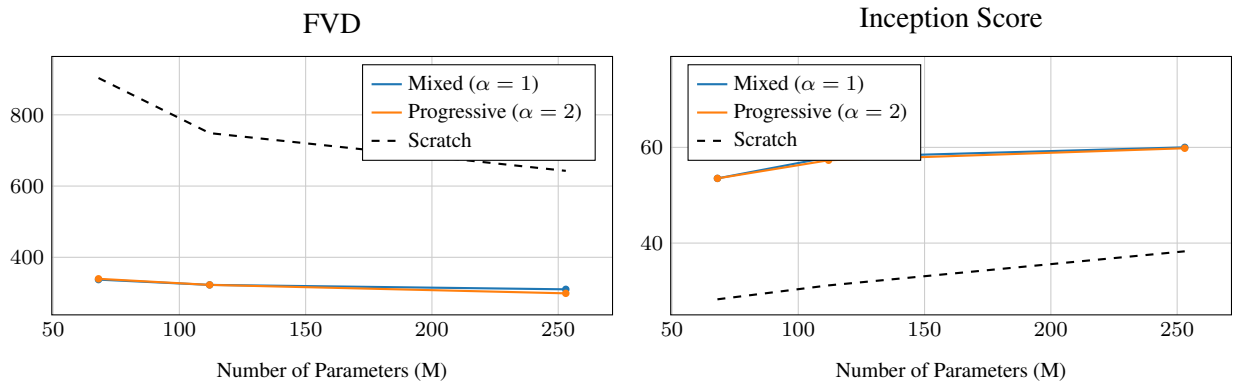


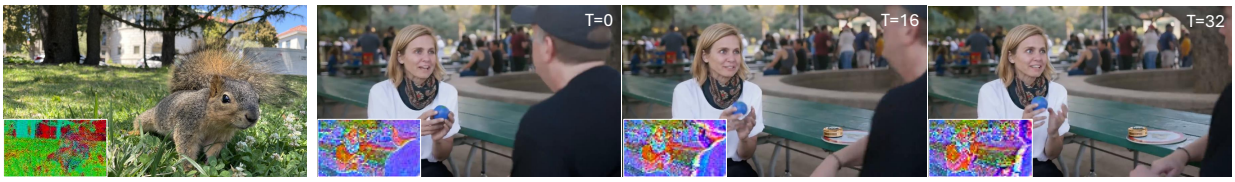
Figure 3.16: **Ablation on model size.** Larger models consistently improve the performance of both finetuning and training from scratch. Finetuning from the image model consistently outperforms training from scratch.

with $\alpha = 0$. On the other hand, when α is too large, the model has difficulty generating proper motions.

Model size We pick the best α for the mixed and progressive noise models and compare them with the model trained from scratch on models with different numbers of parameters, 69M, 112M, and 253M. Figure 3.16 shows that our mixed and progressive models outperform the baseline consistently by a large margin in terms of FVD. Overall, mixed and progressive noising provide similar performance. In our large large-scale experiments, we choose progressive noising with $\alpha = 2$ due to its autoregressive nature.

3.3.6 Conclusion

We proposed a new efficient way of training text-to-video generation models. By observing that the noise maps generating the frames of a video are clustered together, we study mixed and progressive noise priors well-suited for sequential video frame generation. We apply our progressive noise prior to finetuning a state-of-the-art diffusion-based text-to-image model to achieve a state-of-the-art large-scale text-to-video model. The high quality of the generated videos and the



Flux image tokenizer

Cosmos video tokenizer

Figure 3.17: **Redundancy of tokens from existing tokenizers.** We examine the latent space of an image tokenizer from the Flux [279] and Cosmos video tokenizer [280]. We visualize the latent tokens by normalizing the first three channels as the RGB channels in the bottom left. We notice that the overall structure of video tokens resembles the input video. Applying these tokenizers to compress videos will thus create temporal redundancy similar to those in the input video.

state-of-the-art Inception and FID scores demonstrate the strength of our approach.

3.4 Scaling Video Generation with Autoregressive Tokenizers

3.4.1 Introduction

The rise of large-scale generative models like GPTs [234, 273] has propelled AI into an unprecedented era. Video, like language, is another modality with abundant data across the web. It conveys orthogonal knowledge about our physical world and motion dynamics. Training large-scale video generative models has thus sparked broad interest [90, 274–277]. Unlike language, however, video is highly redundant—each frame contains lots of information that is already present in the previous frame, resulting in excessively large representations. Therefore, training video models significantly falls behind the scale of training Large Language Models (LLMs)[5].

Scaling video generative models hinges on developing more compact video representations. Modern Vision-Language Models (VLMs) [97, 99] and video generation models [32, 90] leverage

[5]The Llama-3 model is trained on 15 trillion tokens [278], roughly equivalent to 7 *billion* articles, or only 15 *million* 10-second HD videos.

video tokenizers [111] to compress raw pixels into tokens to reduce the training and inference costs. At training time, the tokenizer encoder transforms videos into latent tokens for a generative model to train on. At inference time, the tokenizer decoder reconstructs videos from the generated tokens.

State-of-the-art video generation models [32, 90, 276] all adopt convolutional tokenizers for their efficiency on high-resolution videos, even though transformers are ubiquitous in visual recognition [260] and generation [281] for their scalability. In this paper, we show that the local inductive bias of convolution leads to redundant video representations (Figure 3.17), motivating us to train transformer-based tokenizers for their minimal inductive bias further compression. To enable high-resolution video tokenization and efficient ablation studies, we train transformer tokenizers in the latent space [85, 282] of a pre-trained continuous convolutional tokenizer. We find that transformer architecture alone does, however, not ensure scalability for tokenizers due to the information bottleneck.

A simple yet essential property of videos is their causal nature—the current frame only depends on the past frames, not future ones. While some recent works [111, 276, 283] incorporate causal structure in the tokenizer architectures, their primary goal is to enable joint training on both images and videos [274]. Joint training, however, can be supported by other techniques such as padding [274].

In this paper, we fully exploit such causal structure and develop the first temporally autoregressive (AR) video tokenizer. We leverage the expressive encoder features of the *previous* frames to enhance the reconstruction of the current frame. This design allows the decoder to partially circumvent the information bottleneck imposed by the tokenizer latent space, *i.e.*, quantization in discrete tokenizers or small channel dimension in continuous tokenizers. When inferring a latent

generative model with the AR tokenizer, since decoding current frames requires the encoder features of the past frames, the decoded past frames need to be additionally encoded. The encoder and decoder operate recursively to produce the entire video.

We systematically study the scalability of transformer-based discrete and continuous tokenizers by training models with varying model sizes and flops. When training continuous tokenizers, we find that both non-AR and AR tokenizers improve reconstruction quality as we scale up the model size and the amount of training data. However, for discrete non-AR tokenizers, due to the presence of a quantization layer, we see minimal scaling gains. On the contrary, we find that the discrete AR tokenizer shows consistent performance improvements when scaling the model or data.

We comprehensively evaluate our tokenizers and compare them against the state-of-the-art methods. We show that AR tokenizers consistently outperform baselines with the same compression by achieving better reconstruction quality. Our largest AR discrete tokenizers achieve competitive reconstruction quality compared with the state-of-the-art tokenizers [105, 280] while achieving $8\times$ **higher compression rates**. Our analysis also reveals that a content-motion decomposition emerges with the autoregressive design, with tokens of current frames only encoding the difference from previous frames.

To further demonstrate the strength of our tokenizer, we train image-to-video generation models using tokenizers with a spectrum of compression rates. We show that the quality improvements in tokenizer reconstruction translate to improved generation quality. AR tokenizers reduce the generation FVD [256] by half with a similar inference speed or achieve similar generation quality with twice the inference speed.

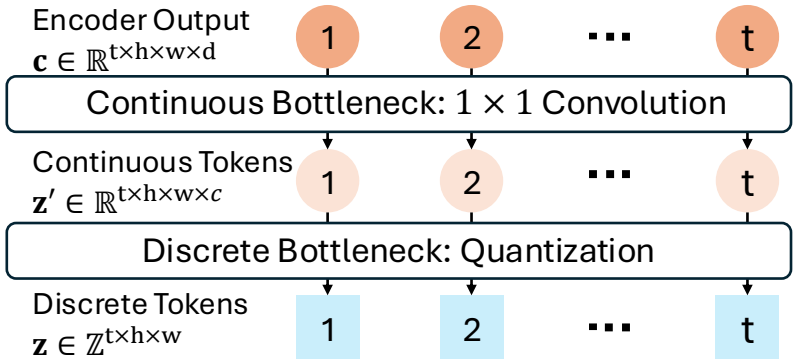


Figure 3.18: **Tokenizer bottlenecks.** There are two common types of bottlenecks in tokenizers. First, a 1×1 convolution layer maps the large encoder channel dimension (*e.g.,* $d = 512$) into a small latent channel dimension (*e.g.,* $c = 6$). Second, a quantizer is used in discrete tokenizers to quantize the tokens into discrete indices.

3.4.2 Approach

3.4.2.1 Background

Tokenizers [85, 95] often include three modules, an encoder network E with a bottleneck transformation B and a decoder network D. Given a video $\mathbf{x} \in \mathbb{R}^{T \times H \times W \times 3}$, we call the encoder output $\mathbf{z} = B(E(\mathbf{x}))$ latent tokens. Depending on whether the tokenizer is continuous or discrete, $\mathbf{z} \in \mathbb{R}^{t \times h \times w \times c}$ or $\mathbf{z} \in \mathbb{Z}^{t \times h \times w}$. The tokenizer is mainly trained to minimize the reconstruction error, *e.g.*, $\mathcal{L}_1 = \|\mathbf{x} - \hat{\mathbf{x}}\|_1$, where $\hat{\mathbf{x}} = D(\mathbf{z})$ is the decoder output.

In Figure 3.18, we illustrate the typical bottleneck B, transforming the encoder output into some lower-dimensional space that is more suitable for generative modeling. In a continuous tokenizer, it arises from the channel dimension of the latent tokens $\mathbf{z} \in \mathbb{R}^{t \times h \times w \times c}$ with $c \ll d$, where d is the encoder channel dimension. This is often implemented using a 1×1 convolution layer. The discrete tokenizer contains an additional bottleneck, the quantization layer, that maps the continuous latents into discrete codes $\mathcal{Q} \in \{q_k\}_{k=1}^{K}$, where K is the codebook size.

The generative model G is trained to model the latent token distribution $p(\mathbf{z})$. At inference

time, G generates tokens $\hat{\mathbf{z}}$ and the tokenizer decoder brings them into the pixel space $\hat{\mathbf{x}} = D(\hat{\mathbf{z}})$. Therefore, the reconstruction quality of the tokenizer establishes an upper bound on the generation quality. Additionally, the compression rate, defined as $f = f_t \cdot f_h \cdot f_w$, where $f_t = \frac{T}{t}$, $f_h = \frac{H}{h}$,and $f_w = \frac{W}{w}$, determines the context length $t \times h \times w$ and thereby the computational cost for training and sampling with G.

While the tokenizer bottlenecks have a profound and compounding effect on the generation quality [103,119,284], we are mostly interested in f for its direct impact on training and inference cost. Specifically, we study the tokenizer performance with varying f and a fixed $c = 16$ for continuous and $K = 65,536$ for the discrete tokenizer.

Obtaining a higher compression rate f and high-quality reconstruction is non-trivial. Most existing video tokenizers [30, 106, 111] adopt a convolutional architecture with a compression rate of $f_t = 4$ and $f_h = f_w = 8$. Prior work [276] shows that increasing the compression rate poses a significant challenge to reconstruction quality.

We hypothesize that the inductive bias of convolution layers limits the approachable compression rate. To support the claim, we visualize the encoded image/video tokens $\mathbf{z}$ by normalizing their first three dimensions as RGB channels in Figure 3.17. We use the Flux tokenizer [279] for images and Cosmos continuous tokenizer [280] for videos. As shown in Figure 3.17, the latent space of both image and video tokenizers closely resembles the original pixel space by preserving most visual structures. Many visual elements, like the edges, can be clearly identified in the latent tokens. When these tokenizers are applied to compress videos, they could induce a significant amount of temporal redundancy in the tokens. For example, the table in the background of the video remains static and uses nearly identical tokens over time.

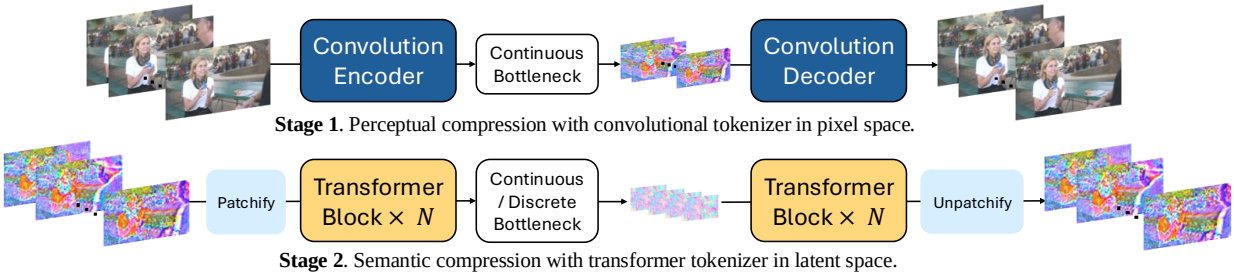


Figure 3.19: **Latent tokenizer framework.** (a) Most video tokenizers [89, 280, 285] employ a convolution-based architecture to perform preceptual compression. (b) We propose to train a transformer tokenizer in the latent space of a pre-trained convolutional tokenizer for semantic compression, which also allows computationally more efficient training.

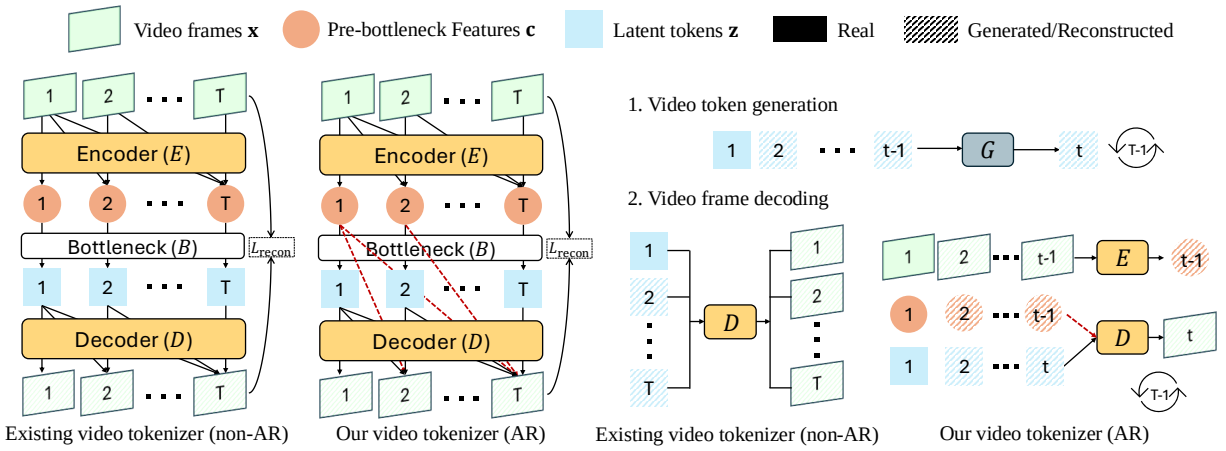
3.4.2.2 Latent Tokenizer with Transformer

We are thus motivated to use transformers for their minimal inductive biases. Training a transformer tokenizer is computationally prohibitive due to the quadratic complexity of attention and the high dimensionality of the videos [6]. One straightforward way to reduce the context length is to increase the patch size, also called the patchify operation. Although this is a common choice in visual pre-training methods [260, 286], it is challenging to achieve reasonable visual quality with large patch sizes in generative tasks [281].

Motivated by the similarity between the pixel and latent space, we seek to train a transformer tokenizer on a compressed latent space, thereby addressing the memory issue, similar to prior works [85, 282]. As shown in Figure 3.19, we use a pre-trained *continuous* convolutional tokenizer to first encode the video for perceptual compression. Then, we freeze the convolutional tokenizer and train a transformer tokenizer on the encoded video latent for semantic compression. As the input is now much smaller, we can train the transformer tokenizer much more efficiently. Although the convolutional tokenizers define the upper bound of the reconstruction quality, we

[6] To put this in perspective, using a single transformer block [20] with a channel size of 512, the forward pass of a 2 second HD video consumes more than 500TB memory.

show that the transformer tokenizer can approach this upper bound but with a much higher compression rate.



(a) Video tokenizer training (b) Video prediction inference

Figure 3.20: **Autoregressive video tokenization.** (a) When training an AR tokenizer, the pre-bottleneck features from previous frames are used to help decode the current frame (shown as red dashed lines). (b) At inference time with video generation process, the AR tokenizer re-encodes the previously generated frames when decoding the current frame to obtain the pre-bottleneck features.

3.4.2.3 Autoregressive Tokenizer

To reduce the temporal redundancy revealed in Figure 3.17, we exploit the video's autoregressive property in the tokenizer architecture. As shown in Figure 3.20(a), we propose to utilize the features from previous frames, $\mathbf{c}_{t-1}$, to help decode the current frame:

$$\hat{\mathbf{x}}_t = D(\mathbf{c}_{t-1}, \mathbf{z}), \tag{3.5}$$

where $\mathbf{c} = E(\mathbf{x})$ can be extracted before passing through the tokenizer bottleneck B, providing features much more informative than the video tokens $\mathbf{z}$.

Nevertheless, generating $\mathbf{c}_{t-1}$ is way more difficult than generating $\mathbf{z}_{t-1}$ due to its large

channel dimension d. Instead, we can obtain these features by encoding the generated frames at inference time:

$$\mathbf{c}_{t-1} = E(\hat{\mathbf{x}}_{0:t-1})_{t-1} = E(D(\hat{\mathbf{z}}_{0:t-1}))_{t-1}. \tag{3.6}$$

This is possible when the past frames $\hat{\mathbf{x}}_{0:t-1}$ can be encoded and decoded from tokens without any information of the current or future frames, necessitating a causal tokenizer structure such that for $0 \leq t \leq T$,

$$D(B(E(\mathbf{x})))_{0:t} = D(B(E(\mathbf{x}_{0:t}))). \tag{3.7}$$

In our latent transformer tokenizer, we block the information flow from future frames to the current frame using block causal attention.

At inference time, as shown in Figure 3.20(b), the tokenizer decodes the tokens in an autoregressive manner. Before decoding the current frame $\hat{\mathbf{x}}_t$, the encoder first produces the required pre-bottleneck features by encoding the previous frames $\hat{\mathbf{x}}_{0:t-1}$. When $t = 0$, in image-to-video generation, the first frame $\mathbf{x}_0$ is given, and thereby, its pre-bottleneck features $\mathbf{c}_0$. Otherwise, this can be obtained with a pre-trained text-to-image model [85, 279]. For $t > 0$, the encoder and decoder operate recursively:

$$\begin{cases} \hat{\mathbf{c}}_{0:t-1} & = E(\hat{\mathbf{x}}_{0:t-1}), \\ \hat{\mathbf{x}}_{0:t} & = D(\hat{\mathbf{c}}_{0:t-1}, \mathbf{z}_{0:t}), \end{cases} \tag{3.8}$$

where $\mathbf{z}_{1:T} = G(\mathbf{z}_0)$ in image-to-video generation. This step repeats until the full video is decoded.

Though simple, the autoregressive design, as we will show, greatly improves the recon-

Table 3.8: **Model configs for different size variants.** We report channel number, layer number, parameter size, and Gflops for models with different number of parameters.

	Channels	Layers	Parameters	Gflops
Small (S)	512	8	51M	1.44
Base (B)	768	12	171M	3.14
Medium (M)	1024	24	607M	6.23
Large (L)	1280	32	1.26B	10.27
Huge (H)	2048	24	2.40B	22.40

struction quality that can transfer to the generation results. It also reduces temporal redundancy and enables larger compression rates. Note that the decoding process does not need to start until all the tokens are generated. In practice, the autoregressive decoding can be parallelized with autoregressive generation to improve the efficiency. We also note that this design is not solely limited to the transformer architecture but can apply to a convolutional architecture.

3.4.3 Experiments

3.4.3.1 Experimental Setup

Tokenizer details. We adopt a ViT transformer [286] design for our tokenizer, containing patchify layers and a stack of transformer blocks. To perform video downsampling, we use a patch size of $2 \times 2 \times 2$ by default. We use the Cosmos continuous video tokenizer [280] with compression rate $8 \times 8 \times 8$ for the stage 1 tokenizer. With the default patch size, our tokenizer offers a total compression rate of $16 \times 16 \times 16$. We also explore an alternative compression approach by masking the latent tokens input to the decoder [287] in the ablation study. We use a fixed $\mathbf{c} = 16$ for the continuous tokenizers. For discrete tokenizers, we employ FSQ [110] with quantization levels set as $[8, 8, 8, 5, 5, 5]$, inducing a codebook size of 64K. We train various

AR and non-AR tokenizers with different model sizes as displayed in Table 3.8. To combine the pre-bottleneck features and video tokens in our AR decoder, we concatenate them in the channel dimension and adjust the decoder's input channel size accordingly.

Training details. We use an $\mathcal{L}_1$ loss for reconstruction. We train our tokenizers on a proprietary dataset of videos where each video is with 512×896 resolution and consists of 49 frames. For image-to-video generation, we train a 1B Llama model [278] on two benchmarks, BDD100K [288] and Kinetics-600 [289]. We filter the Kinetics-600 dataset by removing those with resolutions smaller than 512×896, resulting in a set of $158,654$ videos in total.

Baselines. We compare our methods against several state-of-the-art convolutional video tokenizers, Cosmos tokenizer [280], LARP [105], and OmniTokenizer [285]. We also train non-AR transformer tokenizers. Since Cosmos tokenizer provides model checkpoints with different compression rates, allowing a comprehensive study of efficiency versus quality, we use them as the baselines in the image-to-video evaluation.

Evaluation. For video reconstruction, we compute PSNR, SSIM, LPIPS, and reconstruction FVD (rFVD) on two benchmarks, DAVIS [290] and TokenBench [280]. We evaluate the tokenizers by reconstructing the entire video sequence in chunks. The videos are resized into 512×896 for Cosmos Tokenizer and our AR tokenizer and 512×512 for OmniTokenizer to obtain the best

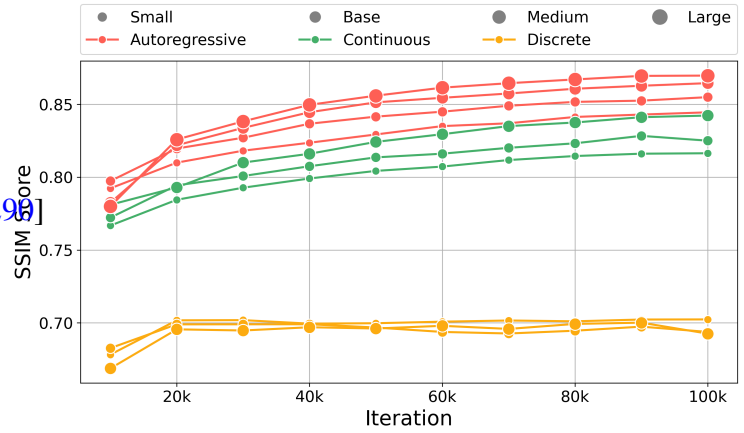


Figure 3.21: **Transformer tokenizer scalability.** We train models of different sizes (Small, Base, Medium, and Large) and types: discrete or continuous, AR or non-AR. Continuous tokenizers and discrete AR tokenizers improve with larger model sizes and longer training, while discrete non-AR tokenizers plateau.

results. We also evaluate AR tokenizers without accessing the ground truth pre-bottleneck features. For image-to-video, we report performance using FVD. We randomly sample 512 videos from the BDD100K training set and the Kinetics-600 validation set.

3.4.3.2 Main Results

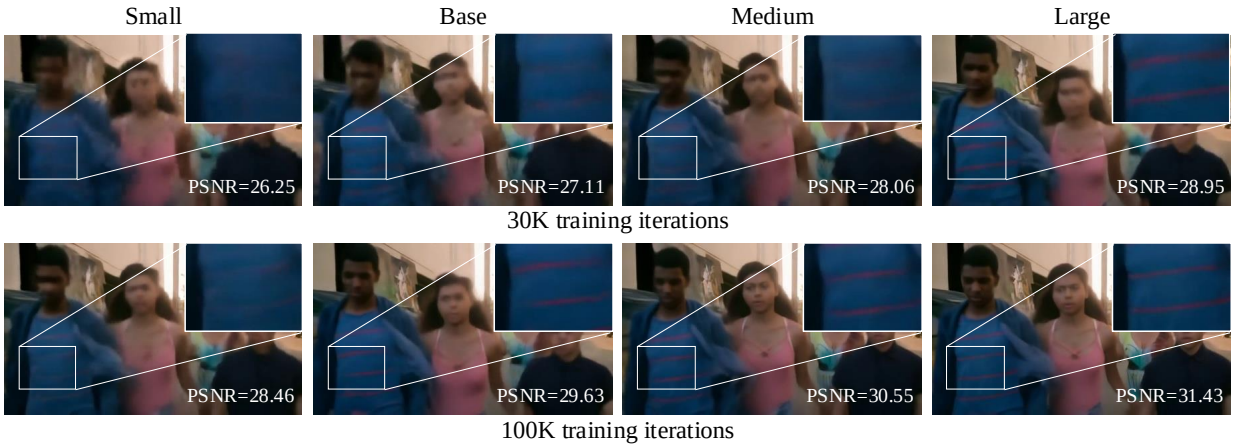


Figure 3.22: **Discrete AR tokenizer reconstruction results at different iterations and model sizes.** Reconstructing the dynamics regions with high-frequency details is extremely challenging. With larger tokenizer model sizes or more training data, the reconstruction quality of our AR tokenizers keeps improving, leading to sharper visual results and higher PSNRs.

AR Tokenizer Scalability. We train four types of transformer-based tokenizers, Continuous-AR, Discrete-AR, Continuous-Non-AR, and Discrete-Non-AR, with different model sizes. For each tokenizer, we plot the TokenBench SSIM scores at different training iterations in Figure 3.21. The discrete non-AR tokenizer barely benefits from larger model sizes or training on more data—the three models achieve similar reconstruction quality. On the contrary, the discrete AR tokenizers improve substantially with a larger training scale, and so do continuous AR and non-AR tokenizers. We hypothesize that the bottleneck imposed by quantizers limits the achievable reconstruction quality; the pre-bottleneck features adopted by the AR tokenizer facilitate scalability by circumventing the bottleneck. In addition, we find that the AR tokenizers consistently outper-

Table 3.9: **Evaluation of discrete video tokenizers.** We quantitatively evaluate our discrete AR tokenizers against multiple baselines on two benchmarks with four evaluation metrics. * indicates that we train the model ourselves following the official receipt. The *continuous* CosmosTokenizer with $8 \times 8 \times 8$ compression forms the quality upper bound for AR tokenizers.

		DAVIS [290]				TokenBench [280]			
Tokenizer	Compression (↑)	PSNR (↑)	SSIM (↑)	LPIPS (↓)	rFVD (↓)	PSNR (↑)	SSIM (↑)	LPIPS (↓)	rFVD (↓)
VideoGPT	$4 \times 4 \times 4$	32.55	0.884	0.241	67.75	35.61	0.925	0.182	13.85
LARP	$4 \times 8 \times 8$	30.01	0.534	0.518	514.60	31.62	0.703	0.434	373.39
OmniTokenizer	$4 \times 8 \times 8$	28.92	0.704	0.359	154.45	30.14	0.837	0.264	47.35
CosmosTokenizer	$4 \times 8 \times 8$	32.04	0.736	0.313	80.98	34.92	0.882	0.214	27.16
CosmosTokenizer	$8 \times 8 \times 8$	31.65	0.723	0.354	164.62	34.55	0.865	0.233	51.06
CosmosTokenizer	$8 \times 16 \times 16$	30.96	0.637	0.431	516.05	33.45	0.807	0.298	166.35
CosmosTokenizer*	$16 \times 16 \times 16$	30.36	0.571	0.543	1085.44	32.61	0.753	0.404	451.34
AR-Tokenizer-S	$16 \times 16 \times 16$	30.40	0.628	0.450	698.56	33.46	0.838	0.254	152.96
AR-Tokenizer-B	$16 \times 16 \times 16$	30.78	0.662	0.417	579.83	33.76	0.851	0.238	116.01
AR-Tokenizer-M	$16 \times 16 \times 16$	30.99	0.682	0.382	488.24	34.07	0.863	0.220	91.95
AR-Tokenizer-L	$16 \times 16 \times 16$	31.16	0.703	0.362	446.58	34.45	0.874	0.212	82.73
AR-Tokenizer-H	$16 \times 16 \times 16$	**31.29**	**0.711**	**0.347**	**407.80**	**34.73**	**0.882**	**0.204**	**73.10**
AR Upper Bound	$8 \times 8 \times 8$	32.80	0.801	0.252	55.84	36.28	0.908	0.156	12.49

form the non-AR tokenizers across all model sizes and tokenizer types. We visualize the effect of scaling on reconstruction quality in Figure 3.22.

Comparison with State-of-the-art Tokenizer. We evaluate our AR tokenizer against different baselines on two video benchmarks, DAVIS [290] and TokenBench [280]. We show quantitative results of discrete tokenizers in Table 3.9. Our AR tokenizers consistently outperform the state-of-the-art video tokenizers with the same compression rate. Moreover, the largest models, Large and Huge, outperform the CosmosTokenizer with $2\times$ less compression and perform comparably with CosmosTokenizer using $8\times$ less compression. Our AR tokenizers improve baselines more on TokenBench than on DAVIS. TokenBench includes diverse videos from various scenarios, while DAVIS, designed for tracking, contains low-fps videos (< 20). This suggests that autoregressive tokenization benefits more from scenarios with frame redundancy, suggesting an interesting future direction of adaptive tokenization as been developed in video codec [291, 292].

Image-to-Video Generation with Discrete AR Tokenizer. While the experiments show that

Baseline Ours

Figure 3.23: **Visual comparison of image-to-video.** We compare video generation results using a baseline tokenizer and ours. The improved reconstruction quality of our AR tokenizer translates to video generation results and induces sharper generation.

Table 3.10: **Evaluation of video prediction with different tokenizers on BDD100k and Kinetics-600 datasets.** We report the wall clock time of sampling one video and FVD scores.

	Compression	Time (s)	FVD (↓)	
			BDD100k	K600
CosmosTokenizer	$4 \times 8 \times 8$	2198.8	202.91	605.41
CosmosTokenizer	$8 \times 8 \times 8$	747.7	205.13	568.01
CosmosTokenizer	$8 \times 16 \times 16$	158.6	252.45	571.32
CosmosTokenizer	$16 \times 16 \times 16$	85.1	570.89	815.87
AR-Tokenizer-S	$16 \times 16 \times 16$	85.7	324.41	629.70
AR-Tokenizer-B	$16 \times 16 \times 16$	85.9	263.70	588.10
AR-Tokenizer-M	$16 \times 16 \times 16$	86.3	270.82	526.93
AR-Tokenizer-L	$16 \times 16 \times 16$	86.9	**250.19**	**509.22**

AR-tokenizer outperforms other SOTA approaches on reconstruction metrics, it is unclear if it improves the video generation quality. To this end, we train image-to-video generation models with different discrete tokenizers. For all experiments, we train a 1B Llama3 transformer model [278] with the next-token prediction objective for the number of iterations to the same number of training tokens. We measure FVD scores on the generated videos in Table 3.10. AR tokenizers consistently outperform CosmosTokenizer with $16 \times 16 \times 16$ compression ratio, while they only induce minimal extra efficiency overhead ($0.7\% - 2.1\%$). Visual comparisons are shown in Figure 3.23 and supplementary website. In addition, AR-Tokenizer-L achieves comparable generation quality with CosmosTokenizer-$8 \times 16 \times 16$ while halving the inference latency with its larger compression ratio.

Table 3.11: **Pre-bottleneck feature ablation.** We quantitatively compare different pre-bottleneck features and decoder designs to leverage the features on the TokenBench dataset.

Tokenizer	Feature	Decoder	PSNR	SSIM
Continuous-Non-AR	-	-	33.08	0.817
Discrete-Non-AR	-	-	30.03	0.688
Discrete-AR	PostConv	ChannelConcat	31.20	0.742
Discrete-AR	PreConv	SequenceConcat	32.95	0.827
Discrete-AR	PreConv	CrossAttnenstion	31.63	0.769
Discrete-AR	PreConv	ChannelConcat	**33.76**	**0.851**

Table 3.12: **Ablation study on downsampling strategy.** We explore using patchify layers or token masking for downsampling in the tokenizer transformers on the DAVIS dataset.

Patch size	Masking	Mode	Compression	PSNR	SSIM
$2 \times 2 \times 2$	$1\times$	-	$16 \times 16 \times 16$	30.99	0.682
$1 \times 2 \times 2$	$4\times$	Last	$8 \times 64 \times 16$	**31.01**	**0.701**
$2 \times 2 \times 2$	$2\times$	Uniform	$16 \times 32 \times 32$	30.80	0.675
$2 \times 2 \times 2$	$2\times$	Last	$16 \times 64 \times 16$	30.95	0.680
$2 \times 2 \times 2$	$4\times$	Uniform	$16 \times 64 \times 64$	30.69	0.656
$2 \times 2 \times 2$	$4\times$	Last	$16 \times 64 \times 16$	30.81	0.665

3.4.3.3 Ablation Study

Pre-bottleneck Feature. There are different pre-bottleneck features and several ways to use them in the decoder. We explore these designs using discrete-AR-tokenizers with the Base model size. For pre-bottleneck features, we investigate using features before (*PreConv*) and after (*PostConv*) the 1×1 convolution. For the feature merging design in the decoder architecture, apart from the default concatenation in the channel dimension (*ChannelConcat*), we investigate concatenation along the sequence dimension, which effectively doubles the context length. We also explore using *Cross Attention* to combine the features (we reduce the total number of layers to keep the model size relatively the same). We show the SSIM scores achieved on TokenBench for different designs in Table 3.11. All models that leverage pre-bottleneck features consistently outperform

Figure 3.24: **Zero-shot editing propagation with AR tokenizer.** We edit only the first frame of the video using an image editing tool. Then, we reconstruct the video using the continuous features of the edited frame and the video tokens of the original video. The edited style propagates smoothly to the other frames. Hence, the video tokens of the current frame primarily encode differences from previous frames.

the baselines that do not utilize these features. The features before the convolution bottleneck are much more informative than those after, enabling AR tokenizers to outperform continuous non-AR tokenizers. The channel concatenation works best among different approaches for mixing the features.

Other Token Downsampling Approaches. Apart from the most common downsampling approach - the patchify operation [286], we also explore the use of video token masking [115, 287]. Given video tokens $\mathbf{z}$, the tokenizer decoder reconstructs the video based on the masked tokens $\hat{\mathbf{x}} = D(\mathbf{M} \cdot \mathbf{z})$, where $\mathbf{M}$ is a predefined binary matrix. We explore two types of masking strategies: (1) *Uniform* masking, which preserves every n tokens, and (2) *Last* masking, which preserves the top $\lfloor \frac{h}{n} \rfloor$ rows. When training a generative model, the masked tokens are dropped. As shown in Table 3.12, *Last* masking outperforms *Uniform* masking. Combining two downsampling strategies holds the promise to achieve better reconstruction quality.

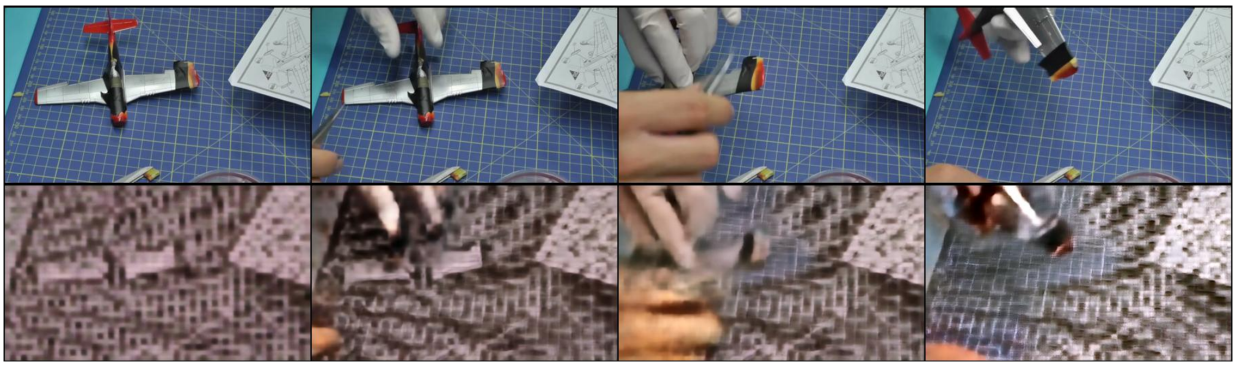


Figure 3.25: **Motion-content decomposition.** Videos reconstructed (bottom) with zero continuous features and tokens from input videos (top) show that video tokens only encode differences.

3.4.3.4 Understanding Autoregressive Tokenizer

Our autoregressive design enforces motion-content decomposition. The AR video tokens only encode the changes from previous frames: the optical flow and the novel content. To illustrate this, we tokenize and autoregressively reconstruct the videos with the continuous features set to zero. As shown in Figure 3.25, only unseen objects, such as hands, are encoded into video tokens and properly reconstructed. This property enables our tokenizer's zero-shot capability in video editing. Given an input video, we edit its first frame using an image editing model, then reconstruct the entire video with the edited continuous features and the original video tokens. As shown in Figure 3.24, the edited style propagates naturally through the video. Such motion-content decomposition significantly reduces temporal redundancy and enables larger compression rates.

3.4.4 Conclusion

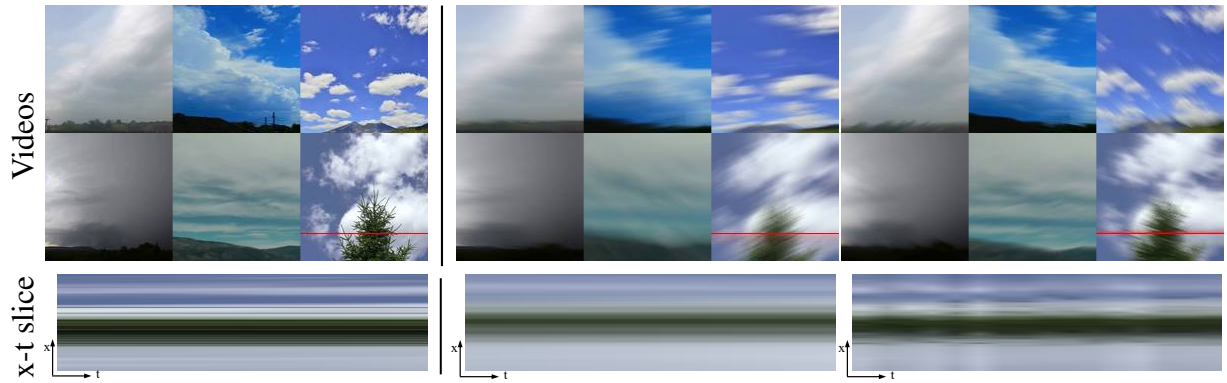
We propose a novel video tokenizer framework. Our tokenizer exploits the pre-bottleneck features from past frames to facilitate decoding the current frames, dramatically reducing the

temporal redundancy by forcing the tokens to encode only the motions. Our AR tokenizer model scales well with more data and larger models and achieves more compression without compromising the reconstruction quality. Nevertheless, we think the current video tokens are still redundant since much more compact forms exist—the underlying 4D world that renders the videos. We hope this work could facilitate research in designing the minimal representation to recreate the dynamic visual world.

Chapter 4: Evaluation

4.1 Overview

"You can't improve what you don't measure." Evaluation of the generative models has long been a critical research direction. Due to the subjective nature of evaluating the generated images and videos, human is often deemed to be the oracle. Several works have thus been motivated to build a common prompt list for the following works to evaluate their models on [47, 92]. However, human evaluation can be tedious, time-consuming, and difficult to reproduce. Therefore, automatic methods are still valuable in different scenarios such as ablation study, benchmarking, and evaluation at scale. Fréchet Inception Distance (FID) [126] and Fréchet Video Distance (FVD) [293] are the two prominent metrics for evaluating image and video generation models. However, they are known to conflict with human perception occasionally. In Section 4.2, we explore the extent of FVD's bias toward per-frame quality over temporal realism and identify its sources. We also show that FVD with features extracted from the recent large-scale self-supervised video models is less biased and can serve as a better quantitative evaluation metric. In addition, multiple new benchmarks [137, 294] and evaluation metrics [137, 138] have been developed. In Section 5.2 on replacing plain text with rich text, we also propose a benchmark for evaluating rich text to image generation.



(a) Reference Videos

(b) Medium Spatial & No Temporal Corruption FVD=317.10

(c) Small Spatial & Severe Temporal Corruption FVD=310.52

Figure 4.1: **FVD is biased towards per-frame quality than temporal consistency.** FVD [293], a commonly used video generation evaluation metric, should ideally capture *both* spatial and temporal aspects. However, our experiments reveal a strong bias toward individual frame quality. (b) First, we apply mild spatial distortions through local warping, which results in an FVD score of 317.10. (c) Next, we induce slightly less spatial corruptions but *severe* temporal inconsistencies by altering each frame differently. These changes create artifacts that are noticeable to humans and evident in the spatiotemporal x-t slice, as seen in the bottom row, but surprisingly lead to a lower FVD score of 310.52. This discrepancy highlights the metric's bias towards individual frame quality. *We encourage readers to view the videos with Acrobat Reader or visit our website to observe the inconsistencies.*

4.2 Analyzing the Bias in Existing Evaluation Metric

4.2.1 Introduction

Video generation [61, 63, 70, 72, 90, 92, 295] has recently accomplished unprecedented advances driven by the scalable models [39, 40] and growing training data [296, 297]. With rapid progress, it is increasingly crucial to evaluate the model performance accurately. Despite the fruitful literature on assessing video quality [298–301] and designing image generation evaluation metrics [125, 126, 128, 129, 134], automatically evaluating the quality and diversity of the generated videos, has received less attention [142, 302]. In this paper, we focus on analyzing the bias of Fréchet Video Distance (FVD) [293], one of the most frequently used metrics for video

evaluation.

FVD extends the image generation metric Fréchet Inception Distance (FID) [126] to measure the quality and diversity of generated videos with respect to the training set. Given N features $\mathbf{f}_i$, which are column vectors, extracted from a pretrained video network, for a set of generated and real videos, we fit a multivariate Gaussian with the mean $\mu = \frac{1}{N}\sum_i \mathbf{f}_i$, and covariance $\mathbf{\Sigma} = \frac{1}{N}\sum_i (\mathbf{f}_i - \mu)(\mathbf{f}_i - \mu)^T$. The performance of the video generator is then measured as the Fréchet distance [303] between the two Gaussian distributions:

$$\text{FVD} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\left(\mathbf{\Sigma}_r + \mathbf{\Sigma}_g - 2\left(\mathbf{\Sigma}_r\mathbf{\Sigma}_g\right)^{\frac{1}{2}}\right), \tag{4.1}$$

where $(\mu_r, \mathbf{\Sigma}_r)$ and $(\mu_g, \mathbf{\Sigma}_g)$ denote the mean and covariance for real and generated data.

Earlier studies have confirmed that FVD reliably reflects the model performance in various cases, such as training convergence [91], hyperparameter tuning [63, 304], and architecture design [38, 62]. However, several recent studies have reported cases where FVD scores contradict human judgment [38, 106, 304]. A recurrent argument is that FVD tends to value the image quality of individual frames more than the realism of motion. We refer to such bias as the *content bias*, which is inspired by the video generation works in decoupling content and motion [10, 14, 305–307].

As shown in Figure 4.1, we motivate our analysis with a simple, controlled setting, where the metric diverges from human perception when weighing spatial and temporal qualities. Specifically, given a set of videos (a), we create two sets of distortion. In (b), we locally warp the frames in each video uniformly, while in (c), we distort the frames differently but with slightly reduced severity. The latter creates additional temporal artifacts. The FVD metric, however, favors video

set (c), while most humans would pick video set (b) to be more similar to the reference videos due to the significant temporal inconsistency presented in video set (c).

Building upon this simple example, we present the first systematic study to quantify the content bias and understand its impact using both synthetic and real-world settings. We first distort videos so that the frame quality deteriorates to the same level while the temporal consistency is either intact or, in the other case, significantly decreased. By comparing FVDs on these distorted videos, we can quantify the relative sensitivity of the FVD metric to the temporal consistency. Next, following the previous work on FID analysis [124], we probe the perceptual null space in the FVD metric. Without improving the temporal quality of the generated videos, we can still greatly reduce the FVD scores. Lastly, we revisit a few real-world examples where FVD presents a notable content bias.

Where does the content bias originate from? Previous studies show that the alignment of the FID metric to human perception depends on the choice of the extracted features [124,127,135, 136]. In practice, FVD employs an Inflated 3D ConvNet (I3D) model [249], originally trained for action recognition on the Kinetics-400 dataset [249]. The feature space is formed as the output of the logit layer. As a result, these features focus on extracting the semantic information about human actions in the videos.

Using I3D features thus raises several practical concerns that could undermine the metric's reliability. First, the Kinetics dataset [249] predominantly comprises videos with humans as the protagonists. However, video content diverging from typical Kinetics-400 categories, such as time-lapse landscape videos [236] or first-person riding and biking videos [304], may not produce a meaningful feature representation. Second, the models trained on the Kinetics dataset are shown biased to the appearance of objects and backgrounds instead of motions [308–313]. For example,

Figure 4.2: **Visualization of the spatial and spatiotemporal corruptions.** Both corruptions yield similar frame quality, while the spatiotemporal corruption induces additional temporal inconsistency in the video. By comparing the FVD of the spatiotemporal corruption with the spatial corruption, we analyze the temporal sensitivity of the metric. *Best viewed with Acrobat Reader. Please check our website for videos.*

to recognize the action "playing saxophone", it is sufficient to detect the presence of a saxophone since this is the only category where a saxophone is presented. Therefore, the features may not capture the musician's motion. Previous works also show that descent classification accuracy can be achieved without modeling the temporal aspect [249, 314].

To verify our hypothesis, we compute FVD scores using features extracted from a self-supervised model [315] trained on diverse dataset and perform the same analysis. Overall, our experiments show that the FVD, computed with I3D features, is strongly biased to the content over the motion, while using features computed with a model trained in a self-supervised manner helps mitigate such bias to a large extent. Our evaluation code and data are available on `https://content-debiased-fvd.github.io/`.

4.2.2 Quantifying the Temporal Sensitivity of FVD

We examine the significance of temporal quality and consistency in FVD calculation. Recent studies suggest that models trained on the Kinetics datasets may not fully leverage the motion information [309, 312–314], raising a similar question about whether the I3D features in FVD truly capture the motion quality of videos. One way to understand video motion *vs.* content is

to undermine one aspect through either spatial or temporal distortion. However, fully decoupling the two aspects is non-trivial [309]. For example, poor frame quality would hinder the creation of a natural motion. As a result, previous approaches suggest creating videos by stitching real frames together, albeit in an incorrect order or from different videos [293, 309, 313].

While this method is useful for analyzing video datasets, it may not be ideal for understanding a video generation metric. This is because generated videos rarely contain frames from irrelevant videos or arrange frames in incorrect order. In contrast, we carefully design distortions that simulate real scenarios to quantify FVD's sensitivity to spatial and temporal video quality.

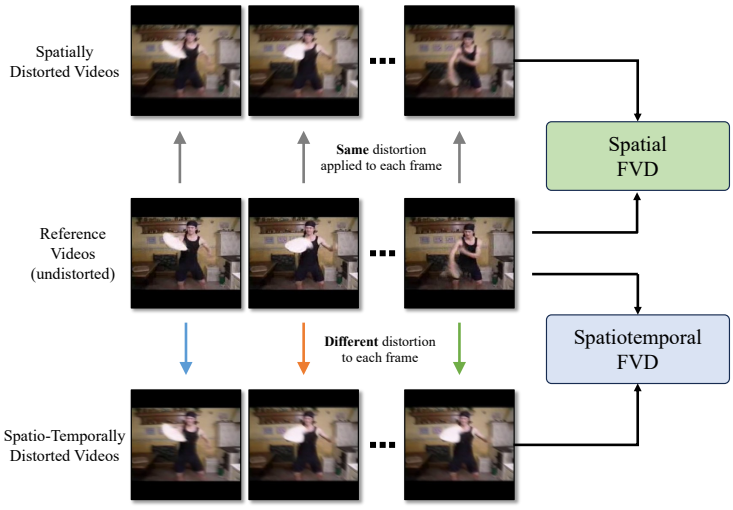


Figure 4.3: **Analyzing the FVD's sensitivity to temporal consistency.** We distort the same set of videos in spatial only or spatiotemporal manners so that the resulting videos have similar frame quality yet only differ in temporal quality. By comparing the FVD scores of the two distorted video sets, we aim at quantifying the temporal sensitivity of the metric.

Video distortion methods. We illustrate our method for adding distortions in Figure 4.3. We apply two relevant distortions to the same set of real videos, aiming to synthesize videos with similar frame quality degradation but large differences in temporal quality. We employ conventional image distortion methods [316, 317] including elastic transformation and motion blur. The elastic transformation locally stretches and contrasts the frames, while the motion blur averages image pixels along a specific direction of motion. We apply the same distortions to each frame to achieve a consistent frame quality drop.

Table 4.1: **Analyzing FVD temporal sensitivity with video distortions.** We apply spatial only (S) or spatiotemporal (ST) distortions to the videos. The two video sets share similar frame quality as assessed by FID. We thus use the FVD ratio to measure the temporal sensitivity of FVD.

Metric	Distortion	UCF-101	Sky Time-lapse	FaceForensics	Taichi-HD	SSv2	Kinectics-400
FID	S	133.15	79.11	80.42	169.76	100.65	112.22
	ST	133.69$_{(+0.4\%)}$	79.35$_{(+0.3\%)}$	79.57$_{(-1.1\%)}$	170.10$_{(+0.2\%)}$	100.62$_{(-0.0\%)}$	112.85$_{(+0.6\%)}$
FVD	S	1460.18	211.08	354.49	1016.78	594.68	996.71
	ST	1705.27$_{(+16.8\%)}$	286.39$_{(+35.7\%)}$	367.35$_{(+3.6\%)}$	1201.35$_{(+18.2\%)}$	678.08$_{(+14.0\%)}$	1155.53$_{(+15.9\%)}$

To create spatiotemporal corruptions, as shown in Figure 4.2, we apply randomly sampled elastic transformation parameters or blur kernels for each frame. This procedure allows us to introduce temporal inconsistency while producing similar frame quality to spatial corruptions.

Experimental setups. After distorting the videos using spatial only and spatiotemporal methods, we compute the FVD score of each video set with respect to the original videos. We apply distortion in five predefined corruption levels [317] and compute the average. To verify that the two distorted sets have similar frame quality, we compute FID [125] on the frames extracted from each set against the original image frames. Finally, we use the relative ratio between changes in FVD and FID to measure temporal sensitivity.

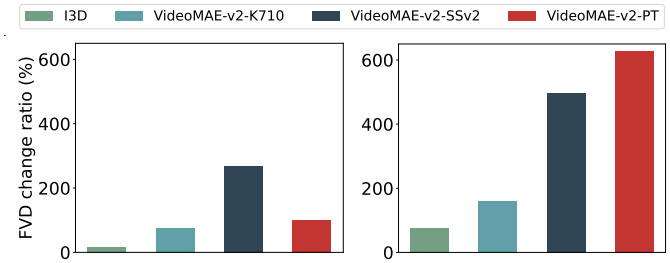
We perform the experiments on several standard video datasets, including Kinetics-400 [249], Something-Something-v2 [318], UCF-101 [235], Sky Time-lapse [236], and FaceForensics [319] datasets. Motivated by the previous finding that the unsupervised models trained on large datasets often produce more reliable features in FID [124, 127], we also compute FVD using a self-supervised video model VideoMAE-v2 [315], which is trained on a mixed set of unlabeled datasets with the Masked Autoencoders (MAE) reconstruction objective [320]. Due to the large gap between the pertaining and downstream tasks, the MAE models are often further fine-tuned on the downstream tasks.

In our experiments, we explore a pretrained model *VideoMAE-v2-PT* and two models fine-tuned on Kinetics-710 dataset [321] (*VideoMAE-v2-K710*) and SSv2 dataset [318] (*VideoMAE-v2-SSv2*).

FVD temporal sensitivity.

We present FID and FVD scores obtained from the videos distorted spatially or spatiotemporally on different datasets in Table 4.1. The minimal FID difference between the spatial and spatiotemporal distortion videos validates our claim that the two distorted video sets share similar frame quality.

Regarding the FVD scores of the spatially distorted videos, several datasets that are not in the distribution of the I3D training data, such as Sky Time-lapse, FaceForensics, and SSv2, generally yield much smaller FVD values compared to in-distribution datasets like Taichi-HD, UCF-101, and Kinectics-400. Given the same level of distortion is applied across different datasets, this suggests that FVD is less sensitive to distortion on the out-of-domain data. We inspect FVD's temporal sensitivity based on its increases induced by temporal inconsistency. Specifically, we compute the relative change of FVD between spatial and spatiotemporal corruptions. We find that FVD sometimes fails to detect the temporal quality decrease. For example, the temporal inconsistency in the FaceForensics dataset only raises FVD by 3%.



(a) Motion Blur (b) Elastic Transformation

Figure 4.4: **FVD sensitivity with different video feature extractors.** We show that by substituting the I3D features with ones computed from the VideoMAE-v2 model, the temporal sensitivity can be significantly improved for both motion blur and elastic transformation distortions.

To grasp the significance of the FVD increase due to temporal inconsistency, we compare it with the FVD values computed using VideoMAE-v2 models in Figure 4.4, where we report the average FVD scores across multiple datasets. When using the VideoMAE models to extract features, we notice a much more pronounced increase of FVD on the videos with temporal inconsistency. For example, when the elastic transformation is adopted to introduce temporal inconsistency, FVD with VideoMAE-v2-PT increases **five** times more than the original FVD with the I3D model.

Compared to VideoMAE variants, the VideoMAE model fine-tuned on the SSv2 dataset consistently exhibits greater temporal sensitivity than the one fine-tuned on the K710 dataset, at least threefold. This difference can be attributed to the SSv2 dataset's emphasis on motion, where different videos share similar visual content, while differences only arise in fine-grained motion cues. Both *VideoMAE-v2-SSv2* and *VideoMAE-v2-PT* exhibit larger sensitivity to the temporal quality. Computing FVD with *VideoMAE-v2-SSv2* features effectively captures mild temporal quality decrease induced by Motion Blur, whereas using *VideoMAE-v2-PT* features proves more sensitive to temporal distortion introduced by Elastic Transformation. Overall, they all exhibit more sensitivity to temporal corruptions compared to FVD computed with the I3D model.

Where does the content bias originate from?

Multiple factors could contribute to the increased temporal sensitivity when using features computed from the VideoMAE-v2 model. These factors may encompass the model architecture, training objectives, model capacity, and the dataset. To unravel these intricacies, we further delve into a comparative study with two other models, VideoMAE-v1 [322] and TimeSFormer [314] models.

The VideoMAE-v1 model uses a smaller ViT model size while sharing the same objective as the VideoMAE-v2 model, which helps us demystify the training scales. Due to limited computing resources, we cannot train the VideoMAE-v2 ViT model from scratch with the supervised objective. Instead, we train the smaller ViT with the size of VideoMAE-v1 using the recipe from TimeSFomer. Both models are trained on the Kinetics-400 dataset, sharing the same training dataset as the I3D model. For VideoMAE-v2, we use the *VideoMAE-v2-K710* model, as it shares the most similar fine-tuning dataset, Kinetics-710, with other models. Note that it has the least temporal sensitivity among the three variants, as shown in Figure 4.4. We use it to make a fair comparison regarding the fine-tuning dataset.

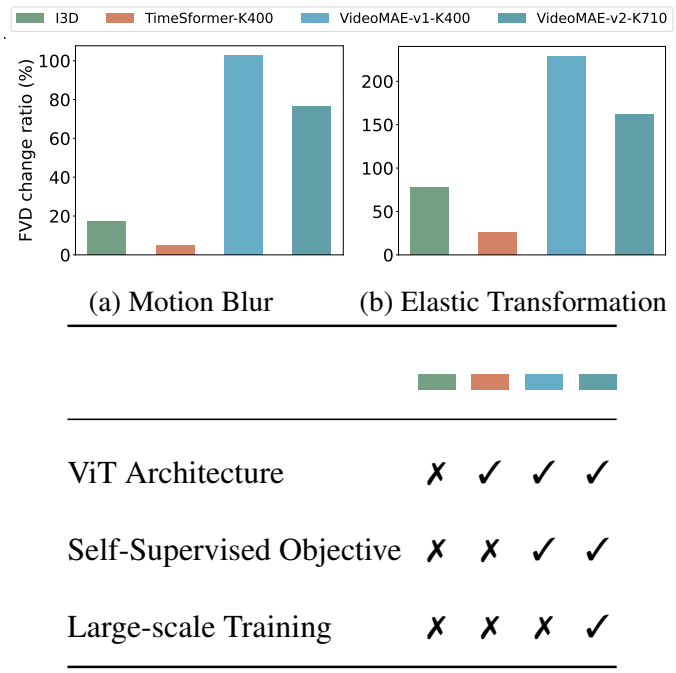


Figure 4.5: **The origin of FVD sensitivity.** We show the temporal sensitivity in the VideoMAE features is mainly attributed to the self-supervised objective.

We summarize the distinctions between these models in the table of Figure 4.5. We perform our temporal sensitivity analysis and report the FVD ratio with different feature extractors in Figure 4.5. The major improvement in the temporal sensitivity arises when comparing the VideoMAE-v1 and TimeSFormer models. Based on these observations, we conclude that the self-supervised training objective contributes the most to mitigating the content bias.

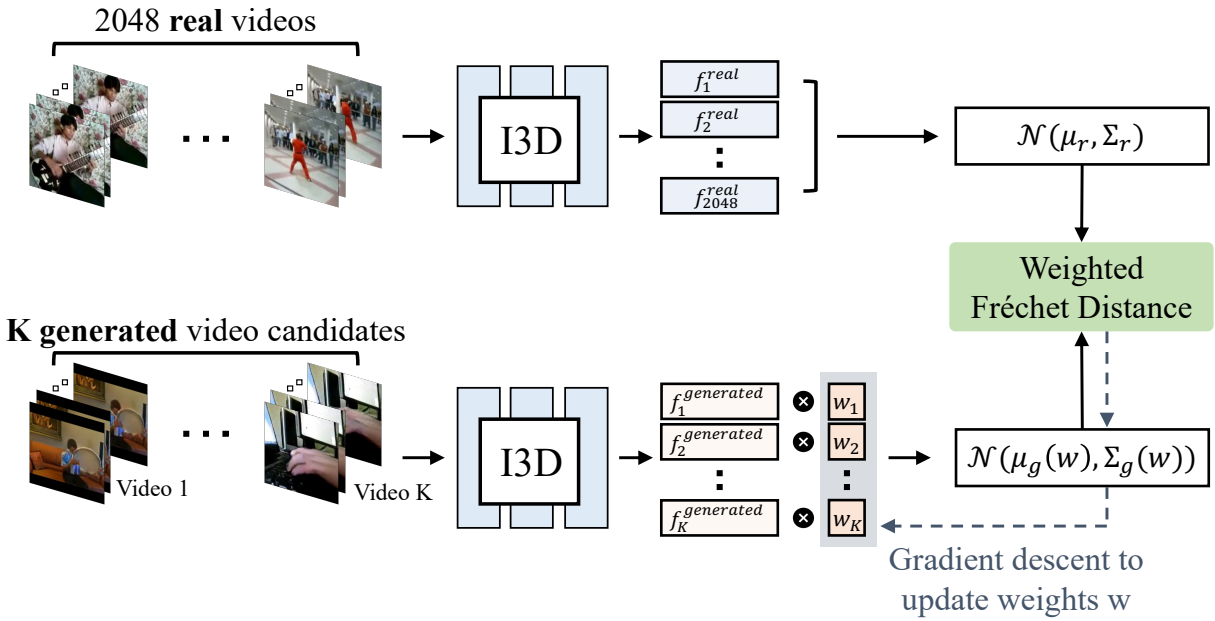
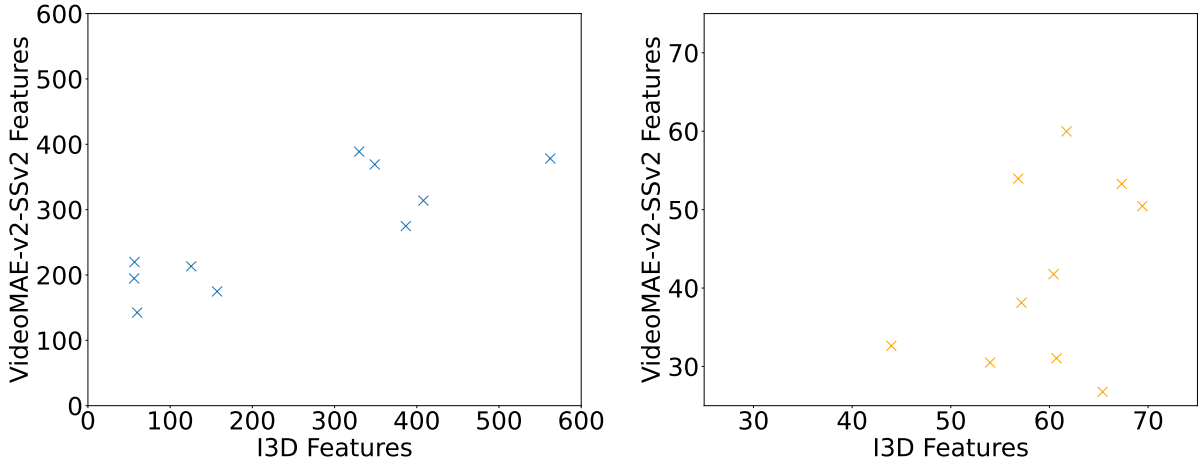


Figure 4.6: **Probing the perceptual null space in FVD.** We sample a $8\times$ larger set of fake videos and compute a weight for each candidate video by optimizing the weighted Frechet distance. We then use the weights to sample $2,048$ videos to compute the final FVD score.

4.2.3 Probing the Perceptual Null Space in FVD

Having observed FID's insensitivity to temporal quality in our synthetic experiments, a natural question arises: how does this bias impact practical evaluation? To address this question, we leverage an analysis method introduced to understand undesired behavior in FID [124]. This tool examines the *perceptual null space*, where the quality of generated images remains similar while the FID score can be effectively adjusted. In our scenario, we generate a large set of candidate videos from the same model and carefully select a subset to lower the FVD score.

As the core assumption of the method, the samples synthesized by the same model should exhibit relatively similar visual quality. However, we observe that the quality may sometimes vary for different generative videos. Therefore, we further extend the analysis to understand the concept of temporal perceptual null space, where we hard-constrain the temporal quality of the



(a) FVD scores (b) FVD decrease by resampling (%)

Figure 4.7: **FVD decrease induced by resampling across different models and datasets.** We compare FVD computed from the VideoMAE-v2-SSv2 and I3D features. Each dot represents a video generation model trained on a specific dataset. (a) We notice a non-monotonic relationship between the FVD computed with I3D and VideoMAE-v2 features. (b) After resampling, the FVD with VideoMAE features generally decreases less than the FVD with I3D features, since most dots are located in the bottom right area.

generated videos to be the same by using frozen videos.

Resampling method. We adopt the resampling technique proposed by Kynkäänniemi et al. [124]. Given a set of K, where $K > N$, generated videos, we assign a weight $w_k \in \mathbb{R}$ for each video, aiming to minimize the *weighted FVD*. Given the weights, the FVD defined in Equation 4.1 is reformulated with the weighted mean $\mu_g(\mathbf{w}) = \frac{\sum_k \exp w_k \mathbf{f}_k}{\sum_k \exp w_k}$, and covariance $\mathbf{\Sigma}_g(\mathbf{w}) = \frac{\sum_k \exp w_k (\mathbf{f}_k - \mu)(\mathbf{f}_k - \mu)^T}{\sum_k \exp w_k}$ as:

$$\|\mu_r - \mu_g(\mathbf{w})\|_2^2 + \mathrm{Tr}\left(\mathbf{\Sigma}_r + \mathbf{\Sigma}_g(\mathbf{w}) - 2\left(\mathbf{\Sigma}_r \mathbf{\Sigma}_g(\mathbf{w})\right)^{\frac{1}{2}}\right). \tag{4.2}$$

This *weighted FVD* serves as the objective for optimizing $\mathbf{w}$. After the optimization process, the resampling is performed with the probability $\frac{\exp w_k}{\sum_k \exp w_k}$. $2,048$ videos are sampled from the candidate set to compute the new FVD score, which we denote as FVD*.

(a) Candidate videos with the smallest weights. (b) Candidate videos with the largest weights.

Figure 4.9: **Candidate videos with the largest and smallest weights.** We visualize the resampling results of DIGAN trained on the Taichi-HD dataset with the 32 largest (most likely to select) and least (least likely to select) weights. We observe clear quality degradation in the samples with the smallest weights. *Best viewed with Acrobat Reader. Please check the website for videos.*

Experimental setups. We experiment with several video generation models, including GANs [37, 38], Transformer [106], and Diffusion models [91]. As these models are evaluated on different benchmarks, we also test with different datasets, including UCF-101 [235], Sky Time-lapse [236], Taichi-HD [237], and FaceForensics [319]. We use the official scripts and checkpoints to sample the videos. For each experiment, we generate a $8\times$ larger candidate set, i.e., $K = 16,384$ videos. To optimize the weights $\mathbf{w}$, we use gradient descent with an initial learning rate of 0.01 and a linear scheduler that decreases the learning rate by 0.1 at every 100 steps. We perform optimization for 300 steps, at which point the weighted FVD scores often converge.

We are especially interested in how much we can reduce the FVD score without improving the temporal quality. To achieve this, we convert each generated video into a *frozen* video by repeating its first frame 16 times. By doing so, we enforce all the videos to contain no motion so that the temporal quality cannot be improved through resampling. Motivated by the temporal sensitivity presented by the VideoMAE-v2 models, we also perform the experiments with the *VideoMAE-v2-SSv2* model as the feature extractor, where the resampling is done by optimizing its specific weights.

Table 4.2: **Results of probing the temporal perceptual null space of FVD.** We report FVDs of normal and frozen generated videos by random sampling (FVD) and resampling to minimize weighted FVD (FVD*). We color the FVD difference for better visualization: $< 20\%$, $20\% - 40\%$ and $> 40\%$. The drop of FVD on the frozen generated videos indicates the volume of the null space where FVD can be reduced without generating a meaningful motion. The gray background indicates the samples where resampling frozen videos can obtain similar or even better FVD than the random generation results with motions.

Model	Dataset	I3D Features			VideoMAE-v2-SSv2 Features		
		FVD	$\text{FVD}_{\text{w/o motion}}$	$\text{FVD}^*_{\text{w/o motion}}$	FVD	$\text{FVD}_{\text{w/o motion}}$	$\text{FVD}^*_{\text{w/o motion}}$
DIGAN [37]	UCF-101 [235]	562.36	1303.13	715.96(−45.1%)	378.19	951.59	859.57(−9.7%)
DIGAN [37]	Sky Time-lapse [236]	157.13	230.64	115.55(−49.9%)	174.79	408.17	362.84(−11.1%)
DIGAN [37]	Taichi-HD [237]	132.26	461.79	276.88(−40.0%)	313.84	578.61	523.20(−9.6%)
TATS [106]	UCF-101 [235]	329.92	1157.69	616.25(−46.8%)	388.79	908.95	805.88(−11.3%)
TATS [106]	Sky Time-lapse [236]	125.62	279.75	126.32(−54.8%)	213.33	375.74	353.15(−6.0%)
TATS [106]	Taichi-HD [237]	124.16	475.99	312.19(−34.4%)	274.81	587.31	530.86(−9.6%)
StyleGAN-V [38]	Sky Time-lapse [236]	56.63	206.56	104.27(−49.5%)	219.85	503.22	456.24(−9.3%)
StyleGAN-V [38]	FaceForensics [319]	56.22	353.79	242.04(−31.6%)	194.68	547.24	520.98(−4.8%)
PVDM [91]	UCF-101 [235]	348.81	1135.61	605.09(−46.7%)	369.14	1032.90	898.48(−13.0%)
PVDM [91]	Sky Time-lapse [236]	59.95	182.77	94.87(−48.1%)	142.50	429.06	395.79(−7.8%)

Observation on the resampling results. We show FVD scores before resampling in Figure 4.7 (a). We first observe a non-monotonic relationship between the FVD computed with I3D and VideoMAE-v2 features, potentially leading to different model rankings when using these two features. For FVD after resampling, we report the change ratio $\frac{\text{FVD}^* - \text{FVD}}{\text{FVD}}$ in Figure 4.7 (b). It illustrates a significant drop ($40\% - 70\%$) in FVD computed with I3D features after resampling, while FVD computed with VideoMAE-v2 features experiences a more moderate drop ($25\% - 60\%$).

We note clear quality differences after inspecting videos with the highest and lowest weights obtained from optimizing weighted FVD. An example of videos generated by DIGAN trained on the Taichi-HD dataset is shown in Figure 4.9. We attribute the different observations from the original paper [124] to the unstable performance of the video generator. Since the video generator sometimes generates nonrealistic videos, resampling is beneficial in selecting a group of higher-quality videos, yielding smaller FVD scores.

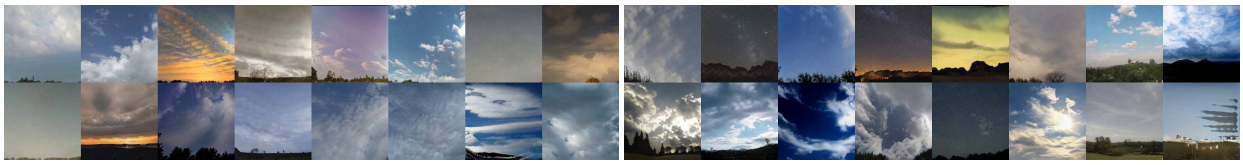
Temporal perceptual null space. Table 4.2 shows extensive results, including the FVD of original videos with motions, the FVD of frozen videos, and the weighted FVD of frozen videos after resampling. Despite the absence of motion in the generated videos, one can still reduce FVD by up to half by selectively choosing from the candidate videos when evaluating the Sky Time-lapse dataset. In the worst case, the same or even smaller FVD scores can be achieved compared with randomly selected generated videos *with* motions.

These findings highlight the pronounced content bias inherent in the FVD metric. Conversely, when computing the features for FVD using the VideoMAE-v2 model, which is sensitive to temporal quality, the gap significantly diminishes, and the FVD scores can hardly be decreased through resampling. This emphasizes that the FVD with VideoMAE-v2 has a much smaller temporal perceptual null space.

4.2.4 Case Study: Long Video Generation

Our experiments have revealed that FVD does not sufficiently account for motion in generated videos. We now dive into two case studies from previous works where FVD scores contradict human perception [37, 38, 106]. In both cases, the video generation models are trained on the Sky Time-lapse dataset, which is out-of-domain for the I3D model, and FVD has been shown not to perform well. In addition, both tasks generate longer videos than the standard 16-frame setting, making the motion artifacts more perceptible to humans. Nevertheless, FVD fails to capture these motion artifacts in both experiments.

Case study I [38]. To synthesize long videos, the StyleGAN-v model [38] employs convolutional layers with large reception fields to predict the parameters of the Fourier temporal encoding.



(a) Default StyleGAN-v. (b) StyleGAN-v with LSTM motion codes.

Figure 4.11: **Videos generated by StyelGAN-v and its LSTM variant.** The default StyleGAN-v synthesizes natural motions, while the variant with LSTM motion codes generates repeated patterns. *Best viewed with Acrobat Reader. Please check the website for videos.*

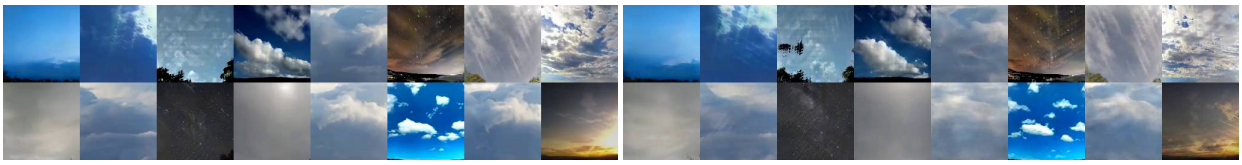
Table 4.3: StyleGAN-v model [38] with LSTM as the motion codes trained on the Sky Time-lapse dataset generate collapsed motions, whereas FVD computed on the 128 frames favors the results. We show that computing FVD with VideoMAE-v2 features calibrates the conclusion.

Frame #	FVD Feature	StylegGAN-v	w/ LSTM codes
16	I3D	120.11	136.65 (+16.54%)
	VideoMAE-SSv2	223.96	247.25(+23.29%)
	VideoMAE-K710	145.37	154.29(+8.92%)
128	I3D	190.82	172.71(−18.11%)
	VideoMAE-SSv2	332.80	616.74(+283.94%)
	VideoMAE-K710	155.51	191.48(+35.97%)

We reproduce one of its baselines by substituting such temporal encoding with an LSTM layer. For generating 128-frame videos, the default StyleGAN-V synthesizes realistic motions (Figure 4.11a), whereas the baseline with continuous LSTM codes leads to videos with noticeable motion collapse (Figure 4.11b).

We follow the original study's evaluation protocol and compute the FVD metric by feeding all the 128 frames to the I3D model, termed as FVD_{128}. Note that the I3D model was initially trained on 64 frames, while the global average pooling and convolutional architecture allow it to be applied to any video length. We also compute FVD_{128} with the VideoMAE. Since the VideoMAE uses a ViT with fixed-size positional encoding, we perform interpolation of positional encodings, similar to DINO [323].

Contrary to the visual evidence, we observe the same trend as noted by the authors that



(a) Frames 0 - 16. (b) Frames 128 - 144.

Figure 4.13: **Videos frames generated by DIGAN at different extrapolated time steps.** The initial 16 frames generated by DIGAN exhibit natural motions, while the extrapolated frames contain periodic artifacts. *Best viewed with Acrobat Reader. Please check the website for videos.*

Table 4.4: DIGAN [37] trained on the Sky Time-lapse dataset with extrapolated time steps generate periodic artifacts, whereas the FVD metric favors the results. We show that computing FVD with VideoMAE-v2 features calibrates the conclusion.

FVD Feature	Frames 0 - 16	Frames 128 - 144
I3D	155.58	141.82(−8.84%)
VideoMAE-SSv2	133.37	150.61(+12.9%)
VideoMAE-K710	250.70	259.29(+3.43%)

FVD_{128} computed using the I3D model is lower for the LSTM variant, compared to the original StyleGAN-v, as shown in Table 4.3. Upon computing FVD_{128} using VideoMAE-v2 features, both using SSv2 and K710, we have them to be in accordance with human preference, i.e., FVD_{128} for the LSTM baseline is much worse compared to the original StyleGAN-v.

Case study II [37, 106]. Though trained on 16-frame video clips, DIGAN can be applied to generate longer videos by extrapolating the temporal encodings. However, in the previous study [106], the authors have observed that motion artifacts in the form of repeated changes in the diagonal direction, while FVD computed on the 16-frame chunks, favor the artifacts.

Specifically, to evaluate the long video generation results, they compute FVD at strides of k frames, 16 frames at a time, for the entire video. We compute FVD on frames $0 \rightarrow 16$, $64 \rightarrow 82$, $128 \rightarrow 144$, and so on. The visualization of videos at $0 \rightarrow 16$ and $128 \rightarrow 144$ Figure 4.13 show more periodic artifacts at $128 \rightarrow 144$ compared to $0 \rightarrow 16$. In contrast, from Table 4.4, we see

that FVD computed using I3D features favors $128 \rightarrow 144$ frames, though FVD computed using VideoMAE features for both SSv2 and K710 follow human judgment.

Recent progress in photorealistic video generation using Diffusion models has enabled the creation of videos with extended durations (≥ 100 frames) [61, 63, 70, 72]. As a result, evaluating long video generation results has become increasingly important. However, according to the two real-world case studies above, FVD with I3D features does not reliably detect motion artifacts in long videos.

4.2.5 Discussion

In this paper, we have studied the bias of the FVD on the frame quality. With experiments spanning from synthetic video distortion, to resampling video generation results, to investigating real-world examples, we have concluded that FVD is highly insensitive to the temporal quality and consistency of the generated videos. We have verified the hypothesis that the bias originates from the content-biased video features and show that self-supervised features can mitigate the issues in all the experiments. We hope our work will draw more attention to studying video generation evaluation and designing better evaluation metrics.

Limitations. Several critical aspects of FVD remain underexplored. For example, in addition to the longer time duration, existing methods also generate megapixel resolution videos [61, 63, 70, 72]. However, to compute FVD (I3D or VideoMAE-v2), the video must be resized to a lower (e.g., 224x224) resolution. In addition, many methods choose to generate videos not limited to the square aspect ratio, e.g., 16:9, while the FVD metric always requires a square video as the input. Computing FVD using VideoMAE-v2 is limited by the quadratic cost of attention layers,

causing issues in evaluating longer video generation. Using efficient networks can reduce the cost, which we leave as future work.

Chapter 5: Applications

5.1 Overview

Although the text has become the ubiquitous condition for image and video generation, it also restricts controllability—what one can imagine is largely beyond what can be described in language. As a result, various applications have been enabled by improving the controllability of the model, such as image editing [169, 182, 324], customization [143, 162], layout conditioning [164, 166, 175, 179], finer-grained spatial control like sketch [163, 325], or just leveraging more complex text prompts [159]. In Section 5.2 of this chapter, we discuss a potential way to overcome the limitations of the text prompt. Specifically, we draw inspiration from the daily text editor by using a rich-text editor supporting formats such as font style, size, color, embedded image, and footnote. We can thus extract each word's attributes from rich text to enable local style control, explicit token reweighting, precise color rendering, and detailed region synthesis for image generation and editing.

Unlike image and video, many domains like sketch and 3D do not have the rich data souce. How to harness pretrained visual generative models in other domains like vector graphics [208, 326], textures [210, 212, 213], 3D [195], and even 4D [216, 217] and discriminative tasks such as correspondence estimation [327, 328], classification [329], and segmentation [330] has been an appealing problem. This requires repurposing diffusion models a general prior for

different vision tasks. However, the optimal way to leverage the pre-trained diffusion models is still unclear. The widely adopted way to compute guidance with diffusion models is through Score Distillation Sampling (SDS) [195, 331]. It feeds a perturbed proxy image (e.g. rendered from NeRF) to a pretrained diffusion model and computes the score to form the main part of the gradient. However, SDS has a number of characteristic artifacts that limit its utility for general-purpose applications. In Section 5.3 of this chapter, we inspect the cause of these artifacts in the lens of casting SDS as a Schrödinger Bridge (SB) problem. We develop a simple yet effective approach to improve SDS on several tasks.

5.2 Enriching Text-Conditioning with Rich Text

5.2.1 Introduction

The development of large-scale text-to-image generative models [4,23,44,48] has propelled image generation to an unprecedented era. The great flexibility of these large-scale models further offers users powerful control of the generation through visual cues [41, 45, 163] and textual inputs [169, 182]. Without exception, existing studies use *plain text* encoded by a pretrained language model to guide the generation. However, in our daily lives, it is rare to use only plain text when working on text-based tasks such as writing blogs or editing essays. Instead, a *rich text* editor [332, 333] is the more popular choice, providing versatile formatting options for writing and editing text. In this paper, we seek to introduce accessible and precise textual control from rich text editors to text-to-image synthesis.

Rich text editors offer unique solutions for incorporating conditional information separate from the text. For example, using the font color, one can indicate an *arbitrary* color. In contrast,

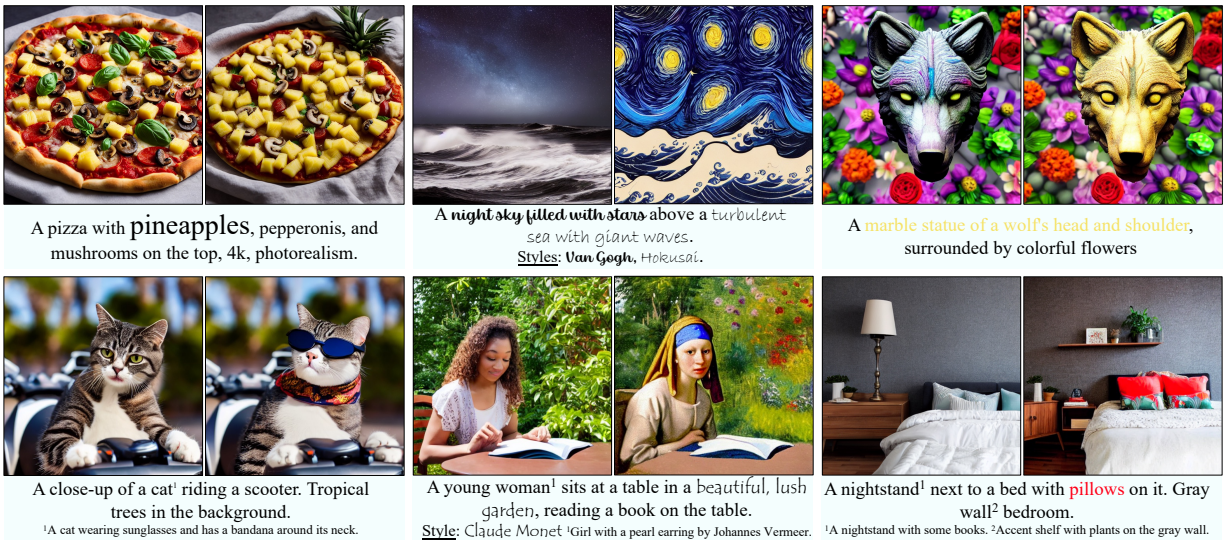


Figure 5.1: **Plain text (left image) vs. Rich text (right image)** Our method allows a user to describe an image using a rich text editor that supports various text attributes like font family, size, color, and footnote. Given these text attributes extracted from rich-text prompts, our method enables precise control of text-to-image synthesis regarding colors, styles, and object details compared to plain text.

describing the precise color with plain text proves more challenging as general text encoders do not understand RGB or Hex triplets, and many color names, such as 'olive' and 'orange', have ambiguous meanings. This font color information can be used to define the color of generated objects. For example, in Figure 5.1, a specific yellow can be selected to instruct the generation of a marble statue with that exact color.

Beyond providing precise color information, various font formats make it simple to augment the word-level information. For example, reweighting token influence [169] can be implemented using the font size, which is challenging to achieve with existing visual or textual interfaces. Rich text editors offer more options than font size – similar to how font style distinguishes the styles of individual text elements, we propose using it to capture the artistic style of specific regions. Furthermore, embedded images provide extra information in visual format. We utilize such images as the reference concepts for the generated objects. This intuitive design

enables personalized/customized generation.

But how can we use rich text? A straightforward implementation is to convert a rich-text prompt with detailed attributes into lengthy plain text and feed it directly into existing methods [48, 169, 182]. Unfortunately, these methods struggle to synthesize images corresponding to lengthy text prompts involving multiple objects with distinct visual attributes, as noted in a recent study [172]. They often mix styles and colors, applying a uniform style to the entire image. Furthermore, the lengthy prompt introduces extra difficulty for text encoders to interpret accurate information, making generating intricate details more demanding.

To address these challenges, our insight is to decompose a rich-text prompt into two components (1) a short plain-text prompt (without formatting) and (2) multiple region-specific prompts that include text attributes, as shown in Figure 5.2. First, we obtain the self- and cross-attention maps using a vanilla denoising process with the short *plain-text* prompt to associate each word with a specific region. Second, we create a prompt for each region using the attributes derived from *rich-text* prompt. For example, we use "mountain in the style of Ukiyo-e" as the prompt for the region corresponding to the word "mountain" with the attribute "font style: Ukiyo-e". For RGB font colors that cannot be converted to the prompts, we iteratively update the region with region-based guidance to match the target color. We apply a separate denoising process for each region and fuse the predicted noises to get the final update. During this process, regions associated with the tokens that do not have any formats are supposed to look the same as the plain-text results. Also, the overall shape of the objects should stay unchanged in cases such as only the color is changed. To this end, we propose to use region-based injection approaches.

We demonstrate qualitatively and quantitatively that our method generates more precise color, distinct styles, and accurate details compared to plain text-based methods. We conduct a

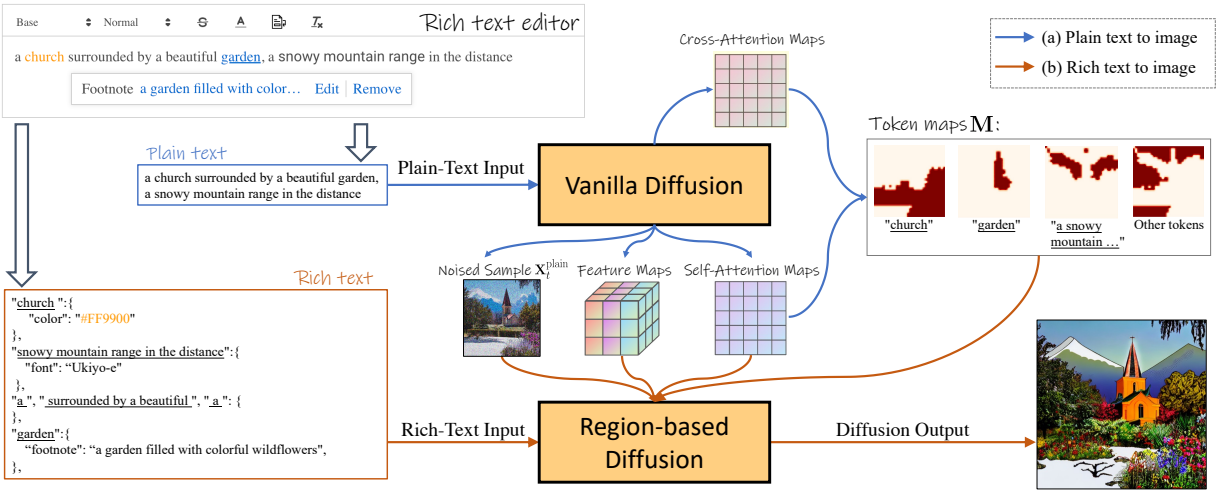


Figure 5.2: **Rich-text-to-image framework.** First, the plain-text prompt is processed by a diffusion model to collect self- and cross-attention maps, noised generation, and residual feature maps at certain steps. The token maps of the input prompt are constructed by first creating a segmentation using the self-attention maps and then labeling each segment using the cross-attention maps. Then the rich texts are processed as JSON to provide attributes for each token span. The resulting token maps and attributes are used to guide our region-based control. We inject the self-attention maps, noised generation, and feature maps to improve fidelity to the plain-text generation.

thorough quantitative evaluation by building a rich-text benchmark via collecting a diverse set of rich-text prompts with font color, style, and footnotes.

5.2.2 Rich Text to Image Generation

From writing messages on communication apps, designing websites [334], to collaboratively editing a document [335, 336], a rich text editor is often the primary interface to edit texts on digital devices. Nonetheless, only plain text has been used in text-to-image generation. To use formatting options in rich-text editors for more precise control over the black-box generation process [337], we first introduce a problem setting called *rich-text-to-image generation*. We then discuss our approach to this task.

5.2.2.1 Problem Setting

As shown in Figure 5.2, a rich text editor supports various formatting options, such as font styles, font size, color, and more. We leverage these text attributes as extra information to increase control of text-to-image generation. We interpret the rich-text prompt as JSON, where each text element consists of a span of tokens $\boldsymbol{e}_i$ (e.g., 'church') and attributes $\boldsymbol{a}_i$ describing the span (e.g., 'color:#FF9900'). Note that some tokens $\boldsymbol{e}_U$ may not have any attributes. Using these annotated prompts, we explore four applications: 1) local style control using *font style*, 2) precise color control using *font color*, 3) detailed region description using *footnotes* and *embedded images*, and 4) explicit token reweighting with *font sizes*.

Font style is used to apply a specific artistic style $\boldsymbol{a}_i^s$, e.g., $\boldsymbol{a}_i^s =$ 'Ukiyo-e', to the synthesis of the span of tokens $\boldsymbol{e}_i$. For instance, in Figure 5.1, we apply the Ukiyo-e painting style to the ocean waves and the style of Van Gogh to the sky, enabling the application of localized artistic styles. This task presents a unique challenge for existing text-to-image models, as there are limited training images featuring multiple artistic styles. Consequently, existing models tend to generate a *uniform* mixed style across the entire image rather than distinct local styles.

Font color indicates a specific color of the modified text span. Given the prompt "a red toy", the existing text-to-image models generate toys in various shades of red, such as light red, crimson, or maroon. The color attribute provides a way for specifying a *precise color* in the RGB color space, denoted as $\boldsymbol{a}_i^c$. For example, to generate a toy in fire brick red, one can change the font color to "a toy", where the word "toy" is associated with the attribute $\boldsymbol{a}_i^c = [178, 34, 34]$. However, as shown in the experiment section, the pretrained text encoder cannot interpret the RGB values and have difficulty understanding obscure color names, such as lime and orange.

Footnote provides supplementary explanations of the target span without hindering readability with lengthy sentences. Writing detailed descriptions of complex scenes is tedious work, and it inevitably creates lengthy prompts [338, 339]. Additionally, existing text-to-image models are prone to ignoring some objects when multiple objects are present [172], especially with long prompts. Moreover, excess tokens are discarded when the prompt's length surpasses the text encoder's maximum length, e.g., 77 tokens for CLIP models [260]. We aim to mitigate these issues using a footnote string $\boldsymbol{a}_i^f$.

Font size can indicate an object's importance, quantity, or size. We use a scalar $\boldsymbol{a}_i^w$ to denote the weight of each token.

Embedded images provide visual cues that are complementary to the textual information. Such visual information is more accurate than texts in describing the objects on the identity and low-level details. For example, by referring the word "cat" in "a cat doing homework" to a specific cat image, one can generate an image of their cat doing homework. Generating customized images of the reference concepts has been an active research question [143, 180, 340, 341, 341–346]. In contrast to free-form generation in prior work, we aim to preserve the original image structure in plain-text generation while synthesizing customized concepts.

5.2.2.2 Method

To utilize rich text annotations, our method consists of two steps, as shown in Figure 5.2. First, we compute the spatial layouts of individual token spans. Second, we use a new region-based diffusion to render each region's attributes into a globally coherent image.

Step 1. Token maps for spatial layout. Several works [41, 169, 172, 186, 190, 324, 347] have

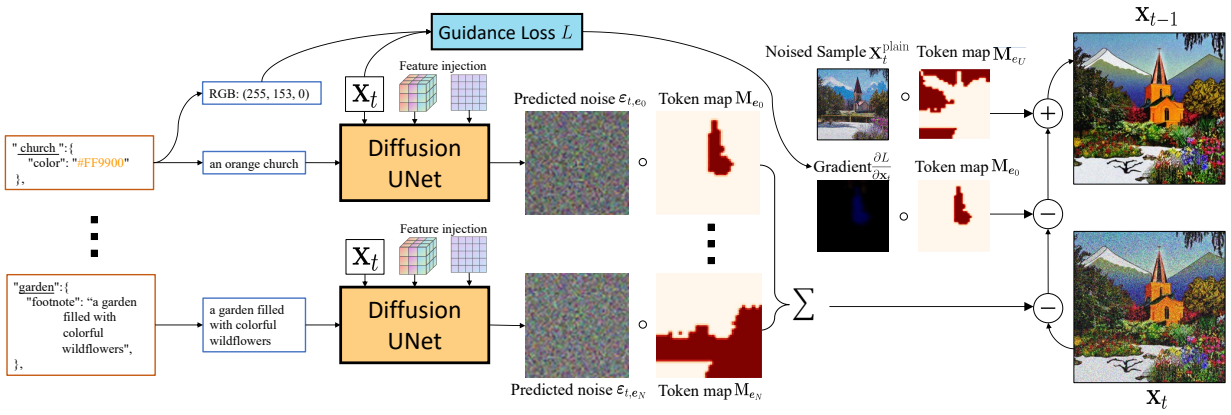


Figure 5.3: **Region-based diffusion.** For each element of the rich-text input, we apply a separate diffusion process to its region. The attributes are either decoded as a region-based guidance target (e.g. re-coloring the church), or as a textual input to the diffusion UNet (e.g. handling the footnote to the garden). The self-attention maps and feature maps extracted from the plain-text generation process are injected to help preserve the structure. The predicted noise $\epsilon_{t,\boldsymbol{e}_i}$, weighted by the token map $\mathbf{M}_{\boldsymbol{e}_i}$, and the guidance gradient $\frac{\partial L}{\partial \mathbf{x}_t}$ are used to denoise and update the previous generation $\mathbf{x}_t$ to $\mathbf{x}_{t-1}$. The noised plain text generation $\mathbf{x}_t^{\text{plain}}$ is blended with the current generation to preserve the exact content in those regions of the unformatted tokens.

discovered that the attention maps in the self- and cross-attention layers of the diffusion UNet characterize the spatial layout of the generation. Therefore, we first use the plain text as the input to the diffusion model and collect self-attention maps of size $32 \times 32 \times 32 \times 32$ across different heads, layers, and time steps. We take the average across all the extracted maps and reshape the result into 1024×1024. Note that the value at i^{th} row and j^{th} column of the map indicates the probability of pixel i attending to pixel j. We average the map with its transpose to convert it to a symmetric matrix. It is used as a similarity map to perform spectral clustering [348, 349] and obtain the binary segmentation maps $\widehat{\mathbf{M}}$ of size $K \times 32 \times 32$, where K is the number of segments.

To associate each segment with a textual span, we also extract cross-attention maps for each token w_j:

$$\mathbf{m}_j = \frac{\exp(\mathbf{s}_j)}{\sum_k \exp(\mathbf{s}_k)}, \tag{5.1}$$

where $\mathbf{s}_j$ is the attention score. We first interpolate each cross-attention map $\mathbf{m}_j$ to the same

resolution as $\widehat{\mathbf{M}}$ of 32×32. Similar to the processing steps of the self-attention maps, we compute the mean across heads, layers, and time steps to get the averaged map $\widehat{\mathbf{m}}_j$. We associate each segment with a texture span $\boldsymbol{e}_i$ following [186]:

$$\mathbb{M}_{\boldsymbol{e}_i} = \{\widehat{\mathbf{M}}_k \mid \left| \widehat{\mathbf{M}}_k \cdot \frac{\widehat{\mathbf{m}}_j - \min(\widehat{\mathbf{m}}_j)}{\max(\widehat{\mathbf{m}}_j) - \min(\widehat{\mathbf{m}}_j)} \right|_1 > \epsilon, \tag{5.2}$$

$$\forall j \; s.t. \; w_j \in \boldsymbol{e}_i\}, \tag{5.3}$$

where ϵ is a hyperparameter that controls the labeling threshold, that is, the segment $\widehat{\mathbf{M}}_k$ is assigned to the span $\boldsymbol{e}_i$ if the normalized attention score of any tokens in this span is higher than ϵ. We associate the segments that are not assigned to any formatted spans with the unformatted tokens $\boldsymbol{e}_U$. Finally, we obtain the *token map* in Figure 5.2 as below:

$$\mathbf{M}_{\boldsymbol{e}_i} = \frac{\sum_{\widehat{\mathbf{M}}_j \in \mathbb{M}_{\boldsymbol{e}_i}} \widehat{\mathbf{M}}_j}{\sum_i \sum_{\widehat{\mathbf{M}}_j \in \mathbb{M}_{\boldsymbol{e}_i}} \widehat{\mathbf{M}}_j} \tag{5.4}$$

Step 2. Region-based denoising and guidance. As shown in Figure 5.2, given the text attributes and *token maps*, we divide the overall image synthesis into several region-based denoising and guidance processes to incorporate each attribute, similar to an ensemble of diffusion models [143, 175]. More specifically, given the span $\boldsymbol{e}_i$, the region defined by its *token map* $\mathbf{M}_{\boldsymbol{e}_i}$, and the attribute $\boldsymbol{a}_i$, the predicted noise $\boldsymbol{\epsilon}_t$ for noised generation $\mathbf{x}_t$ at time step t is

$$\boldsymbol{\epsilon}_t = \sum_i \mathbf{M}_{\boldsymbol{e}_i} \cdot \epsilon_{t,\boldsymbol{e}_i} = \sum_i \mathbf{M}_{\boldsymbol{e}_i} \cdot D(\mathbf{x}_t, f(\boldsymbol{e}_i, \boldsymbol{a}_i), t), \tag{5.5}$$

where D is the pretrained diffusion model, and $f(\boldsymbol{e}_i, \boldsymbol{a}_i)$ is a plain text representation derived

from text span $\boldsymbol{e}_i$ and attributes a_i using the following process:

1. Initially, we set $f(\boldsymbol{e}_i, \boldsymbol{a}_i) = \boldsymbol{e}_i$.

2. If an embedded image is available, we convert it into a footnote $\boldsymbol{a}_i^f$ using the gradient-based discrete optimization [350].

3. If footnote $\boldsymbol{a}_i^f$ is available, we set $f(\boldsymbol{e}_i, \boldsymbol{a}_i) = \boldsymbol{a}_i^f$.

4. The style $\boldsymbol{a}_i^s$ is appended if it exists. $f(\boldsymbol{e}_i, \boldsymbol{a}_i) = f(\boldsymbol{e}_i, \boldsymbol{a}_i) +$ 'in the style of' $+ \boldsymbol{a}_i^s$.

5. The closest color name (string) of font color $\hat{\boldsymbol{a}}_i^c$ from a predefined set $\mathcal{C}$ is prepended. $f(\boldsymbol{e}_i, \boldsymbol{a}_i) = \hat{\boldsymbol{a}}_i^c + f(\boldsymbol{e}_i, \boldsymbol{a}_i)$. For example, $\hat{\boldsymbol{a}}_i^c$ = 'brown' for RGB color $\boldsymbol{a}_i^c$ = [136,68,20].

We use $f(\boldsymbol{e}_i, \boldsymbol{a}_i)$ as the original plain text prompt of Step 1 for the unformatted tokens $\boldsymbol{e}_U$.

Guidance. By default, we use classifier-free guidance [351] for each region to better match the prompt $f(\boldsymbol{e}_i, \boldsymbol{a}_i)$. In addition, if the font color is specified, to exploit the RGB values information further, we apply gradient guidance [62, 171, 352] on the current clean image prediction:

$$\widehat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}_t}{\sqrt{\bar{\alpha}_t}}, \tag{5.6}$$

where $\mathbf{x}_t$ is the noisy image at time step t, and $\bar{\alpha}_t$ is the coefficient defined by noise scheduling strategy [39]. Here, we compute an MSE loss $\mathcal{L}$ between the average color of $\widehat{\mathbf{x}}$ weighted by the *token map* $\mathbf{M}_{\boldsymbol{e}_i}$ and the RGB triplet $\boldsymbol{a}_i^c$. The gradient is calculated below,

$$\frac{d\mathcal{L}}{d\mathbf{x}_t} = \frac{d\| \sum_p (\mathbf{M}_{\boldsymbol{e}_i} \cdot \widehat{\mathbf{x}}_0) / \sum_p \mathbf{M}_{\boldsymbol{e}_i} - \boldsymbol{a}_i^c \|_2^2}{\sqrt{\bar{\alpha}_t} d\widehat{\mathbf{x}}_0}, \tag{5.7}$$

where the summation is over all pixels p. We then update $\mathbf{x}_t$ with the following equation:

$$\mathbf{x}_t \leftarrow \mathbf{x}_t - \lambda \cdot \mathbf{M}_{\boldsymbol{e}_i} \cdot \frac{d\mathcal{L}}{d\mathbf{x}_t}, \tag{5.8}$$

where λ is a hyperparameter to control the strength of the guidance. We use $\lambda = 1$ unless denoted otherwise.

Token reweighting with font size. Last, to re-weight the impact of the token w_j according to the font size $\boldsymbol{a}_j^w$, we modify its cross-attention maps $\mathbf{m}_j$. However, instead of applying direct multiplication as in Prompt-to-Prompt [169] where $\sum_j \boldsymbol{a}_j^w \mathbf{m}_j \neq 1$, we find that it is critical to preserve the probability property of $\mathbf{m}_j$. We thus propose the following reweighting approach:

$$\widehat{\mathbf{m}}_j = \frac{\boldsymbol{a}_j^w \exp(\mathbf{s}_j)}{\sum_k \boldsymbol{a}_k^w \exp(\mathbf{s}_k)}. \tag{5.9}$$

We can compute the token map (Equation 5.4) and predict the noise (Equation 5.5) with the reweighted attention map.

Preserve the fidelity against plain-text generation. Although our region-based method naturally maintains the layout, there is no guarantee that the details and shape of the objects are retained when no rich-text attributes or only the color is specified, as shown in Figure 5.14. To this end, we follow Plug-and-Play [324] to inject the self-attention maps and the residual features extracted from the plain-text generation process when $t > T_{\text{pnp}}$ to improve the structure fidelity. In addition, for the regions associated with the unformatted tokens $\boldsymbol{e}_U$, stronger content preservation is desired. Therefore, at certain $t = T_{\text{blend}}$, we blend the noised sample $\mathbf{x}_t^{\text{plain}}$ based on the

Figure 5.4: **Qualitative comparison on style control.** We show images generated by Prompt-to-Prompt, InstructPix2Pix, and our method using prompts with multiple styles. Only our method can generate distinct styles for both regions.

plain text into those regions:

$$\mathbf{x}_t \leftarrow \mathbf{M}_{\boldsymbol{e}_U} \cdot \mathbf{x}_t^{\text{plain}} + (1 - \mathbf{M}_{\boldsymbol{e}_U}) \cdot \mathbf{x}_t \tag{5.10}$$

5.2.3 Experimental Results

Implementation details. We use Stable Diffusion V1-5 [48] for our experiments. To create the token maps, we use the cross-attention layers in all blocks, excluding the first encoder and last decoder blocks, as the attention maps in these high-resolution layers are often noisy. We discard the maps at the initial denoising steps with $T > 750$. We use $K = 15$, $\epsilon = 0.3, T_{\text{pnp}} = 0.3, T_{\text{blend}} = 0.3$, and report the results averaged from three random seeds for all quantitative

Figure 5.5: **Qualitative comparison on precise color generation.** We show images generated by Prompt-to-Prompt [169], InstructPix2Pix [182], and our method using prompts with font colors. Our method generates precise colors according to either color names or RGB values. Both baselines generate plausible but inaccurate colors given color names, while neither understands the color defined by RGB values. InstructPix2Pix tends to apply the color globally, even outside the target object.

experiments.

Font style evaluation. We compute CLIP scores [260] for each local region to evaluate the stylization quality. Specifically, we create prompts of two objects and styles. We create combinations using 7 popular styles and 10 objects, resulting in 420 prompts. For each generated image, we mask it by the token maps of each object and attach the masked output to a black background. Then, we compute the CLIP score using the region-specific prompt. For example, for the prompt "a lighthouse (Cyberpunk) among the turbulent waves (Ukiyo-e)", the local CLIP score of the lighthouse region is measured by comparing its similarity with the prompt "lighthouse in the style of cyberpunk." In this example, we refer to "lighthouse" as the first region and "waves" as the second region.

Font color evaluation. We divide colors into three categories to evaluate a method's capacity to understand and generate a specific color. The *Common color* category contains 17 standard names, such as "red", "yellow", and "pink". The *HTML color* names are selected from the web

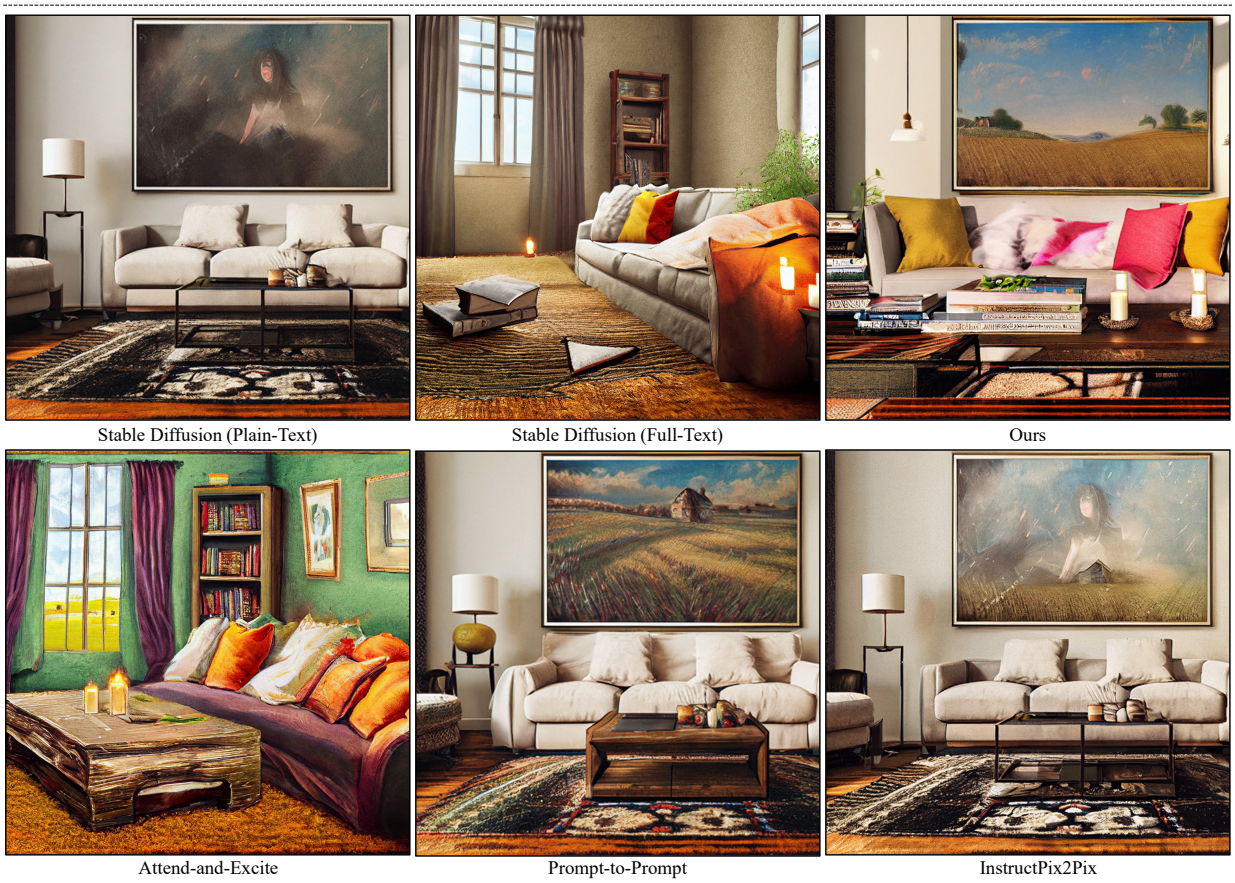


Figure 5.6: **Qualitative comparison on detailed description generation.** We show images generated by Attend-and-Excite, Prompt-to-Prompt, InstructPix2Pix, and our method using complex prompts. Our method is the only one that can generate all the details faithfully.

color names [1] used for website design, such as "sky blue", "lime green", and "violet purple". The *RGB color* category contains 50 randomly sampled RGB triplets to be used as "color of RGB values $[128, 128, 128]$". To create a complete prompt, we use 12 objects exhibiting different colors, such as "flower", "gem", and "house". This gives us a total of $1,200$ prompts. We evaluate color accuracy by computing the mean L2 distance between the region and target RGB values. We also compute the minimal L2 distance as sometimes the object should contain other colors for fidelity, e.g., the "black tires" of a "yellow car".

[1] `https://simple.wikipedia.org/wiki/Web_color`

Figure 5.7: **Qualitative comparison on token reweighting.** We show images generated by our method and Prompt-to-Prompt using token weight of 13 for 'mushrooms'. Prompt-to-Prompt suffers from artifacts due to the large weight. Heuristic methods like repeating and parenthesis do not work well.

Figure 5.8: **Qualitative results on customized concept generation.** We underline the modified concepts in the prompts and display the reference image on the top right of each rich-text generation result. Our method is able to synthesize the image with the object according to the reference image without changing the overall structure.

Footnote evaluation. Most existing studies on text-to-image generation [23, 44, 169] are only evaluated on the relatively short prompts [137, 138]. However, long text prompts are still useful for accurately describing a scene.

To understand how well a model performs when a lengthy text prompt is given, we collect a set of long prompts using the GPT-4 model [353]. To improve the stability and quality of the collected prompts, we ask the language model first to generate a shorter caption for a scene and then generate detailed descriptions for the objects that appear in the scene. Such a format also makes it easy to convert to rich-text prompts. We manually create a few hierarchical descriptions as the in-context examples.

Table 5.1: The statistics of text prompts collected in the previous benchmarks or adopted in the previous studies.

Benchmark or paper	Number of words			Number of tokens		
	Avg	Max	Min	Avg	Max	Min
DALLE [23]	15.8	36	6	18.35	42	9
Prompt-to-Prompt [169]	7.43	12	2	8.56	14	3
DrawBench [44]	11.37	51	1	14.03	57	3
HRS Bench [137]	12.85	36	1	14.94	42	3
TIFA [138]	11.46	67	3	11.63	82	3
ParaImage-Big [161]	132.9	-	-	-	-	-
ParaImage-Small [161]	70.6	-	-	-	-	-
Ours	46.56	68	19	57.68	90	22

After collecting the results from GPT-4, we then manually filter and edit the individual descriptions. For example, we remove the descriptions of objects not shown in the scene. We collect 100 prompts in total. Then, we use GPT-4 to produce single, long, plain-text prompts and write scripts to produce rich-text prompts from these hierarchical prompts. We summarize the statistics of the plain-text prompts in Table 5.1. Note that the average length of the collected prompts is around $4\times$ longer than the prior arts and closer to the concurrent work [161].

To evaluate the alignment between the generated images and the lengthy text prompts, we follow the process in TIFA [138] to sample question-answer pairs for both scene and object prompts using language models. For example, for the scene prompt "a nightstand next to a bed in the bedroom," the question-and-answer pair could be "Q: Is there a bed? A: Yes." Through this process, we collect 2921 pairs. Then, either a VQA model or human annotators can answer the question according to the generated image and check whether it is consistent with the ground truth answer.

We manually modify the questions based on the object prompts to reflect that the object is part of the scene. For example, for the object prompt "a nightstand with some books," we modify

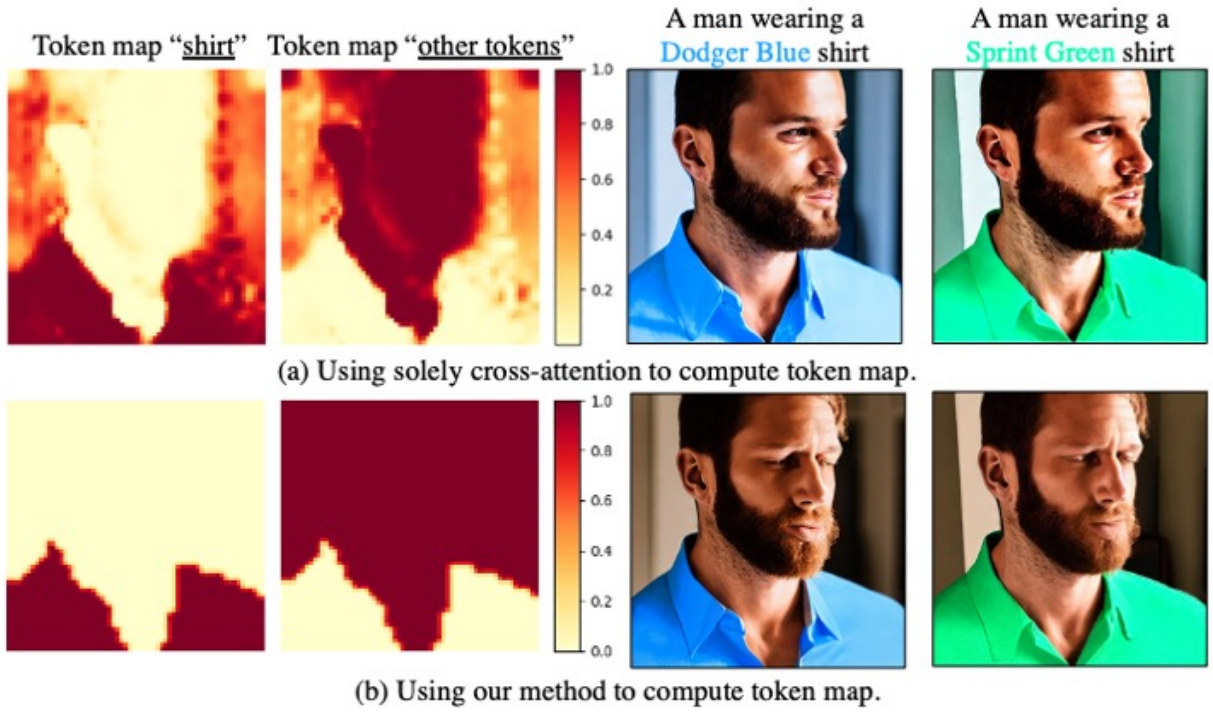


Figure 5.9: **Ablation of token maps.** Using solely cross-attention maps to create token maps leads to inaccurate segmentations, causing the background to be colored in an undesired way.

the question "Are there books?" to "Are there books on the nightstand?" We manually review and drop low-quality, irrelevant, or repeated questions. For example, we drop the pair "Q: What is in the bedroom? A: A bed." since "a nightstand" or other reasonable objects could be the correct answer. Instead, we prefer to ask the more concrete question, "Is there a bed in the bedroom?" After the manual processing, we obtain 1974 question-answer pairs in total.

Baselines. For font color and style, we quantitatively compare our method with two strong baselines, Prompt-to-Prompt [169] and InstructPix2Pix [182]. When two instructions exist for each image in our font style experiments, we apply them in parallel (InstructPix2Pix-para) and sequential manners (InstructPix2Pix-seq). For re-weighting token importance, we visually compare with Prompt-to-Prompt [169] and two heuristic methods, repeating and adding parentheses. For complex scene generation with footnotes, we also compare with Attend-and-Excite [172].

Table 5.2: **Quantitative evaluation of long prompt generation.**. We report the percentage of the time that the VQA output is aligned with the answer to a question regarding the generation. Our method consistently improves Stable Diffusion models.

	Stable Diffusion	Stable Diffusion XL
Plain-text generation	72.86 ± 1.13	79.05 ± 0.41
Rich-text generation	73.24 ± 1.00	81.08 ± 0.66

5.2.3.1 Quantitative Comparison

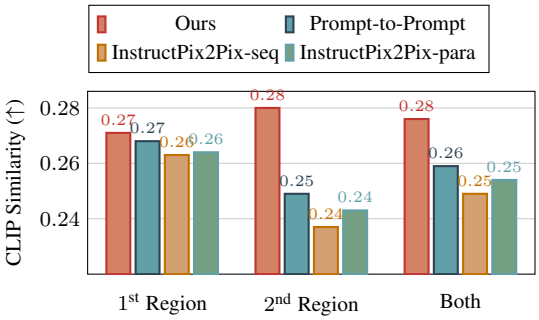


Figure 5.10: **Quantitative evaluation of local style control.** We report the CLIP similarity between each stylized region and its region prompt. Our method achieves the best stylization.

We report the local CLIP scores computed by a ViT-B/32 model in Figure 5.10. Our method achieves the best overall CLIP score compared to the two baselines. This demonstrates the advantage of our region-based diffusion method for localized stylization. To further understand the each model's capacity for generating multiple styles, we report the metric on each region. Prompt-to-Prompt and InstructPix2Pix-para achieve a decent score on the 1st Region, i.e., the region first occurs in the sentence. However, they often fail to fulfill the style in the 2nd Region. We conjecture that the Stable Diffusion model tends to generate a uniform style for the entire image, which can be attributed to single-style training images. Furthermore, InstructPix2Pix-seq performs the worst in 2nd Region. This is because the first instruction contains no information about the second region, and the second region's content could be compromised when we apply the first instruction.

We show quantitative results of precise color generation in Figure 5.11. The distance of *HTML color* is generally the lowest for baseline methods, as they provide the most interpretable textual information for text encoders. This aligns with our expectation that the diffusion model can handle simple color names, whereas they struggle to handle the RGB triplet. Our rich-text-to-image generation method consistently improves on the three categories and two metrics over the baselines.

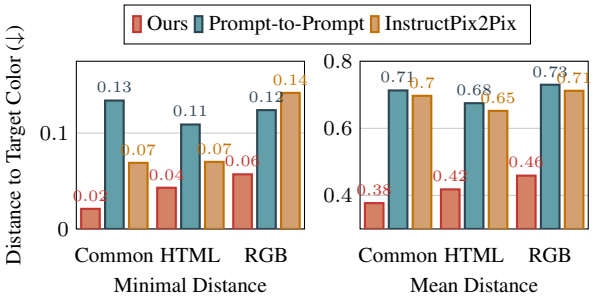


Figure 5.11: **Quantitaive evaluation on precise color generation.** Distance against target color is reported (lower is better). Our method consistently outperforms baselines.

In Table 5.2, we follow TIFA [138] to evaluate the long prompt generation using either Stable Diffusion or Stable Diffusion XL as the base models. Specifically, we generate 5 images for each text prompt using different random seeds and use the *mplug-large* model [354] to generate the answers to each benchmark question. We report how often the generated answers are consistent with the benchmark answers by averaging across different seeds. We also report the standard deviation. Our rich-text method consistently improves the generation quality with different base models when a lengthy prompt is given. Moreover, we find that the Stable Diffusion XL significantly outperforms the Stable Diffusion model.

5.2.4 Visual Comparison

Precise color generation. We show qualitative comparison on precise color generation in Figure 5.5. InstructPix2Pix [182] is prone to create global color effects rather than accurate local

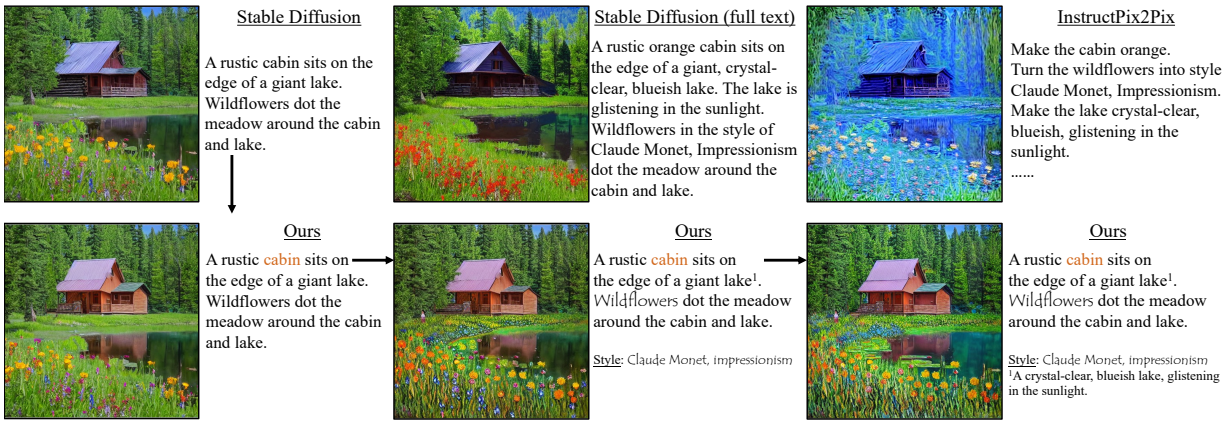


Figure 5.12: **Our workflow.** (top left) A user begins with an initial plain-text prompt and wishes to refine the scene by specifying the color, details, and styles. (top center) Naively inputting the whole description in plain text does not work. (top right) InstructPix2Pix [182] fails to make accurate editing. (bottom) Our method supports precise refinement with region-constrained diffusion processes. Moreover, our framework can naturally be integrated into a rich text editor, enabling a tight, streamlined UI.

control. For example, in the flower results, both the vase and background change to the target colors. Prompt-to-Prompt [169] provides more precise control over the target region. However, neither Prompt-to-Prompt nor InstructPix2Pix can generate precise colors. In contrast, our method can generate precise colors for all categories and prompts.

Local style generation. Figure 5.4 visually compares local style generation. When applying InstructPix2Pix-seq, the style in the first instruction dominates the entire image and undermines the second region. Similar to our observation in the quantitative evaluation, our baselines tend to generate the image in a globally uniform style instead of distinct local styles for each region. In contrast, our method synthesizes the correct styles for both regions. One may suggest independently applying baselines with two stylization processes and composing the results using token maps.

Complex scene generation. Figure 5.6 shows comparisons on complex scene generation. Attend-and-Excite [172] uses the tokens missing in the full-text generation result as input to fix the miss-

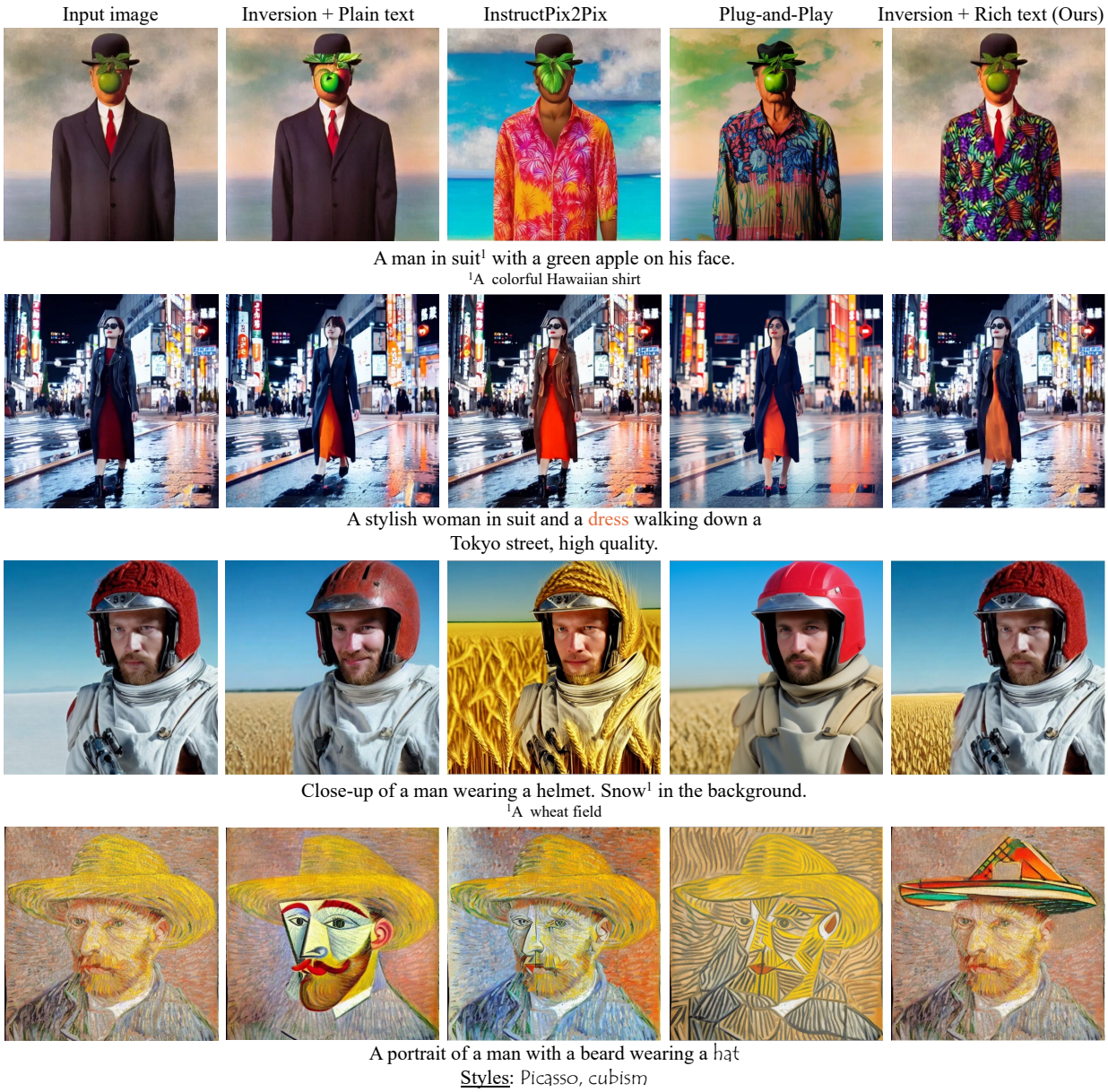


Figure 5.13: **Image editing with rich text.** We show that with the off-the-shelf inversion techniques, our methods easily slot in as an alternative to plain-text editing. Using rich text with our region-based guidance and diffusion allows more precise control in editing real images than existing methods.

ing objects, like the coffee table and carpet in the living room example. However, it still fails to generate all the details correctly, e.g., the books, the painting, and the blanket. Prompt-to-Prompt [169] and InstructPix2Pix [182] can edit the painting accordingly, but many objects, like the colorful pillows and stuff on the table, are still missing. In contrast, our method faithfully

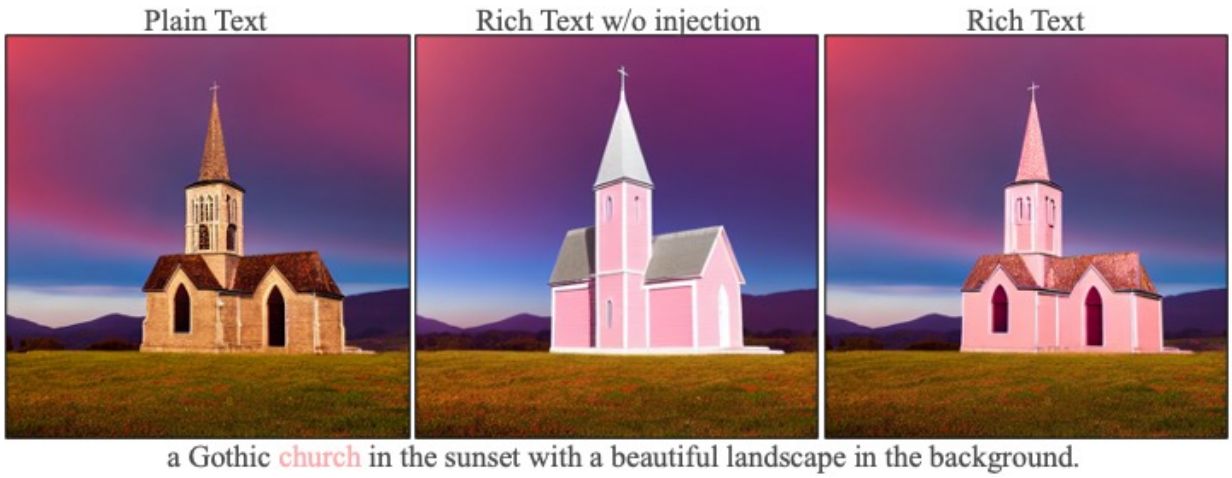


Figure 5.14: **Ablation of injection method.** We show images generated based on plain text and rich text with or without injection methods. Injecting features and noised samples help preserve the structure of the church and unformatted token regions.

synthesizes all these details described in the target region.

Token importance control. Figure 5.7 shows the qualitative comparison on token reweighting. When using a large weight for 'mushroom,' Prompt-to-Prompt generates clear artifacts as it modifies the attention probabilities to be unbounded and creates out-of-distribution intermediate features. Heuristic methods fail when adding more mushrooms, while our method generates more mushrooms and preserves the quality.

Customized concept generation. We show reference-driven image generation in Figure 5.8. Our method can generate the customized concepts specified by the embedded images without making undesired changes to the plain-text generation, such as the street scene on the left and the dog's pose on the right.

Interactive editing. In Figure 5.12, we showcase a sample workflow to illustrate our method's interactive strength and editing capacity over InstructPix2Pix [182].

Real image editing. We also explore using rich text as an alternative to plain text in real image

editing [170, 182, 324]. Given a real image, we manually create the text prompt and use the off-the-shelf diffusion inversion methods to obtain the noise latent that reconstructs the image [355, 356]. Unlike the image generation setting, the source image is already available here. We thus use the state-of-the-art grounded segmentation methods [357–359] to produce the token maps. This is generally more robust than the automatic methods since the manually created text prompts for inversion may not accurately describe the image as the model expects.

We adopt the same configuration as the image generation for the rest, including the region-based guidance denoising and injection methods. We compare with existing editing methods, including inversion and editing with plain text [355], InstructPix2Pix [182], and Plug-and-Play [324].

As shown in Figure 5.13, the rich text allows editing the regions' color, style, and content with accurate controllability. Specifically, all existing methods struggle with preserving the details in the non-edited regions. For example, in the Van Gogh portrait example, all the baselines change the styles of areas other than the hat. Instead, our injection method and the region information in the token masks greatly improve the fidelity in these regions. In addition, as we allow direct usage of the color RGB information, we edit the dress color (in the example of the second row) more accurately.

5.2.4.1 Ablation Study

Generating token maps solely from cross-attention. The other straightforward way to create token maps is to use cross-attention maps directly. To ablate this, we first take the average of cross-attention maps across heads, layers, and time steps and then take the maximum across tokens. Finally, we apply softmax across all the spans to normalize the token maps. How-

ever, as shown by the example in Figure 5.9, since the prompt has no correspondence with the background, the token map of "shirt" also covers partial background regions. Note that simple thresholding is ineffective as some regions still have high values, e.g., the right shoulder. As a result, the target color *bleeds* into the background. Our methods obtain more accurate token maps and, consequently, more precise colorization.

Ablation of the injection methods. To demonstrate the effectiveness of our injection method, we compare image generation with and without it in Figure 5.14. In the font color example, we show that applying the injection effectively preserves the shape and details of the target church and the structure of the sunset in the background. In the footnote example, we show that the injection keeps the look of the black door and the color of the floor.

5.2.5 Conclusion

In this paper, we have expanded the controllability of text-to-image models by incorporating rich-text attributes as the input. We have demonstrated the potential for generating images with local styles, precise colors, different token importance, reference images, and complex descriptions. Nevertheless, numerous formatting options remain unexplored, such as bold/italic, hyperlinks, spacing, and bullets/numbering. Also, there are multiple ways to use the same formatting options. For example, one can use font style to characterize the shape of the objects. We hope this paper encourages further exploration of integrating accessible user interfaces into text-based content creation tasks, even beyond images.

5.3 Score Distillation as a General Image Prior

5.3.1 Introduction

Diffusion models have shown tremendous success in modeling complex data distributions like images [41, 46, 47, 63], videos [53, 61] and robot action policies [360]. In domains where data is plentiful, they produce state-of-the-art results. Many data modalities, however, cannot enjoy the same scaling benefits due to their lack of sufficiently large datasets. In these cases, it is useful to exploit diffusion models trained on domains with rich data sources as a prior in an optimization framework. Score Distillation Sampling (SDS) [195, 331] and its variants [197, 202, 204] are a widely adopted way to optimize parametric images, *i.e.*, images produced by a model like NeRF, with a pre-trained diffusion model. Despite being applicable to a wide range of applications, SDS is also known to suffer from several significant artifacts, such as oversaturation and oversmoothing. As such, several variants have been proposed to alleviate these artifacts [202, 204, 361], often at the cost of efficiency, diversity, or other artifacts.

In this paper, we investigate the core issues with SDS by casting the class of score distillation optimization problems as a Schrödinger Bridge (SB) problem [362–365], which finds the optimal transport between two distributions. Specifically, given some images from the current optimized distribution (*e.g.*let@tokeneonedot, renderings from a NeRF), applying the transport maps them to their pair images in a target distribution (*e.g.*let@tokeneonedot, text-conditioned natural image distribution). The density flow formed by these mappings is transport-optimal, as defined in the SB problem. In an optimization framework, the difference between paired source and target samples, computed with an SB, can be used as a gradient to update the source.

Su *et al*let@tokeneonedot [366] have shown that this path can be explicitly solved using two pre-trained diffusion models. We show that one can also compose these models as an optimizer to approximate transport paths on the fly. Under this framework, we can understand SDS and its variants as approximating a source-to-target distribution bridge with the difference of two denoising directions. The denoising scores point to the source and target distributions respectively, with the source representing the current optimized image that updates with each optimization step.

This framing reveals two sources of errors. First, these methods are a first-order approximation of the diffusion bridge. Specifically, Gaussian noise is sampled to perturb the current optimized image, and single denoising steps, instead of the full PF-ODE simulation, are used to estimate the transport. This induces error in estimating the desired path. Recent works [367,368] that use multi-step estimation can be explained as mitigating this error. Second, estimating the denoising direction to the current source distribution is non-trivial, since the current optimized image may not necessarily look like a real image (*e.g*let@tokeneonedot, initializing with Gaussian noise or starting from a render of an untextured 3D model). Our analysis reveals that SDS approximates the current distribution with the unconditional image distribution, which is not accurate and results in a *distribution mismatch error*. We show that recent SDS variants [202, 204, 361] can be seen as proposals to improve this distribution mismatch error.

Finally, our analysis motivates a simple method that rectifies the distribution mismatch issue without additional computational overhead. Our insight is that the large-scale text-to-image diffusion models learn from billions of caption-image pairs [64], where a breadth of image corruptions are present in their training sets. They are also equipped with powerful pre-trained text encoders, which empower the models with zero-shot capacity in generating unseen concepts [23, 42]. As such, simply describing the current source distribution with text, even if it is

not part of the real image manifold, can approximate the distribution of the current optimized image, leading to improved transport paths. Our simple and efficient solution can be easily applied to any existing application that uses SDS. We show that it consistently improves the visual quality in the desired domain. We comprehensively compare our approach with standard distillation sampling methods over several generation tasks, where our approach matches or outperforms the baselines.

Our contributions are as follows:

- We propose to cast the problem of using a pre-trained diffusion model as a prior in an optimization problem as solving the Schrödinger Bridge (SB) problem between two image distributions. Specifically, it can be seen as bridging the distribution of the current optimized image to the target distribution under a dual-bridge framework.
- We analyze recent SDS-based methods under the lens of our framework and explain the pros and cons of the individual methods.
- Our analysis motivates a simple yet effective alternative to SDS by using textual descriptions to specify the current optimized image distribution. It achieves consistently more realistic results than SDS, producing quality comparable with VSD [202] without its computational overhead. We compare various generation tasks to show its wall-clock efficiency and quality generations against state-of-the-art methods.

5.3.2 Method

In this section, we present an analytical framework that casts the score distillation sampling (SDS) family of methods as instantiations of a Schrödinger Bridge problem. We show that many

recent SDS based methods can be interpreted as an online solver for the problem. That is, each SDS optimization step is a first-order approximation of a dual diffusion bridge formed by two probability flow (PF) ODEs [366]. We analyze SDS and its variants under this general framework. Then, we present a simple solution based on the analysis, which leads to significant quality improvement with little extra computational overhead.

5.3.2.1 Background

Diffusion models define a forward "noising" process that degrades data samples $\mathbf{x}$ gradually from the image distribution to noised samples $\mathbf{z}_t$, and eventually the i.i.d. Gaussian distribution [39,40]. This process is indexed by timesteps t, where $t = 1$ indexes the full Gaussian noise distribution and $t = 0$ indexes the data distribution. A diffusion model, parameterized by ϕ, is then trained to reverse this encoding process, iteratively transforming the noise distribution into the data distribution with the following denoising objective:

$$\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x}) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[w(t)\left\|\epsilon_\phi\left(\alpha_t\mathbf{x} + \sigma_t\epsilon; y, t\right) - \epsilon\right\|_2^2\right], \tag{5.11}$$

where $w(t)$ is a loss weighting function, y is a conditioning text prompt, and α_t and σ_t are hyperparameters from the predefined noise schedule.

Probability Flow ODE. Denoising score matching [264,266,369] shows that the diffusion model denoising prediction can be rewritten as a score vector field:

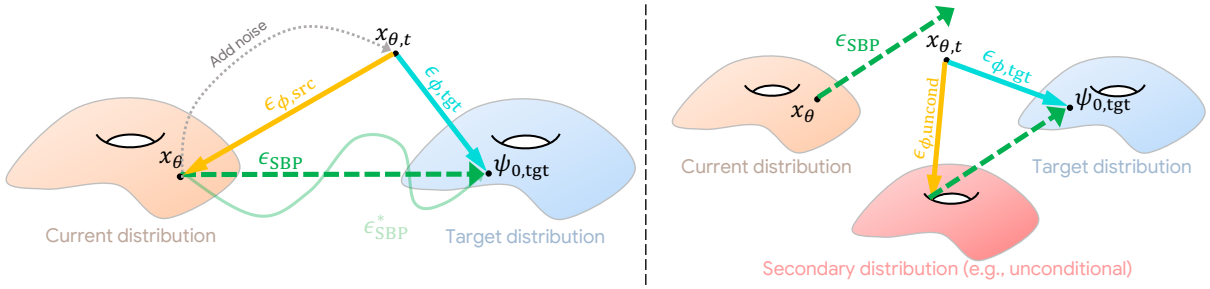
$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) = -\frac{1}{\sqrt{1-\alpha_t}}\epsilon_t. \tag{5.12}$$

Because of its special connection to marginal probability densities, the resulting ODE is named the probability flow (PF) ODE with the following expression:

$$dx = [f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x}))]dt, \tag{5.13}$$

where $f(\mathbf{x}, t)$ and $g(t)$ are pre-defined schedule parameters. This PF-ODE can be solved deterministically [370], mapping a noise sample to its corresponding data sample through the reverse process and the opposite through the forward process (inversion). This cycle-consistent conversion between image and latent representations is important in establishing dual diffusion implicit bridges.

Dual Diffusion Implicit Bridges. Dual Diffusion Implicit Bridges (DDIBs) [366] compose a diffusion inversion and generation process for solving image-to-image translation problems without requiring a paired image dataset. Instead, DDIBs use two diffusion models trained on different domains (or, analogously, one model with two different text conditions). DDIB inverts the source image into a noise latent via the forward PF-ODE and then decodes the latent in the target domain via the reverse PF-ODE. DDIBs can be interpreted as a concatenation of the Schrödinger Bridges from source-to-latent and latent-to-target, hence the dual bridges in its name. DDIBs enable solving transport between two distributions using a single pre-trained diffusion model. We build on this insight in an optimization context.



(a) Score distillation as a bridge between two distributions.

(b) Distribution mismatch error.

Figure 5.15: **Optimization with diffusion models as approximation of a Schrödinger Bridge Problem (SBP).** (a) We propose to formulate optimization with diffusion models as bridging the distribution of the current optimized image x_θ to the target distribution under a dual-bridge framework (a). Current methods can be interpreted as approximating the optimal transport ϵ^*_{SBP} between these distributions via the difference between projections of a noised image $x_{\theta,t}$ onto the two distributions. This analysis reveals two sources of error: (1) these gradients are linear approximations of the optimal path, as illustrated in (a), and (2) the source distribution used for computing this approximation (*e.g.*, the unconditional distribution in SDS [195]) may not be aligned with the current distribution, illustrated in (b).

5.3.2.2 Optimization with Diffusion Model Approximates a Dual Schrödinger Bridge

Many generative vision tasks involve optimizing corrupted images to the image manifold. For example, in 3D generation, a 3D representation like NeRF is optimized to render natural images matching a prescribed text prompt. Methods like SDS enable this by using a pre-trained diffusion model as a prior. We propose formulating such optimization problems as solutions to an instantiation of a Schrödinger Bridges Problem (SBP). SBP finds cost-optimal paths between a source image distribution p_{src} and a target image distribution p_{tgt} [371, 372]. Optimizing a parametrized image toward the natural image distribution can be cast as finding the optimal paths between the current optimized image(s) and the natural image distribution. Instead of solving this problem directly, which would require training a generative model from scratch [372–374], we show that pre-trained diffusion models can be exploited as an optimizer that approximates the

path. Further, the gradient computed by the existing score distillation methods can be viewed as the first-order approximation of this path. This formulation is illustrated in Figure 5.15.

Let $\mathbf{x}_\theta \in \mathbb{R}^d$ represent a parametric image, *i.e*let@tokeneonedot, an image produced differentiably by a model with parameter θ, such as a NeRF. To leverage the pretrained diffusion model, we add noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to obtain a latent at timestep t:

$$\mathbf{x}_{\theta,t} = \alpha_t \mathbf{x}_\theta + \sigma_t \epsilon \tag{5.14}$$

Suppose that $\psi_{t',\text{src}}$ and $\psi_{t',\text{tgt}}$ denote the paths obtained by solving the PF ODE as in Eq. 5.13 from t to 0, both starting from $\mathbf{x}_{\theta,t}$, such that $\psi_{0,\text{src}} \in p_{\text{src}}$, $\psi_{0,\text{tgt}} \in p_{\text{tgt}}$, $\psi_{t,\text{src}} = \psi_{t,\text{tgt}} = \mathbf{x}_{\theta,t}$. This forms a dual diffusion bridge [366] from $\psi_{0,\text{src}}$ to $\psi_{0,\text{tgt}}$. We approximate this path *per-iteration* using a pretrained diffusion model. We denote the displacement of this path as:

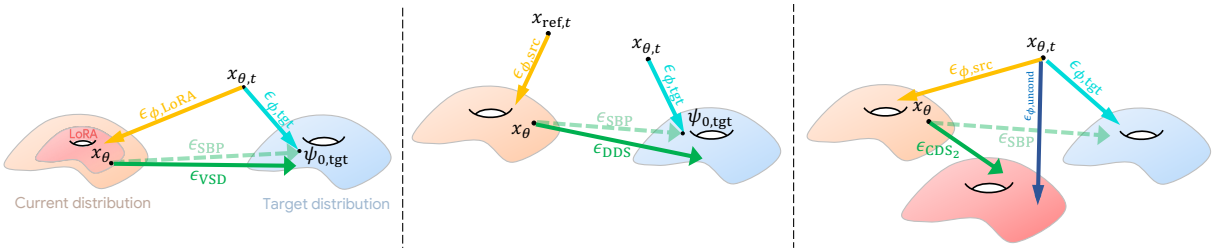
$$\epsilon^*_{\text{SBP}} = \psi_{0,\text{tgt}} - \psi_{0,\text{src}}. \tag{5.15}$$

Fully simulating this bridge involves solving two PF ODEs, which invokes dozens of neural function evaluations (NFEs) to estimate the gradient of each iteration. Instead, one can estimate each half of the bridge with a single-step prediction by computing two denoising directions $\epsilon_{\phi,\text{src}}$ and $\epsilon_{\phi,\text{tgt}}$. We thus obtain a first-order approximation of a dual diffusion bridge with the difference vector:

$$\epsilon_{\text{SBP}} = \epsilon_{\phi,\text{tgt}} - \epsilon_{\phi,\text{src}}, \tag{5.16}$$

which is subject to the following sources of errors.

1. **First-order approximation error**. Instead of performing full PF-ODE simulations, the



(a) Variational Score Distillation (b) Delta Distillation Sampling (c) Classifier Score Distillation (2nd term)

Figure 5.16: **Comparision of SDS variants under our analysis**. We illustrate the major gradient components of different SDS variants and provide a straightforward comparison with ϵ_{SBP}.

single-step noising and prediction are less accurate and induce errors. Recent work ISM [367] can be interpreted as reducing this error with a multi-step simulation to obtain $\mathbf{x}_{\theta,t}$.

2. **Source distribution mismatch**. The dual diffusion bridge relies on $\epsilon_{\phi,\text{src}}$ accurately estimating the distribution of the current sample, $\mathbf{x}_\theta$. A series of works can be viewed as improving this error [200, 202, 204] by computing more accurate $\epsilon_{\phi,\text{src}}$.

We show that $\epsilon_{\phi,\text{tgt}} - \epsilon_{\phi,\text{src}}$ is an effective gradient when both the source and target distribution are well expressed. Next, we discuss the popular score distillation methods under this analysis. We argue that their characteristic artifacts can largely be understood due to the errors above.

5.3.2.3 Analyzing Existing Score Distillation Methods

We analyze SDS and its variants through our framework by inspecting each component in the computed gradient. For notation, y_{tgt} is the text prompt representing the target distribution, and $\varnothing$ denotes the unconditional prompt. For each method, we present its gradient update and discuss its implications.

Score Distillation Sampling [195]:

$$\epsilon_{\text{SDS}} = \epsilon_\phi\left(\mathbf{x}_{\theta,t}; \varnothing, t\right) + s \cdot \left(\epsilon_\phi\left(\mathbf{x}_{\theta,t}; y_{\text{tgt}}, t\right) - \epsilon_\phi\left(\mathbf{x}_{\theta,t}; \varnothing, t\right)\right) - \epsilon,$$

where s is the strength of classifier-free guidance. When s is small, the ϵ functions as an averaging term to regress the image to the mean. However, the SDS gradient has been shown to work best with extreme values of classifier-free guidance s like 100. We can rewrite the gradient to emphasize how the conditional-unconditional delta dominates at high CFG scales.

$$\epsilon_{\text{SDS}} = \underbrace{s \cdot \left(\epsilon_\phi\left(\mathbf{x}_{\theta,t}; y_{\text{tgt}}, t\right) - \epsilon_\phi\left(\mathbf{x}_{\theta,t}; \varnothing, t\right)\right)}_{\text{Dominant when } s \gg 1} + \epsilon_\phi\left(\mathbf{x}_{\theta,t}; \varnothing, t\right) - \epsilon,$$

Experimentally, we produce very similar results at high CFG with or without the non-dominant terms. We argue that SDS should be interpreted through the dominant term, which fits within our analysis. Under this interpretation, the unconditional direction $\phi\left(\mathbf{x}_{\theta,t}; \varnothing, t\right)$ approximates the source distribution of $\mathbf{x}_\theta$ poorly, instead representing images of any identity with low contrast and geometric artifacts. Figure 5.15(b) illustrates the effect of a poor approximation. The bridge from the unconditional to conditional distribution leads to the characteristic oversaturation and smoothing of SDS results.

Delta Distillation Sampling [197]:

$$\epsilon_{\text{DDS}} = \epsilon_\phi\left(\mathbf{x}_{\theta,t}; y_{\text{tgt}}, t\right) - \epsilon_\phi\left(\mathbf{x}_{\text{ref},t}; y_{\text{src}}, t\right),$$

where $\mathbf{x}_{\text{ref},t}$ is a noised version of a reference image in the image editing task. As shown in Figure 5.16 (b), this increases the *source distribution mismatch* since $\epsilon_{\phi,\text{src}}$ is not calculated based

on the current optimized image $\mathbf{x}_{\theta,t}$.

Noise Free Score Distillation [200]:

$$\epsilon_{\text{NFSD}} = (\epsilon_\phi(\mathbf{x}_{\theta,t}; \varnothing, t) - (t < 0.2) \cdot \epsilon_\phi(\mathbf{x}_{\theta,t}; y_{\text{neg}}, t)) + s \cdot (\epsilon_\phi(\mathbf{x}_{\theta,t}; y_{\text{tgt}}, t) - \epsilon_\phi(\mathbf{x}_{\theta,t}; \varnothing, t)),$$

where the strength of classifier-free guidance s is set to 7.5 and y_{neg} ="unrealistic, blurry, low quality ...". NFSD greatly reduces the guidance strength while it is observed to perform very similarly to SDS in practice. We can better explain this phenomenon since the prompt y_{neg} does not accurately describe the source distribution as it omits the image's content. In addition, the second component with weight $s = 7.5$ still forms the major part of the gradient, which is the dominant term in SDS.

Classifier Score Distillation [204]:

$$\epsilon_{\text{CSD}} = w_1 \cdot (\epsilon_\phi(\mathbf{x}_{\theta,t}; y_{\text{tgt}}, t) - \epsilon_\phi(\mathbf{x}_{\theta,t}; \varnothing, t)) + w_2 \cdot (\epsilon_\phi(\mathbf{x}_{\theta,t}; \varnothing, t) - \epsilon_\phi(\mathbf{x}_{\theta,t}; y_{\text{src}}, t)),$$

where w_1 and w_2 are hyperparameters. As shown in Figure 5.16 (c), the second term approximates the bridge from the source distribution to the unconditional distribution, which is not ideal since it does not point to the target distribution. It explains the observation made by the authors [204] that this undermines the alignment with the text prompt. Therefore, the authors always anneal w_2 to 0 during the optimization. However, we show this often reintroduces the SDS artifacts in practice.

Variational Score Distillation [202, 361]:

$$\epsilon_{\text{VSD}} = \epsilon_\phi\left(\mathbf{x}_{\theta,t}; \varnothing, t\right) + s \cdot \left(\epsilon_\phi\left(\mathbf{x}_{\theta,t}; y_{\text{tgt}}, t\right) - \epsilon_\phi\left(\mathbf{x}_{\theta,t}; \varnothing, t\right)\right) - \epsilon_{LoRA}\left(\mathbf{x}_{\theta,t}; y_{\text{tgt}}, t\right).$$

Out of all the discussed methods, VSD attempts to minimize the *source distribution mismatch* error most directly by test-time finetuning a copy of the diffusion model with LoRA on the current set of $\mathbf{x}_\theta$. Note that in the original paper, the use of LoRA was motivated based on a particle-based variational framework. Our analysis enables an alternative understanding of VSD. As shown in Figure 5.16 a), this approach is well-justified in our dual diffusion bridge framework. However, training a LoRA *every iteration* is computationally expensive, adds complexity, and introduces its own low-rank approximation errors. Given this insight, we propose a simple yet efficient approach to mitigating source distribution without LoRA.

5.3.2.4 Mitigating Source Distribution Mismatch with Textual Descriptions

Our analysis reveals that the LoRA model in VSD most closely approximates the distribution of the current optimized parametrized image, addressing the distribution mismatch error. Unfortunately, it incurs $200 - 300\%$ runtime overhead on top of SDS, making it impractical, despite its significant performance gains. With this understanding, we propose a simple approach that better expresses the source distribution. Our insight is that pre-trained diffusion models have learned the distribution of natural and corrupted images through a combination of powerful text representation and enormous image-caption datasets. We find that by simply describing image corruptions with a text prompt, we can improve our estimate of the source distribution.

Specifically, we propose to use the gradient

$$\epsilon_{\text{ours}} = w \cdot (\epsilon_\phi(\mathbf{x}_{\theta,t}; y_{\text{tgt}}, t) - \epsilon_\phi(\mathbf{x}_{\theta,t}; y_{\text{src}}, t)),$$

where we get y_{src} by adding descriptions of the current image distribution to y_{tgt} (the base prompt). The remaining question is how to set this description. In generation tasks, we propose a simple two-stage solution.

1. We use ϵ_{SDS} to produce a generation with the method's characteristic artifacts:
2. We switch to optimization with our gradient, ϵ_{ours}, to transport the image parameter toward the natural image distribution.

To describe the artifacts produced by SDS, we append the descriptors ", `oversaturated, smooth, pixelated, cartoon, foggy, hazy, blurry, bad structure, noisy, malformed`" and drop the descriptors of the high-quality generation. This description y_{src} does not require hand-crafting based on problem domains—it is fixed across all shown examples and use cases.

In editing tasks, we have an initialization that y_{src} describes accurately. In such cases, we omit the first SDS stage and only apply our gradient to optimization. We also append a "domain descriptor." For instance, in painting-to-real, this is simply ", painting" to represent the initial distribution.

While the use of such negative prompting has been explored before, such as in NFSD, our analysis motivates a principled way to incorporate it into score distillation. We find that these simple modifications significantly narrow the quality gap between SDS and resource-intensive methods like VSD. We verify this finding experimentally with qualitative results and quantitative

Figure 5.17: **Text-to-image generation results with COCO Captions.** We compare different score distillation methods for generating images with COCO captions by optimizing a randomly initialized image. DDIM sampling indicates the lower bound that the diffusion model can achieve. VSD [202] and our method generate the least color artifacts while ours is more efficient than VSD.

comparisons across applicable tasks.

5.3.3 Experiments

In this section, we test our proposed method on several generation problems where SDS is adopted. We compare against SDS and other task-specific baselines. Note that our goal is not to show another state-of-the-art text-to-3D generation method, but to verify our findings, where the proposed score distillation approach based on textual description efficiently improves the results by mitigating the source distribution mismatch error. We first perform a thorough experiment in a controlled setting on text-to-image generation. Then, we compare it on text-guided NeRF optimization to SDS and VSD and evaluate the painting-to-real image translation task against image editing baselines.

Table 5.3: **Zero-shot FID comparison with different score distillation methods.** We report FID scores of text-to-image generation using 5K captions randomly sampled from the COCO dataset. The best score distillation result is indicated in **bold**, while the second best is underlined.

	DDIM (lower bound)	SDS [195]	NFSD [200]	CSD [204]	VSD [202]	Ours
Zero-Shot FID (↓)	49.12	86.02	91.70	89.96	**59.22**	67.89
Zero-Shot CLIP FID (↓)	16.56	28.39	29.25	27.07	**18.86**	20.31
Time per Sample (mins)	0.05	**4.48**	7.20	6.21	16.02	**4.48**

5.3.3.1 Zero-Shot Text-to-Image Generation with Score Distillation

To verify our analysis of existing SDS variants and the proposed method, we perform text-to-image generation by optimizing an image of size $64 \times 64 \times 4$ in the Stable Diffusion latent space [200, 202]. The benefit of choosing image generation as the evaluation task is that its generation quality has the least confounding variables among other tasks. (*e.g.,* in text-to-3D, many designs like regularizations [203], initialization [222], 3D representations [220, 221, 223, 375], and 2D prior models [205, 224–227] could affect the final quality.)

We use the MS-COCO [376] dataset for the evaluation. Consistent with the prior study [41], we randomly sample 5K captions from the COCO validation set as conditions for generating images. For each caption, we optimize a randomly initialized the image with the score distillation gradients. We compare our method with several SDS variants including SDS [195], NFSD [200], CSD [204], and VSD [202]. For all the methods, we use the same learning rate of 0.01 and optimize for $2,500$ steps where we generally observe convergence. We compute the zero-shot FID [126] and CLIP FID scores [124] between these generated images and the ground truth images. We also report results generated by DDIM with 20 steps as a lower bound for renference.

We report the FID scores and the time to optimize one image in Table 5.3. Among all the score distillation methods, VSD [202] achieves the lowest FID scores. However, it requires

training a LoRA along the optimization process. Instead, ours achieves a comparable FID score with over $3\times$ faster speed. We visualize random examples generated by different score distillation methods in Figure 5.17. We notice that SDS and NSFD suffer from the over-saturation and over-smoothness issues. CDS has slightly fewer color artifacts. VSD and ours generate the samples that most closely resemble the DDIM sampling.

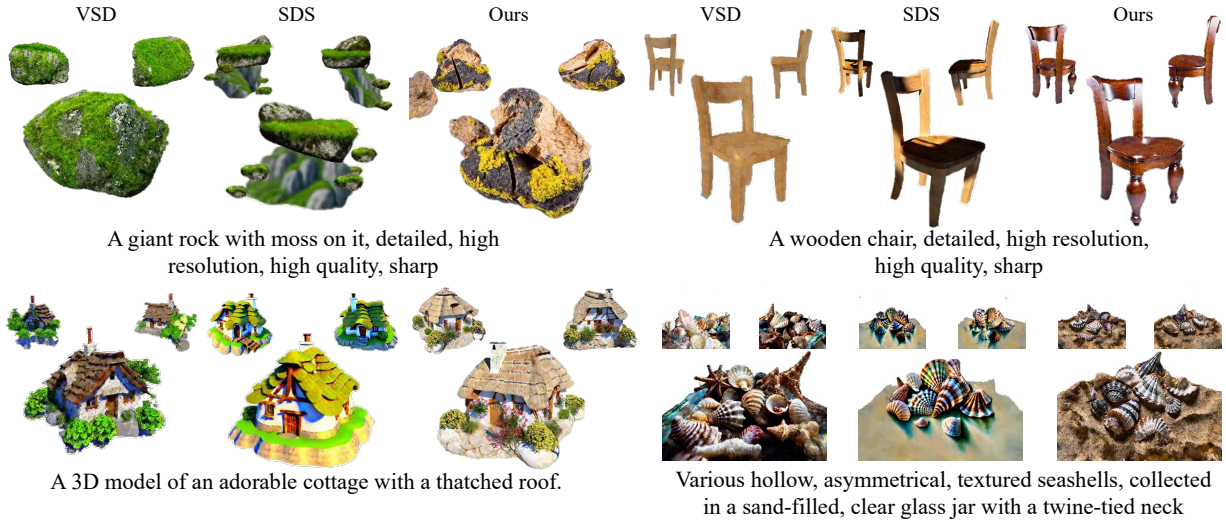


Figure 5.18: **Text-guided NeRF optimization with different score distillation methods.** We make a fair comparison of SDS and VSD for text-to-3D generation. For each generation, we show three uniformly sampled views. SDS results like the cottage and pepper mill still suffer from over-saturation problems, while ours and VSD can produce realistic details, color, and texture.

5.3.3.2 Text-guided NeRF Optimization

We now evaluate the text-to-3D generation problem, where we intentionally aim to exclude variables that could affect the generation quality other than the score distillation methods. We use the ThreeStudio [377] repository to optimize a NeRF with settings tuned for ProlificDreamer stage 1 (NeRF optimization) [202]. Note that we do not perform stages 2 and 3, *i.e*let@tokeneonedotgeometry fine-tuning and texture refinement. Specifically, we initialize the NeRF with the method proposed by Magic3D [222], use the regularization losses on the sparsity

and opacity, and optimize for 25K steps. We adopt the native SDS and VSD guidance implementations for comparison.

We first show visual comparisons of different score distillation methods in Figure 5.18. We notice that SDS tends to generate fewer details, as shown by the rock and chair examples, and sometimes suffers from over-saturation issues, as in 2D, as demonstrated by the cottage and seashell examples. Instead, both VSD and ours can generate highly photo-realistic 3D objects, while ours does not require training a LoRA model and shares a similar computational cost as SDS.

Table 5.4: **Quantitative comparisons of NeRF optimization**. We measure the average CLIP similarity of rendered views using SDS, VSD and our.

	ViT-L/14	ViT-B/16	ViT-B/32
SDS [195]	0.2811	0.3196	0.3139
VSD [202]	0.2837	0.3292	0.3166
Ours	0.2848	0.3282	0.3148

We also perform a quantitative evaluation and user study on the NeRFs optimized based on 31 different text prompts. Note that this number is similar to the choice of existing works on the text-to-3D task [361, 367, 378]. However, different from these works that ignore the confounding 3D variables that contribute to the generation quality, we disentangle this by isolating the score distillation method as the only comparison variable. We follow these works to evaluate the generation quality with CLIP [260]. We report the CLIP similarity in Table 5.4. Our method consistently outperforms SDS and achieves comparable results with VSD. In addition, in a user study consisting of 37 users, shown pairwise comparisons of rotating 3D renders (*i.e.,* comparisons of our result and a random choice of VSD or SDS, with the prompt: "For a text-to-3D system, given the prompt *[p]*, which result would you be happiest with?"), our results were chosen in 75.7% of all responses.

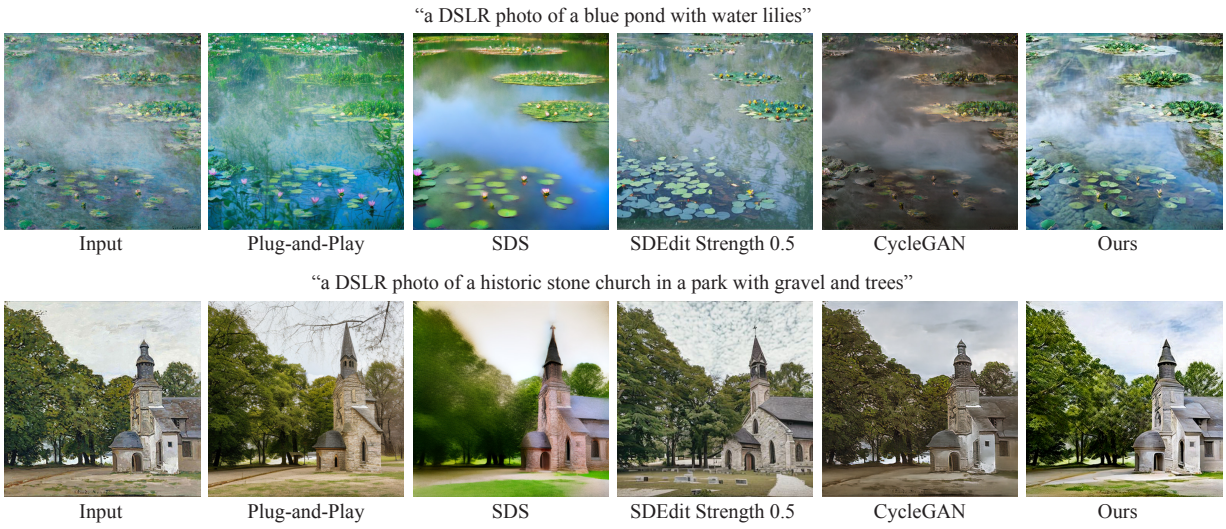


Figure 5.19: **Painting-to-Real comparison.** We compare our gradient in optimization to image restoration and image-conditional generation baselines. While SDEdit produces convincing textures, it is difficult to find a strength value that balances structure and quality. Other baselines fail to reproduce natural image quality, while our method produces the best combination of quality and faithfulness.

5.3.3.3 Painting-to-Real

We examine our method's ability to serve as a general-purpose realism prior. Paintings are "near-manifold" images, meaning they do not possess natural image statistics but live near the image distribution in image space. An effective image prior should guide a painting toward a nearby natural image through optimization.

We initialize a latent image by encoding scans of the artwork through Stable Diffusion's encoder. We specify a prompt for each painting to condition the diffusion model and then apply the second optimization stage of our method (SDS stage omitted). We experimented with automatically generating prompts via pretrained vision language models but found the results inconsistent, so we leave this to future work. Since the large image datasets used to train diffusion models contain artwork, we append the domain descriptor ", painting" to y_{src} to optimize away from this distribution.

Table 5.5: **Reducing first-order approximation error improves generation quality.** Using full PF-ODE simulation ("Full-path") to replace single-step prediction improves visual quality in all settings.

Approximate the bridge	Single-step			Full-path		
Estimate source distribution	Uncond. (SDS)	Bridge	LoRA (VSD)	Uncond.	Bridge	LoRA
Zero-Shot COCO FID (↓)	86.02	67.89	59.22	63.31	60.07	55.65

While SDS is proposed to leverage a pretrained text-to-image diffusion model as an image prior, its artifacts make it ineffective in practice. In comparison, our method realistically synthesizes details and relights the image naturally. We observe that SDS methods diverge more easily in 2D experiments than in 3D but that the issue can be mostly resolved with tuning. A future goal is to formulate a gradient that can be applied idempotently [379]. We compare with image reconstruction baselines in Figure 5.19 and provide a small gallery of painting-to-real results in Figure 5.20.

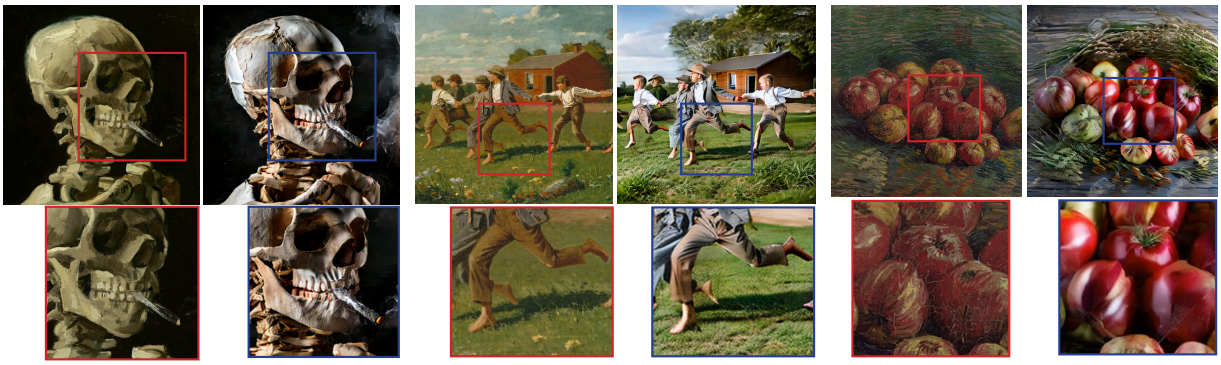


Figure 5.20: **Painting-to-Real results.**We show selected Painting-to-Real samples with diverse art styles and subjects. Initialization images are shown on the left, optimized images are shown on the right.

5.3.4 Discussion on Solving the Linear Approximation Error

As we have shown that reducing the distribution mismatching error can significantly improve the generation quality of the score distillation optimization, it is natural to ask whether

one can also reduce the first-order approximation error, induced by linear bridge estimation, to improve the results further. Several recent studies, including SDI [368] and ISM [367], can be viewed as mitigating this error by replacing the single-step estimation with a multi-step estimation to an intermediate timestep. Under our framework, one can estimate the entire dual bridge by solving both PF-ODE paths. Specifically, via inversion, one can solve the PF-ODE path from $\psi_{0,\text{src}}$ to $x_{\theta,T}$, and then walk to the $\psi_{0,\text{tgt}}$ via sampling. In this way, it is possible to obtain the most accurate gradient direction with little approximation error $\epsilon^*_{\text{SBP}} = w \cdot (\psi_{0,\text{tgt}} - \psi_{0,\text{src}})$. We refer to this approach as "full path". Note that this resolves the linear approximation error, and it is independent of handling the source approximation error, which could be addressed via the discussed text description or LoRA.

However, solving the inversion ODE is not trivial [266]. We noticed that the inversion can exaggerate the distribution mismatch error and cause the optimization to get stuck at a local optimum at the beginning of the optimization. Instead, the stochasticity of the single-step methods often shows more robustness to the input image. Therefore, we first perform the single-step score distillation optimization to obtain reasonable results and then switch to solving the full bridge. We also anneal the timestep endpoint of the bridge throughout the optimization. With this approach, we can now explore addressing both the first and second sources of error. The first source (linear approximation) has "full-path," and the second source (source distribution mismatch error) has "Bridge" or "LoRA". We find that using the "full-path" multi-step (mitigating linear approximation error) always outperforms the single-step methods, achieving a lower FID, as shown in Table 5.5. However, the same trend does not fully transfer to the text-to-3D experiments. We observe that solving the entire bridge typically introduces additional artifacts and makes the optimization less stable. We leave the best way of leveraging this gradient for future

research exploration.

5.3.5 Conclusion

We present an analysis that formulates the use of a pre-trained diffusion model in an optimization framework as seeking an optimal transport between two distributions. Under this lens, we analyze SDS variants with a unified framework. We also develop a simple approach based on textual descriptions that work comparably well to the best-performing approach, VSD, without its significant computational burden. However, neither approach has yet to achieve the quality and diversity of images generated by the reverse process. We hope that our analysis enables the development of a more sophisticated solution that can one day achieve the same quality and diversity as the reverse process in an optimization framework. Combining our proposed method with multi-step approximations like ISM [367] or schedules like DreamFlow [361] could mitigate the first-order approximation error and further improve the efficiency, which is an interesting future research direction. With the rise of high-quality video diffusion models, we anticipate that the question of how to effectively use such models as a prior in various problems will become even more important.

Bibliography

[1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *NeurIPS*, 2014.

[2] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *CVPR*, 2019.

[3] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan. In *CVPR*, 2020.

[4] Minguk Kang, Jun-Yan Zhu, Richard Zhang, Jaesik Park, Eli Shechtman, Sylvain Paris, and Taesung Park. Scaling up gans for text-to-image synthesis. In *CVPR*, 2023.

[5] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *ICCV*, 2017.

[6] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks, 2020.

[7] Carl Vondrick, Hamed Pirsiavash, and Antonio Torralba. Generating videos with scene dynamics. *Advances in neural information processing systems*, 29, 2016.

[8] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 1997.

[9] Masaki Saito, Eiichi Matsumoto, and Shunta Saito. Temporal generative adversarial nets with singular value clipping. In *ICCV*, 2017.

[10] Sergey Tulyakov, Ming-Yu Liu, Xiaodong Yang, and Jan Kautz. Mocogan: Decomposing motion and content for video generation. In *CVPR*, June 2018.

[11] Aidan Clark, Jeff Donahue, and Karen Simonyan. Adversarial video generation on complex datasets. *arXiv preprint arXiv:1907.06571*, 2019.

[12] Masaki Saito, Shunta Saito, Masanori Koyama, and Sosuke Kobayashi. Train sparsely, generate densely: Memory-efficient unsupervised training of high-resolution temporal gan. *International Journal of Computer Vision*, 128(10-11):2586–2606, 2020.

[13] Andres Munoz, Mohammadreza Zolfaghari, Max Argus, and Thomas Brox. Temporal shift gan for large scale video generation. In *WACV*, 2021.

[14] Yu Tian, Jian Ren, Menglei Chai, Kyle Olszewski, Xi Peng, Dimitris N. Metaxas, and Sergey Tulyakov. A good image generator is what you need for high-resolution video synthesis. In *ICLR*, 2021.

[15] MarcAurelio Ranzato, Arthur Szlam, Joan Bruna, Michael Mathieu, Ronan Collobert, and Sumit Chopra. Video (language) modeling: a baseline for generative models of natural videos. *arXiv preprint arXiv:1412.6604*, 2014.

[16] Nitish Srivastava, Elman Mansimov, and Ruslan Salakhudinov. Unsupervised learning of video representations using lstms. In *ICML*, 2015.

[17] Nal Kalchbrenner, Aäron Oord, Karen Simonyan, Ivo Danihelka, Oriol Vinyals, Alex Graves, and Koray Kavukcuoglu. Video pixel networks. In *ICML*, 2017.

[18] Aäron van den Oord, Nal Kalchbrenner, Oriol Vinyals, Lasse Espeholt, Alex Graves, and Koray Kavukcuoglu. Conditional image generation with pixelcnn decoders. In *NeurIPS*, 2016.

[19] Dirk Weissenborn, Oscar Täckström, and Jakob Uszkoreit. Scaling autoregressive video models. In *ICLR*, 2020.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[21] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *NeurIPS*, 2017.

[22] Ruslan Rakhimov, Denis Volkhonskiy, Alexey Artemov, Denis Zorin, and Evgeny Burnaev. Latent video transformer. *arXiv preprint arXiv:2006.10704*, 2020.

[23] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8821–8831. PMLR, 18–24 Jul 2021.

[24] Jiahui Yu, Yuanzhong Xu, Jing Yu Koh, Thang Luong, Gunjan Baid, Zirui Wang, Vijay Vasudevan, Alexander Ku, Yinfei Yang, Burcu Karagol Ayan, et al. Scaling autoregressive models for content-rich text-to-image generation. *Transactions on Machine Learning Research*, 2022.

[25] Huiwen Chang, Han Zhang, Jarred Barber, AJ Maschinot, Jose Lezama, Lu Jiang, Ming-Hsuan Yang, Kevin Murphy, William T Freeman, Michael Rubinstein, et al. Muse: Text-to-image generation via masked generative transformers. *arXiv preprint arXiv:2301.00704*, 2023.

[26] Ming Ding, Wendi Zheng, Wenyi Hong, and Jie Tang. Cogview2: Faster and better text-to-image generation via hierarchical transformers. *arXiv preprint arXiv:2204.14217*, 2022.

[27] Chenfei Wu, Jian Liang, Lei Ji, Fan Yang, Yuejian Fang, Daxin Jiang, and Nan Duan. N\" uwa: Visual synthesis pre-training for neural visual world creation. *arXiv preprint arXiv:2111.12417*, 2021.

[28] Guillaume Le Moing, Jean Ponce, and Cordelia Schmid. Ccvs: Context-aware controllable video synthesis. *NeurIPS*, 2021.

[29] Wilson Yan, Yunzhi Zhang, Pieter Abbeel, and Aravind Srinivas. Videogpt: Video generation using vq-vae and transformers. *arXiv preprint arXiv:2104.10157*, 2021.

[30] Lijun Yu, Yong Cheng, Kihyuk Sohn, José Lezama, Han Zhang, Huiwen Chang, Alexander G Hauptmann, Ming-Hsuan Yang, Yuan Hao, Irfan Essa, and Lu Jiang. MAGVIT: Masked generative video transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[31] Zhuoyan Luo, Fengyuan Shi, Yixiao Ge, Yujiu Yang, Limin Wang, and Ying Shan. Open-magvit2: An open-source project toward democratizing auto-regressive visual generation. *arXiv preprint arXiv:2409.04410*, 2024.

[32] Dan Kondratyuk, Lijun Yu, Xiuye Gu, Jose Lezama, Jonathan Huang, Grant Schindler, Rachel Hornung, Vighnesh Birodkar, Jimmy Yan, Ming-Chang Chiu, Krishna Somandepalli, Hassan Akbari, Yair Alon, Yong Cheng, Joshua V. Dillon, Agrim Gupta, Meera Hahn, Anja Hauth, David Hendon, Alonso Martinez, David Minnen, Mikhail Sirotenko, Kihyuk Sohn, Xuan Yang, Hartwig Adam, Ming-Hsuan Yang, Irfan Essa, Huisheng Wang, David A Ross, Bryan Seybold, and Lu Jiang. VideoPoet: A large language model for zero-shot video generation. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 25105–25124. PMLR, 21–27 Jul 2024.

[33] Vincent Sitzmann, Julien Martel, Alexander Bergman, David Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *NeurIPS*, 2020.

[34] Matthew Tancik, Pratul Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *NeurIPS*, 2020.

[35] Ivan Skorokhodov, Savva Ignatyev, and Mohamed Elhoseiny. Adversarial generation of continuous images. In *CVPR*, 2021.

[36] Ivan Anokhin, Kirill Demochkin, Taras Khakhulin, Gleb Sterkin, Victor Lempitsky, and Denis Korzhenkov. Image generators with conditionally-independent pixel synthesis. In *CVPR*, 2021.

[37] Sihyun Yu, Jihoon Tack, Sangwoo Mo, Hyunsu Kim, Junho Kim, Jung-Woo Ha, and Jinwoo Shin. Generating videos with dynamics-aware implicit generative adversarial networks. In *ICLR*, 2022.

[38] Ivan Skorokhodov, Sergey Tulyakov, and Mohamed Elhoseiny. Stylegan-v: A continuous video generator with the price, image quality and perks of stylegan2. In *CVPR*, 2022.

[39] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

[40] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *ICLR*, 2021.

[41] Yogesh Balaji, Seungjun Nah, Xun Huang, Arash Vahdat, Jiaming Song, Karsten Kreis, Miika Aittala, Timo Aila, Samuli Laine, Bryan Catanzaro, et al. ediffi: Text-to-image diffusion models with an ensemble of expert denoisers. *arXiv preprint arXiv:2211.01324*, 2022.

[42] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 2022.

[43] Alexander Quinn Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob Mcgrew, Ilya Sutskever, and Mark Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. In *ICML*, 2022.

[44] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, S. Sara Mahdavi, Rapha Gontijo Lopes, Tim Salimans, Jonathan Ho, David J Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding. In *NeurIPS*, 2022.

[45] Oran Gafni, Adam Polyak, Oron Ashual, Shelly Sheynin, Devi Parikh, and Yaniv Taigman. Make-a-scene: Scene-based text-to-image generation with human priors. In *ECCV*, 2022.

[46] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 2022.

[47] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, S Sara Mahdavi, Rapha Gontijo Lopes, et al. Photorealistic text-to-image diffusion models with deep language understanding. *arXiv preprint arXiv:2205.11487*, 2022.

[48] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.

[49] Vikram Voleti, Alexia Jolicoeur-Martineau, and Christopher Pal. Masked conditional video diffusion for prediction, generation, and interpolation. *arXiv preprint arXiv:2205.09853*, 2022.

[50] William Harvey, Saeid Naderiparizi, Vaden Masrani, Christian Weilbach, and Frank Wood. Flexible diffusion modeling of long videos. *arXiv preprint arXiv:2205.11495*, 2022.

[51] Daquan Zhou, Weimin Wang, Hanshu Yan, Weiwei Lv, Yizhe Zhu, and Jiashi Feng. Magicvideo: Efficient video generation with latent diffusion models. *arXiv preprint arXiv:2211.11018*, 2022.

[52] Jay Zhangjie Wu, Yixiao Ge, Xintao Wang, Stan Weixian Lei, Yuchao Gu, Yufei Shi, Wynne Hsu, Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Tune-a-video: One-shot tuning of image diffusion models for text-to-video generation. In *ICCV*, 2023.

[53] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. In *CVPR*, 2023.

[54] Levon Khachatryan, Andranik Movsisyan, Vahram Tadevosyan, Roberto Henschel, Zhangyang Wang, Shant Navasardyan, and Humphrey Shi. Text2video-zero: Text-to-image diffusion models are zero-shot video generators. *arXiv preprint arXiv:2303.13439*, 2023.

[55] Tobias Höppe, Arash Mehrjou, Stefan Bauer, Didrik Nielsen, and Andrea Dittadi. Diffusion models for video prediction and infilling. *arXiv preprint arXiv:2206.07696*, 2022.

[56] Ruihan Yang, Prakhar Srivastava, and Stephan Mandt. Diffusion probabilistic modeling for video generation. *arXiv preprint arXiv:2203.09481*, 2022.

[57] Yaniv Nikankin, Niv Haim, and Michal Irani. Sinfusion: Training diffusion models on a single image or video. *arXiv preprint arXiv:2211.11743*, 2022.

[58] Zhengxiong Luo, Dayou Chen, Yingya Zhang, Yan Huang, Liang Wang, Yujun Shen, Deli Zhao, Jingren Zhou, and Tieniu Tan. Videofusion: Decomposed diffusion models for high-quality video generation. In *CVPR*, 2023.

[59] Jie An, Songyang Zhang, Harry Yang, Sonal Gupta, Jia-Bin Huang, Jiebo Luo, and Xi Yin. Latent-shift: Latent diffusion with temporal shift for efficient text-to-video generation. *arXiv preprint arXiv:2304.08477*, 2023.

[60] Wenjing Wang, Huan Yang, Zixi Tuo, Huiguo He, Junchen Zhu, Jianlong Fu, and Jiaying Liu. Videofactory: Swap attention in spatiotemporal diffusions for text-to-video generation. *arXiv preprint arXiv:2305.10874*, 2023.

[61] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, Devi Parikh, Sonal Gupta, and Yaniv Taigman. Make-a-video: Text-to-video generation without text-video data. In *ICLR*, 2023.

[62] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *NeurIPS*, 2022.

[63] Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P Kingma, Ben Poole, Mohammad Norouzi, David J Fleet, et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022.

[64] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. Laion-5b: An open large-scale dataset for training next generation image-text models. In *NeurIPS*, 2022.

[65] Minwoo Byeon, Beomhee Park, Haecheon Kim, Sungjun Lee, Woonhyuk Baek, and Saehoon Kim. Coyo-700m: Image-text pair dataset. `https://github.com/kakaobrain/coyo-dataset`, 2022.

[66] Tsai-Shien Chen, Aliaksandr Siarohin, Willi Menapace, Ekaterina Deyneka, Hsiang wei Chao, Byung Eun Jeon, Yuwei Fang, Hsin-Ying Lee, Jian Ren, Ming-Hsuan Yang, and Sergey Tulyakov. Panda-70m: Captioning 70m videos with multiple cross-modality teachers, 2024.

[67] Jonathan Ho, Chitwan Saharia, William Chan, David J Fleet, Mohammad Norouzi, and Tim Salimans. Cascaded diffusion models for high fidelity image generation. *Journal of Machine Learning Research*, 23(47):1–33, 2022.

[68] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.

[69] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. *arXiv preprint arXiv:2112.10741*, 2021.

[70] Songwei Ge, Seungjun Nah, Guilin Liu, Tyler Poon, Andrew Tao, Bryan Catanzaro, David Jacobs, Jia-Bin Huang, Ming-Yu Liu, and Yogesh Balaji. Preserve your own correlation: A noise prior for video diffusion models. In *ICCV*, 2023.

[71] Shuyang Gu, Dong Chen, Jianmin Bao, Fang Wen, Bo Zhang, Dongdong Chen, Lu Yuan, and Baining Guo. Vector quantized diffusion model for text-to-image synthesis. In *CVPR*, pages 10696–10706, 2022.

[72] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22563–22575, June 2023.

[73] Yaohui Wang, Xinyuan Chen, Xin Ma, Shangchen Zhou, Ziqi Huang, Yi Wang, Ceyuan Yang, Yinan He, Jiashuo Yu, Peiqing Yang, et al. Lavie: High-quality video generation with cascaded latent diffusion models. *arXiv preprint arXiv:2309.15103*, 2023.

[74] Levon Khachatryan, Andranik Movsisyan, Vahram Tadevosyan, Roberto Henschel, Zhangyang Wang, Shant Navasardyan, and Humphrey Shi. Text2video-zero: Text-to-image diffusion models are zero-shot video generators. In *CVPR*, 2023.

[75] Zhen Xing, Qi Dai, Han Hu, Zuxuan Wu, and Yu-Gang Jiang. Simda: Simple diffusion adapter for efficient video generation, 2023.

[76] Jinbo Xing, Menghan Xia, Yong Zhang, Haoxin Chen, Xintao Wang, Tien-Tsin Wong, and Ying Shan. Dynamicrafter: Animating open-domain images with video diffusion priors, 2023.

[77] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, Varun Jampani, and Robin Rombach. Stable video diffusion: Scaling latent video diffusion models to large datasets, 2023.

[78] Jiaxi Gu, Shicong Wang, Haoyu Zhao, Tianyi Lu, Xing Zhang, Zuxuan Wu, Songcen Xu, Wei Zhang, Yu-Gang Jiang, and Hang Xu. Reuse and diffuse: Iterative denoising for text-to-video generation, 2023.

[79] Agrim Gupta, Lijun Yu, Kihyuk Sohn, Xiuye Gu, Meera Hahn, Li Fei-Fei, Irfan Essa, Lu Jiang, and José Lezama. Photorealistic video generation with diffusion models, 2023.

[80] Xiang Wang, Shiwei Zhang, Han Zhang, Yu Liu, Yingya Zhang, Changxin Gao, and Nong Sang. Videolcm: Video latent consistency model, 2023.

[81] Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *CVPR*, 2021.

[82] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. SDXL: Improving latent diffusion models for high-resolution image synthesis. In *ICLR*, 2024.

[83] FLUX. FLUX.1: Image generation, 2024.

[84] Peize Sun, Yi Jiang, Shoufa Chen, Shilong Zhang, Bingyue Peng, Ping Luo, and Zehuan Yuan. Autoregressive model beats diffusion: Llama for scalable image generation. *arXiv preprint arXiv:2406.06525*, 2024.

[85] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2022.

[86] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

[87] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[88] Yan Zeng, Guoqiang Wei, Jiani Zheng, Jiaxin Zou, Yang Wei, Yuchen Zhang, and Hang Li. Make pixels dance: High-dynamic video generation, 2023.

[89] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. *arXiv preprint arXiv:2408.06072*, 2024.

[90] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators, 2024.

[91] Sihyun Yu, Kihyuk Sohn, Subin Kim, and Jinwoo Shin. Video probabilistic diffusion models in projected latent space. In *CVPR*, 2023.

[92] Rohit Girdhar, Mannat Singh, Andrew Brown, Quentin Duval, Samaneh Azadi, Sai Saketh Rambhatla, Akbar Shah, Xi Yin, Devi Parikh, and Ishan Misra. Emu video: Factorizing text-to-video generation by explicit image conditioning, 2023.

[93] OpenAI. ChatGPT.

[94] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[95] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning, 2018.

[96] Yecheng Wu, Zhuoyang Zhang, Junyu Chen, Haotian Tang, Dacheng Li, Yunhao Fang, Ligeng Zhu, Enze Xie, Hongxu Yin, Li Yi, Song Han, and Yao Lu. Vila-u: a unified foundation model integrating visual understanding and generation, 2024.

[97] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models, 2024.

[98] Quan Sun, Yufeng Cui, Xiaosong Zhang, Fan Zhang, Qiying Yu, Yueze Wang, Yongming Rao, Jingjing Liu, Tiejun Huang, and Xinlong Wang. Generative multimodal models are in-context learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14398–14409, 2024.

[99] Xinlong Wang, Xiaosong Zhang, Zhengxiong Luo, Quan Sun, Yufeng Cui, Jinsheng Wang, Fan Zhang, Yueze Wang, Zhen Li, Qiying Yu, et al. Emu3: Next-token prediction is all you need. *arXiv preprint arXiv:2409.18869*, 2024.

[100] Chengyue Wu, Xiaokang Chen, Zhiyu Wu, Yiyang Ma, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, Chong Ruan, et al. Janus: Decoupling visual encoding for unified multimodal understanding and generation. *arXiv preprint arXiv:2410.13848*, 2024.

[101] Ruben Villegas, Mohammad Babaeizadeh, Pieter-Jan Kindermans, Hernan Moraldo, Han Zhang, Mohammad Taghi Saffar, Santiago Castro, Julius Kunze, and Dumitru Erhan. Phenaki: Variable length video generation from open domain textual description. *arXiv preprint arXiv:2210.02399*, 2022.

[102] Chenfei Wu, Jian Liang, Lei Ji, Fan Yang, Yuejian Fang, Daxin Jiang, and Nan Duan. Nüwa: Visual synthesis pre-training for neural visual world creation. In *ECCV*, 2022.

[103] Junyu Chen, Han Cai, Junsong Chen, Enze Xie, Shang Yang, Haotian Tang, Muyang Li, Yao Lu, and Song Han. Deep compression autoencoder for efficient high-resolution diffusion models. *arXiv preprint arXiv:2410.10733*, 2024.

[104] Qihang Yu, Mark Weber, Xueqing Deng, Xiaohui Shen, Daniel Cremers, and Liang-Chieh Chen. An image is worth 32 tokens for reconstruction and generation. *arxiv: 2406.07550*, 2024.

[105] Hanyu Wang, Saksham Suri, Yixuan Ren, Hao Chen, and Abhinav Shrivastava. Larp: Tokenizing videos with a learned autoregressive generative prior. *arXiv preprint arXiv:2410.21264*, 2024.

[106] Songwei Ge, Thomas Hayes, Harry Yang, Xi Yin, Guan Pang, David Jacobs, Jia-Bin Huang, and Devi Parikh. Long video generation with time-agnostic vqgan and time-sensitive transformer. In *ECCV*, 2022.

[107] Huiwon Jang, Sihyun Yu, Jinwoo Shin, Pieter Abbeel, and Younggyo Seo. Efficient long video tokenization via coordinated-based patch reconstruction. *arXiv preprint arXiv:2411.14762*, 2024.

[108] Aniruddha Mahapatra, Long Mai, Yitian Zhang, David Bourgin, and Feng Liu. Progressive growing of video tokenizers for highly compressed latent spaces. *arXiv preprint arXiv:2501.05442*, 2025.

[109] Yue Zhao, Yuanjun Xiong, and Philipp Krähenbühl. Image and video tokenization with binary spherical quantization, 2024.

[110] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: Vq-vae made simple, 2023.

[111] Lijun Yu, Jose Lezama, Nitesh Bharadwaj Gundavarapu, Luca Versari, Kihyuk Sohn, David Minnen, Yong Cheng, Agrim Gupta, Xiuye Gu, Alexander G Hauptmann, Boqing Gong, Ming-Hsuan Yang, Irfan Essa, David A Ross, and Lu Jiang. Language model beats diffusion - tokenizer is key to visual generation. In *The Twelfth International Conference on Learning Representations*, 2024.

[112] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11523–11532, 2022.

[113] Mark Weber, Lijun Yu, Qihang Yu, Xueqing Deng, Xiaohui Shen, Daniel Cremers, and Liang-Chieh Chen. Maskbit: Embedding-free image generation via bit tokens. *TMLR*, 2024.

[114] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *CVPR*, 2022.

[115] Wilson Yan, Matei Zaharia, Volodymyr Mnih, Pieter Abbeel, Aleksandra Faust, and Hao Liu. Elastictok: Adaptive tokenization for image and video. *arXiv preprint arXiv:2410.08368*, 2024.

[116] Kaiwen Zha, Lijun Yu, Alireza Fathi, David A Ross, Cordelia Schmid, Dina Katabi, and Xiuye Gu. Language-guided image tokenization for generation. *arXiv preprint arXiv:2412.05796*, 2024.

[117] Tianhong Li, Yonglong Tian, He Li, Mingyang Deng, and Kaiming He. Autoregressive image generation without vector quantization. In *NeurIPS*, 2024.

[118] Dongwon Kim, Ju He, Qihang Yu, Chenglin Yang, Xiaohui Shen, Suha Kwak, and Liang-Chieh Chen. Democratizing text-to-image masked generative models with compact text-aware one-dimensional tokens. *arXiv preprint arXiv:2501.07730*, 2025.

[119] Philippe Hansen-Estruch, David Yan, Ching-Yao Chung, Orr Zohar, Jialiang Wang, Tingbo Hou, Tao Xu, Sriram Vishwanath, Peter Vajda, and Xinlei Chen. Learnings from scaling visual tokenizers for reconstruction and generation. *arXiv preprint arXiv:2501.09755*, 2025.

[120] Wenyi Hong, Ming Ding, Wendi Zheng, Xinghan Liu, and Jie Tang. Cogvideo: Large-scale pretraining for text-to-video generation via transformers. *arXiv preprint arXiv:2205.15868*, 2022.

[121] Yue Ma, Yingqing He, Xiaodong Cun, Xintao Wang, Ying Shan, Xiu Li, and Qifeng Chen. Follow your pose: Pose-guided text-to-video generation using pose-free videos, 2023.

[122] Patrick Esser, Johnathan Chiu, Parmida Atighehchian, Jonathan Granskog, and Anastasis Germanidis. Structure and content-guided video synthesis with diffusion models. *arXiv preprint arXiv:2302.03011*, 2023.

[123] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. In *NeurIPS*, 2016.

[124] Tuomas Kynkäänniemi, Tero Karras, Miika Aittala, Timo Aila, and Jaakko Lehtinen. The role of imagenet classes in fréchet inception distance. In *ICLR*, 2022.

[125] Gaurav Parmar, Richard Zhang, and Jun-Yan Zhu. On aliased resizing and surprising subtleties in gan evaluation. In *CVPR*, 2022.

[126] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017.

[127] Stanislav Morozov, Andrey Voynov, and Artem Babenko. On self-supervised image representations for gan evaluation. In *ICLR*, 2020.

[128] Mehdi SM Sajjadi, Olivier Bachem, Mario Lucic, Olivier Bousquet, and Sylvain Gelly. Assessing generative models via precision and recall. In *NeurIPS*, 2018.

[129] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. In *NeurIPS*, 2019.

[130] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, June 2016.

[131] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, pages 248–255, 2009.

[132] Sharon Zhou, Mitchell Gordon, Ranjay Krishna, Austin Narcomey, Li F Fei-Fei, and Michael Bernstein. Hype: A benchmark for human eye perceptual evaluation of generative models. In *NeurIPS*, volume 32, 2019.

[133] Mikolaj Binkowski, Danica J. Sutherland, Michal Arbel, and Arthur Gretton. Demystifying mmd gans. In *ICLR*, 2018.

[134] Muhammad Ferjad Naeem, Seong Joon Oh, Youngjung Uh, Yunjey Choi, and Jaejun Yoo. Reliable fidelity and diversity metrics for generative models. In *ICML*, pages 7176–7185. PMLR, 2020.

[135] Motasem Alfarra, Juan C Pérez, Anna Frühstück, Philip HS Torr, Peter Wonka, and Bernard Ghanem. On the robustness of quality measures for gans. In *ECCV*, 2022.

[136] Ali Borji. Pros and cons of gan evaluation measures: New developments. *Computer Vision and Image Understanding*, 215:103329, 2022.

[137] Eslam Mohamed Bakr, Pengzhan Sun, Xiaoqian Shen, Faizan Farooq Khan, Li Erran Li, and Mohamed Elhoseiny. Hrs-bench: Holistic, reliable and scalable benchmark for text-to-image models. In *ICCV*, 2023.

[138] Yushi Hu, Benlin Liu, Jungo Kasai, Yizhong Wang, Mari Ostendorf, Ranjay Krishna, and Noah A Smith. Tifa: Accurate and interpretable text-to-image faithfulness evaluation with question answering. In *ICCV*, 2023.

[139] Kaiyi Huang, Kaiyue Sun, Enze Xie, Zhenguo Li, and Xihui Liu. T2i-compbench: A comprehensive benchmark for open-world compositional text-to-image generation. In *NeurIPS*, 2024.

[140] Maitreya Patel, Tejas Gokhale, Chitta Baral, and Yezhou Yang. Conceptbed: Evaluating concept learning abilities of text-to-image diffusion models. In *AAAI*, 2024.

[141] Lin Zhao, Tianchen Zhao, Zinan Lin, Xuefei Ning, Guohao Dai, Huazhong Yang, and Yu Wang. Flasheval: Towards fast and accurate evaluation of text-to-image diffusion generative models. *arXiv preprint arXiv:2403.16379*, 2024.

[142] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. Vbench: Comprehensive benchmark suite for video generative models, 2023.

[143] Nupur Kumari, Bingliang Zhang, Richard Zhang, Eli Shechtman, and Jun-Yan Zhu. Multi-concept customization of text-to-image diffusion. In *CVPR*, June 2023.

[144] Taesung Park, Ming-Yu Liu, Ting-Chun Wang, and Jun-Yan Zhu. Semantic image synthesis with spatially-adaptive normalization. In *CVPR*, 2019.

[145] Justin Johnson, Agrim Gupta, and Li Fei-Fei. Image generation from scene graphs. In *CVPR*, 2018.

[146] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros. Image-to-image translation with conditional adversarial networks. In *CVPR*, 2017.

[147] Guillaume Couairon, Marlene Careil, Matthieu Cord, Stéphane Lathuiliere, and Jakob Verbeek. Zero-shot spatial layout conditioning for text-to-image diffusion models. In *ICCV*, 2023.

[148] Minho Park, Jooyeol Yun, Seunghwan Choi, and Jaegul Choo. Learning to generate semantic layouts for higher text-image correspondence in text-to-image synthesis. In *ICCV*, 2023.

[149] Weixi Feng, Wanrong Zhu, Tsu-jui Fu, Varun Jampani, Arjun Akula, Xuehai He, Sugato Basu, Xin Eric Wang, and William Yang Wang. Layoutgpt: Compositional visual planning and generation with large language models. 2024.

[150] Leigang Qu, Shengqiong Wu, Hao Fei, Liqiang Nie, and Tat-Seng Chua. Layoutllm-t2i: Eliciting layout guidance from llm for text-to-image generation. In *ACM MM*, 2023.

[151] Leon A Gatys, Alexander S Ecker, and Matthias Bethge. Image style transfer using convolutional neural networks. In *CVPR*, 2016.

[152] Fujun Luan, Sylvain Paris, Eli Shechtman, and Kavita Bala. Deep photo style transfer. In *CVPR*, 2017.

[153] E. Reinhard, M. Adhikhmin, B. Gooch, and P. Shirley. Color transfer between images. *IEEE Computer Graphics and Applications*, 21(5):34–41, 2001.

[154] Yu-Wing Tai, Jiaya Jia, and Chi-Keung Tang. Local color transfer via probabilistic segmentation by expectation-maximization. In *CVPR*, 2005.

[155] Li Xu, Qiong Yan, and Jiaya Jia. A sparse control model for image and video editing. *ACM Transactions on Graphics (TOG)*, 32:1 – 10, 2013.

[156] Anat Levin, Dani Lischinski, and Yair Weiss. Colorization using optimization. *SIG*, 2004.

[157] Richard Zhang, Phillip Isola, and Alexei A Efros. Colorful image colorization. In *ECCV*, 2016.

[158] Richard Zhang, Jun-Yan Zhu, Phillip Isola, Xinyang Geng, Angela S Lin, Tianhe Yu, and Alexei A Efros. Real-time user-guided image colorization with learned deep priors. *ACM Transactions on Graphics (TOG)*, 9(4), 2017.

[159] James Betker, Gabriel Goh, Li Jing, Tim Brooks, Jianfeng Wang, Linjie Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, et al. Improving image generation with better captions. *Computer Science. https://cdn. openai. com/papers/dall-e-3. pdf*, 2023.

[160] Xudong Wang, Trevor Darrell, Sai Saketh Rambhatla, Rohit Girdhar, and Ishan Misra. Instancediffusion: Instance-level control for image generation. *arXiv preprint arXiv:2402.03290*, 2024.

[161] Weijia Wu, Zhuang Li, Yefei He, Mike Zheng Shou, Chunhua Shen, Lele Cheng, Yan Li, Tingting Gao, Di Zhang, and Zhongyuan Wang. Paragraph-to-image generation with information-enriched diffusion model. *arXiv preprint arXiv:2311.14284*, 2023.

[162] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation. In *CVPR*, 2023.

[163] Lvmin Zhang and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *ICCV*, 2023.

[164] Omri Avrahami, Thomas Hayes, Oran Gafni, Sonal Gupta, Yaniv Taigman, Devi Parikh, Dani Lischinski, Ohad Fried, and Xi Yin. Spatext: Spatio-textual representation for controllable image generation. In *CVPR*, 2023.

[165] Bahjat Kawar, Shiran Zada, Oran Lang, Omer Tov, Huiwen Chang, Tali Dekel, Inbar Mosseri, and Michal Irani. Imagic: Text-based real image editing with diffusion models. In *CVPR*, 2023.

[166] Yuheng Li, Haotian Liu, Qingyang Wu, Fangzhou Mu, Jianwei Yang, Jianfeng Gao, Chunyuan Li, and Yong Jae Lee. Gligen: Open-set grounded text-to-image generation. In *CVPR*, 2023.

[167] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. Sdedit: Guided image synthesis and editing with stochastic differential equations. In *International Conference on Learning Representations*, 2021.

[168] Jooyoung Choi, Sungwon Kim, Yonghyun Jeong, Youngjune Gwon, and Sungroh Yoon. Ilvr: Conditioning method for denoising diffusion probabilistic models. In *ICCV*, 2021.

[169] Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-or. Prompt-to-prompt image editing with cross-attention control. In *ICLR*, 2023.

[170] Gaurav Parmar, Krishna Kumar Singh, Richard Zhang, Yijun Li, Jingwan Lu, and Jun-Yan Zhu. Zero-shot image-to-image translation. In *SIGGRAPH*, 2023.

[171] Arpit Bansal, Hong-Min Chu, Avi Schwarzschild, Soumyadip Sengupta, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Universal guidance for diffusion models. *arXiv preprint arXiv:2302.07121*, 2023.

[172] Hila Chefer, Yuval Alaluf, Yael Vinker, Lior Wolf, and Daniel Cohen-Or. Attend-and-excite: Attention-based semantic guidance for text-to-image diffusion models. *ACM Transactions on Graphics (TOG)*, 42(4):1–10, 2023.

[173] Omri Avrahami, Ohad Fried, and Dani Lischinski. Blended latent diffusion. *arXiv preprint arXiv:2206.02779*, 2022.

[174] Álvaro Barbero Jiménez. Mixture of diffusers for scene composition and high resolution image generation. *arXiv preprint arXiv:2302.02412*, 2023.

[175] Omer Bar-Tal, Lior Yariv, Yaron Lipman, and Tali Dekel. Multidiffusion: Fusing diffusion paths for controlled image generation. In *ICML*, 2023.

[176] Vishnu Sarukkai, Linden Li, Arden Ma, Christopher R'e, and Kayvon Fatahalian. Collage diffusion. *ArXiv*, abs/2303.00262, 2023.

[177] Qinsheng Zhang, Jiaming Song, Xun Huang, Yongxin Chen, and Ming-Yu Liu. Diffcollage: Parallel generation of large content with diffusion models. In *CVPR*, 2023.

[178] Mingdeng Cao, Xintao Wang, Zhongang Qi, Ying Shan, Xiaohu Qie, and Yinqiang Zheng. Masactrl: Tuning-free mutual self-attention control for consistent image synthesis and editing. In *ICCV*, 2023.

[179] Quynh Phung, Songwei Ge, and Jia-Bin Huang. Grounded text-to-image synthesis with attention refocusing. In *CVPR*, 2024.

[180] Guangxuan Xiao, Tianwei Yin, William T Freeman, Frédo Durand, and Song Han. Fastcomposer: Tuning-free multi-subject image generation with localized attention. *arXiv preprint arXiv:2305.10431*, 2023.

[181] Weixi Feng, Xuehai He, Tsu-Jui Fu, Varun Jampani, Arjun Reddy Akula, Pradyumna Narayana, Sugato Basu, Xin Eric Wang, and William Yang Wang. Training-free structured diffusion guidance for compositional text-to-image synthesis. In *ICLR*, 2023.

[182] Tim Brooks, Aleksander Holynski, and Alexei A Efros. Instructpix2pix: Learning to follow image editing instructions. In *CVPR*, 2023.

[183] Ruoshi Liu, Rundi Wu, Basile Van Hoorick, Pavel Tokmakov, Sergey Zakharov, and Carl Vondrick. Zero-1-to-3: Zero-shot one image to 3d object. In *ICCV*, 2023.

[184] Hung-Yu Tseng, Qinbo Li, Changil Kim, Suhib Alsisan, Jia-Bin Huang, and Johannes Kopf. Consistent view synthesis with pose-guided diffusion models. In *CVPR*, 2023.

[185] Daniel Watson, William Chan, Ricardo Martin-Brualla, Jonathan Ho, Andrea Tagliasacchi, and Mohammad Norouzi. Novel view synthesis with diffusion models, 2022.

[186] Or Patashnik, Daniel Garibi, Idan Azuri, Hadar Averbuch-Elor, and Daniel Cohen-Or. Localizing object-level shape variations with text-to-image diffusion models. In *ICCV*, 2023.

[187] Shaoteng Liu, Yuechen Zhang, Wenbo Li, Zhe Lin, and Jiaya Jia. Video-p2p: Video editing with cross-attention control. *arXiv:2303.04761*, 2023.

[188] Chenyang QI, Xiaodong Cun, Yong Zhang, Chenyang Lei, Xintao Wang, Ying Shan, and Qifeng Chen. Fatezero: Fusing attentions for zero-shot text-based video editing. In *ICCV*, 2023.

[189] Duygu Ceylan, Chun-Hao Huang, and Niloy J. Mitra. Pix2video: Video editing using image diffusion. *arXiv:2303.12688*, 2023.

[190] Wan-Duo Kurt Ma, JP Lewis, W Bastiaan Kleijn, and Thomas Leung. Directed diffusion: Direct control of object placement through attention guidance. *arXiv preprint arXiv:2302.13153*, 2023.

[191] Ajay Jain, Ben Mildenhall, Jonathan T Barron, Pieter Abbeel, and Ben Poole. Zero-shot text-guided object generation with dream fields. In *CVPR*, pages 867–876, 2022.

[192] Kevin Frans, Lisa Soros, and Olaf Witkowski. Clipdraw: Exploring text-to-drawing synthesis through language-image encoders. *Advances in Neural Information Processing Systems*, 35:5207–5218, 2022.

[193] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)*, 39(6):1–15, 2020.

[194] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.

[195] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. In *ICLR*, 2023.

[196] Juil Koo, Chanho Park, and Minhyuk Sung. Posterior distillation sampling. *arXiv preprint arXiv:2311.13831*, 2023.

[197] Amir Hertz, Kfir Aberman, and Daniel Cohen-Or. Delta denoising score. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2328–2337, 2023.

[198] Hyelin Nam, Gihyun Kwon, Geon Yeong Park, and Jong Chul Ye. Contrastive denoising score for text-guided latent diffusion image editing. *CVPR*, 2022.

[199] Subin Kim, Kyungmin Lee, June Suk Choi, Jongheon Jeong, Kihyuk Sohn, and Jinwoo Shin. Collaborative score distillation for consistent visual editing. In *NeurIPS*, 2023.

[200] Oren Katzir, Or Patashnik, Daniel Cohen-Or, and Dani Lischinski. Noise-free score distillation. *arXiv preprint arXiv:2310.17590*, 2023.

[201] Thiemo Alldieck, Nikos Kolotouros, and Cristian Sminchisescu. Score distillation sampling with learned manifold corrective, 2024.

[202] Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. *NeurIPS*, 2023.

[203] Junzhe Zhu and Peiye Zhuang. Hifa: High-fidelity text-to-3d generation with advanced diffusion guidance, 2023.

[204] Xin Yu, Yuan-Chen Guo, Yangguang Li, Ding Liang, Song-Hai Zhang, and Xiaojuan Qi. Text-to-3d with classifier score distillation, 2023.

[205] Zi-Xin Zou, Weihao Cheng, Yan-Pei Cao, Shi-Sheng Huang, Ying Shan, and Song-Hai Zhang. Sparse3d: Distilling multiview-consistent diffusion for object reconstruction from sparse views, 2023.

[206] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.

[207] Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, and Qian Yu. Svgdreamer: Text guided svg generation with diffusion model. *arXiv preprint arXiv:2312.16476*, 2023.

[208] Ximing Xing, Chuang Wang, Haitao Zhou, Jing Zhang, Qian Yu, and Dong Xu. Diffsketcher: Text guided vector sketch synthesis through latent diffusion models. 2023.

[209] Gal Metzer, Elad Richardson, Or Patashnik, Raja Giryes, and Daniel Cohen-Or. Latent-nerf for shape-guided generation of 3d shapes and textures. In *CVPR*, pages 12663–12673, 2023.

[210] Tianshi Cao, Karsten Kreis, Sanja Fidler, Nicholas Sharp, and KangXue Yin. Texfusion: Synthesizing 3d textures with text-guided image diffusion models. In *ICCV*, 2023.

[211] Dave Zhenyu Chen, Haoxuan Li, Hsin-Ying Lee, Sergey Tulyakov, and Matthias Nießner. Scenetex: High-quality texture synthesis for indoor scenes via diffusion priors. *arXiv preprint arXiv:2311.17261*, 2023.

[212] Dave Zhenyu Chen, Yawar Siddiqui, Hsin-Ying Lee, Sergey Tulyakov, and Matthias Nießner. Text2tex: Text-driven texture synthesis via diffusion models. *arXiv preprint arXiv:2303.11396*, 2023.

[213] Kim Youwang, Tae-Hyun Oh, and Gerard Pons-Moll. Paint-it: Text-to-texture synthesis via deep convolutional texture map optimization and physically-based rendering. In *CVPR*, 2024.

[214] Shir Iluz, Yael Vinker, Amir Hertz, Daniel Berio, Daniel Cohen-Or, and Ariel Shamir. Word-as-image for semantic typography. *SIGGRAPH*, 2023.

[215] Lea Muller, Vickie Ye, Georgios Pavlakos, Michael J. Black, and Angjoo Kanazawa. Generative proxemics: A prior for 3D social interaction from images. 2024.

[216] Sherwin Bahmani, Ivan Skorokhodov, Victor Rong, Gordon Wetzstein, Leonidas Guibas, Peter Wonka, Sergey Tulyakov, Jeong Joon Park, Andrea Tagliasacchi, and David B. Lindell. 4d-fy: Text-to-4d generation using hybrid score distillation sampling. *arXiv preprint arXiv:2311.17984*, 2023.

[217] Uriel Singer, Shelly Sheynin, Adam Polyak, Oron Ashual, Iurii Makarov, Filippos Kokkinos, Naman Goyal, Andrea Vedaldi, Devi Parikh, Justin Johnson, et al. Text-to-4d dynamic scene generation. *arXiv preprint arXiv:2301.11280*, 2023.

[218] Huan Ling, Seung Wook Kim, Antonio Torralba, Sanja Fidler, and Karsten Kreis. Align your gaussians: Text-to-4d with dynamic 3d gaussians and composed diffusion models, 2024.

[219] Ryan Burgert, Xiang Li, Abe Leite, Kanchana Ranasinghe, and Michael Ryoo. Diffusion illusions: Hiding images in plain sight, June 2023.

[220] Taoran Yi, Jiemin Fang, Junjie Wang, Guanjun Wu, Lingxi Xie, Xiaopeng Zhang, Wenyu Liu, Qi Tian, and Xinggang Wang. Gaussiandreamer: Fast generation from text to 3d gaussians by bridging 2d and 3d diffusion models, 2023.

[221] Jiaxiang Tang, Jiawei Ren, Hang Zhou, Ziwei Liu, and Gang Zeng. Dreamgaussian: Generative gaussian splatting for efficient 3d content creation, 2023.

[222] Chen-Hsuan Lin, Jun Gao, Luming Tang, Towaki Takikawa, Xiaohui Zeng, Xun Huang, Karsten Kreis, Sanja Fidler, Ming-Yu Liu, and Tsung-Yi Lin. Magic3d: High-resolution text-to-3d content creation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 300–309, June 2023.

[223] Rui Chen, Yongwei Chen, Ningxin Jiao, and Kui Jia. Fantasia3d: Disentangling geometry and appearance for high-quality text-to-3d content creation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023.

[224] Yichun Shi, Peng Wang, Jianglong Ye, Long Mai, Kejie Li, and Xiao Yang. MVDream: Multi-view diffusion for 3d generation. In *The Twelfth International Conference on Learning Representations*, 2024.

[225] Yuan Liu, Cheng Lin, Zijiao Zeng, Xiaoxiao Long, Lingjie Liu, Taku Komura, and Wenping Wang. Syncdreamer: Generating multiview-consistent images from a single-view image. In *The Twelfth International Conference on Learning Representations*, 2024.

[226] Ruoshi Liu, Rundi Wu, Basile Van Hoorick, Pavel Tokmakov, Sergey Zakharov, and Carl Vondrick. Zero-1-to-3: Zero-shot one image to 3d object. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9298–9309, October 2023.

[227] Guocheng Qian, Jinjie Mai, Abdullah Hamdi, Jian Ren, Aliaksandr Siarohin, Bing Li, Hsin-Ying Lee, Ivan Skorokhodov, Peter Wonka, Sergey Tulyakov, and Bernard Ghanem. Magic123: One image to high-quality 3d object generation using both 2d and 3d diffusion priors. In *The Twelfth International Conference on Learning Representations*, 2024.

[228] Xudong Mao, Qing Li, Haoran Xie, Raymond YK Lau, Zhen Wang, and Stephen Paul Smolley. Least squares generative adversarial networks. In *ICCV*, 2017.

[229] Rewon Child. Very deep vaes generalize autoregressive models and can outperform them on images. In *ICLR*, 2020.

[230] Gaurav Parmar, Dacheng Li, Kwonjoon Lee, and Zhuowen Tu. Dual contradistinctive generative autoencoder. In *CVPR*, 2021.

[231] Arash Vahdat and Jan Kautz. Nvae: A deep hierarchical variational autoencoder. *NeurIPS*, 2020.

[232] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *NeurIPS*, 2021.

[233] Pauline Luc, Aidan Clark, Sander Dieleman, Diego de Las Casas, Yotam Doron, Albin Cassirer, and Karen Simonyan. Transformation-based adversarial video prediction on large-scale data. *arXiv preprint arXiv:2003.04035*, 2020.

[234] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *NeurIPS*, 2020.

[235] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. Ucf101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012.

[236] Wei Xiong, Wenhan Luo, Lin Ma, Wei Liu, and Jiebo Luo. Learning to generate time-lapse videos using multi-stage dynamic generative adversarial networks. In *CVPR*, 2018.

[237] Aliaksandr Siarohin, Stéphane Lathuilière, Sergey Tulyakov, Elisa Ricci, and Nicu Sebe. First order motion model for image animation. In *NeurIPS*, December 2019.

[238] Jort F Gemmeke, Daniel PW Ellis, Dylan Freedman, Aren Jansen, Wade Lawrence, R Channing Moore, Manoj Plakal, and Marvin Ritter. Audio set: An ontology and human-labeled dataset for audio events. In *ICASSP*, 2017.

[239] Md Amirul Islam, Sen Jia, and Neil DB Bruce. How much position information do convolutional neural networks encode? In *ICLR*, 2019.

[240] Osman Semih Kayhan and Jan C van Gemert. On translation invariance in cnns: Convolutional layers can exploit absolute spatial location. In *CVPR*, 2020.

[241] Rui Xu, Xintao Wang, Kai Chen, Bolei Zhou, and Chen Change Loy. Positional encoding as spatial inductive bias in gans. In *CVPR*, 2021.

[242] Bilal Alsallakh, Narine Kokhlikyan, Vivek Miglani, Jun Yuan, and Orion Reblitz-Richardson. Mind the pad – CNNs can develop blind spots. In *ICLR*, 2021.

[243] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *ECCV*, 2016.

[244] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.

[245] Ting-Chun Wang, Ming-Yu Liu, Jun-Yan Zhu, Andrew Tao, Jan Kautz, and Bryan Catanzaro. High-resolution image synthesis and semantic manipulation with conditional gans. In *CVPR*, 2018.

[246] Ting-Chun Wang, Ming-Yu Liu, Jun-Yan Zhu, Guilin Liu, Andrew Tao, Jan Kautz, and Bryan Catanzaro. Video-to-video synthesis. In *NeurIPS*, 2018.

[247] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[248] Tero Karras, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Alias-free generative adversarial networks. *NeurIPS*, 2021.

[249] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *CVPR*, pages 6299–6308, 2017.

[250] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *ACL*, 2018.

[251] Lluis Castrejon, Nicolas Ballas, and Aaron Courville. Hierarchical video generation for complex data. *arXiv preprint arXiv:2106.02719*, 2021.

[252] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. In *ICML*, pages 8821–8831. PMLR, 2021.

[253] Moitreya Chatterjee and Anoop Cherian. Sound2sight: Generating visual dynamics from sound and context. In *ECCV*, 2020.

[254] Chenfei Wu, Lun Huang, Qianxi Zhang, Binyang Li, Lei Ji, Fan Yang, Guillermo Sapiro, and Nan Duan. Godiva: Generating open-domain videos from natural descriptions. *arXiv preprint arXiv:2104.14806*, 2021.

[255] Thomas Hayes, Songyang Zhang, Xi Yin, Guan Pang, Sasha Sheng, Harry Yang, Songwei Ge, Qiyuan Hu, and Devi Parikh. Mugen: A playground for video-audio-text multimodal understanding and generation. *arXiv preprint arXiv:2204.08058*, 2022.

[256] Thomas Unterthiner, Sjoerd van Steenkiste, Karol Kurach, Raphael Marinier, Marcin Michalski, and Sylvain Gelly. Towards accurate generative models of video: A new metric & challenges. *ICLR*, 2019.

[257] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. Learning spatiotemporal features with 3d convolutional networks. In *ICCV*, 2015.

[258] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI blog*, 2018.

[259] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.

[260] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

[261] Dinesh Acharya, Zhiwu Huang, Danda Pani Paudel, and Luc Van Gool. Towards high resolution video generation with progressive growing of sliced wasserstein gans. *arXiv preprint arXiv:1810.02419*, 2018.

[262] Emmanuel Kahembwe and Subramanian Ramamoorthy. Lower dimensional kernels for video discriminators. *Neural Networks*, 2020.

[263] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis. In *ICLR*, 2018.

[264] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.

[265] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided image synthesis and editing with stochastic differential equations. In *ICLR*, 2022.

[266] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *arXiv preprint arXiv:2206.00364*, 2022.

[267] Qinsheng Zhang and Yongxin Chen. Fast sampling of diffusion models with exponential integrator. In *ICLR*, 2023.

[268] Luping Liu, Yi Ren, Zhijie Lin, and Zhou Zhao. Pseudo numerical methods for diffusion models on manifolds. *arXiv preprint arXiv:2202.09778*, 2022.

[269] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver++: Fast solver for guided sampling of diffusion probabilistic models. *arXiv preprint arXiv:2211.01095*, 2022.

[270] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

[271] Jun Xu, Tao Mei, Ting Yao, and Yong Rui. Msr-vtt: A large video description dataset for bridging video and language. In *CVPR*, 2016.

[272] Tuomas Kynkäänniemi, Tero Karras, Miika Aittala, Timo Aila, and Jaakko Lehtinen. The role of imagenet classes in fréchet inception distance. In *ICLR*, 2023.

[273] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[274] Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. Movie gen: A cast of media foundation models. *arXiv preprint arXiv:2410.13720*, 2024.

[275] Genmo. Mochi 1. `https://github.com/genmoai/models`, 2024.

[276] Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025.

[277] Veo-Team, :, Agrim Gupta, Ali Razavi, Andeep Toor, Ankush Gupta, Dumitru Erhan, Eleni Shaw, Eric Lau, Frank Belletti, Gabe Barth-Maron, Gregory Shaw, Hakan Erdogan, Hakim Sidahmed, Henna Nandwani, Hernan Moraldo, Hyunjik Kim, Irina Blok, Jeff Donahue, José Lezama, Kory Mathewson, Kurtis David, Matthieu Kim Lorrain, Marc van Zee, Medhini Narasimhan, Miaosen Wang, Mohammad Babaeizadeh, Nelly Papalampidi, Nick Pezzotti, Nilpa Jha, Parker Barnes, Pieter-Jan Kindermans, Rachel Hornung, Ruben Villegas, Ryan Poplin, Salah Zaiem, Sander Dieleman, Sayna Ebrahimi, Scott Wisdom, Serena Zhang, Shlomi Fruchter, Signe Nørly, Weizhe Hua, Xinchen Yan, Yuqing Du, and Yutian Chen. Veo 2. 2024.

[278] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[279] BlackForestLabs. Flux. `https://blackforestlabs.ai/`.

[280] NVIDIA. Cosmos tokenizer: A suite of image and video neural tokenizers. *Tech Blog*, 2024.

[281] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *CVPR*, 2023.

[282] Pablo Pernias, Dominic Rampas, Mats Leon Richter, Christopher Pal, and Marc Aubreville. Würstchen: An efficient architecture for large-scale text-to-image diffusion models. In *The Twelfth International Conference on Learning Representations*, 2024.

[283] Agrim Gupta, Lijun Yu, Kihyuk Sohn, Xiuye Gu, Meera Hahn, Fei-Fei Li, Irfan Essa, Lu Jiang, and José Lezama. Photorealistic video generation with diffusion models. In *European Conference on Computer Vision*, pages 393–411. Springer, 2025.

[284] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.

[285] Junke Wang, Yi Jiang, Zehuan Yuan, Binyue Peng, Zuxuan Wu, and Yu-Gang Jiang. Omnitokenizer: A joint image-video tokenizer for visual generation. *arXiv preprint arXiv:2406.09399*, 2024.

[286] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*, 2021.

[287] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16000–16009, 2022.

[288] Fisher Yu, Haofeng Chen, Xin Wang, Wenqi Xian, Yingying Chen, Fangchen Liu, Vashisht Madhavan, and Trevor Darrell. Bdd100k: A diverse driving dataset for heterogeneous multitask learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2636–2645, 2020.

[289] Will Kay, Joao Carreira, Karen Simonyan, Brian Zhang, Chloe Hillier, Sudheendra Vijayanarasimhan, Fabio Viola, Tim Green, Trevor Back, Paul Natsev, et al. The kinetics human action video dataset. *arXiv preprint arXiv:1705.06950*, 2017.

[290] Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alexander Sorkine-Hornung, and Luc Van Gool. The 2017 davis challenge on video object segmentation. *arXiv:1704.00675*, 2017.

[291] Thomas Wiegand, Gary J Sullivan, Gisle Bjontegaard, and Ajay Luthra. Overview of the h. 264/avc video coding standard. *IEEE Transactions on circuits and systems for video technology*, 13(7):560–576, 2003.

[292] William B. Pennebaker, Joan L. Mitchell, Glen G Langdon, and Ronald B Arps. An overview of the basic principles of the q-coder adaptive binary arithmetic coder. *IBM Journal of research and development*, 32(6):717–726, 1988.

[293] Thomas Unterthiner, Sjoerd Van Steenkiste, Karol Kurach, Raphael Marinier, Marcin Michalski, and Sylvain Gelly. Towards accurate generative models of video: A new metric & challenges. *arXiv preprint arXiv:1812.01717*, 2018.

[294] Tony Lee, Michihiro Yasunaga, Chenlin Meng, Yifan Mai, Joon Sung Park, Agrim Gupta, Yunzhi Zhang, Deepak Narayanan, Hannah Teufel, Marco Bellagente, Minguk Kang, Taesung Park, Jure Leskovec, Jun-Yan Zhu, Fei-Fei Li, Jiajun Wu, Stefano Ermon, and Percy S Liang. Holistic evaluation of text-to-image models. In *NeurIPS*, 2023.

[295] Omer Bar-Tal, Hila Chefer, Omer Tov, Charles Herrmann, Roni Paiss, Shiran Zada, Ariel Ephrat, Junhwa Hur, Guanghui Liu, Amit Raj, Yuanzhen Li, Michael Rubinstein, Tomer Michaeli, Oliver Wang, Deqing Sun, Tali Dekel, and Inbar Mosseri. Lumiere: A space-time diffusion model for video generation, 2024.

[296] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade W Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa R Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. LAION-5b: An open large-scale dataset for training next generation image-text models. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022.

[297] Max Bain, Arsha Nagrani, Gül Varol, and Andrew Zisserman. Frozen in time: A joint video and image encoder for end-to-end retrieval. In *ICCV*, pages 1728–1738, 2021.

[298] Zeina Sinno and Alan Conrad Bovik. Large-scale study of perceptual video quality. *IEEE Transactions on Image Processing*, 28(2):612–627, 2018.

[299] Zhou Wang, Ligang Lu, and Alan C Bovik. Video quality assessment based on structural distortion measurement. *Signal processing: Image communication*, 19(2):121–132, 2004.

[300] Kalpana Seshadrinathan, Rajiv Soundararajan, Alan Conrad Bovik, and Lawrence K Cormack. Study of subjective and objective quality assessment of video. *IEEE transactions on Image Processing*, 19(6):1427–1441, 2010.

[301] Dingquan Li, Tingting Jiang, and Ming Jiang. Quality assessment of in-the-wild videos. In *Proceedings of the 27th ACM International Conference on Multimedia*, pages 2351–2359, 2019.

[302] Yaofang Liu, Xiaodong Cun, Xuebo Liu, Xintao Wang, Yong Zhang, Haoxin Chen, Yang Liu, Tieyong Zeng, Raymond Chan, and Ying Shan. Evalcrafter: Benchmarking and evaluating large video generation models, 2023.

[303] DC Dowson and BV Landau. The fréchet distance between multivariate normal distributions. *Journal of multivariate analysis*, 12(3):450–455, 1982.

[304] Tim Brooks, Janne Hellsten, Miika Aittala, Ting-Chun Wang, Timo Aila, Jaakko Lehtinen, Ming-Yu Liu, Alexei Efros, and Tero Karras. Generating long videos of dynamic scenes. *NeurIPS*, 35:31769–31781, 2022.

[305] Ruben Villegas, Jimei Yang, Seunghoon Hong, Xunyu Lin, and Honglak Lee. Decomposing motion and content for natural video sequence prediction. In *ICLR*, 2017.

[306] Yunseok Jang, Gunhee Kim, and Yale Song. Video prediction with appearance and motion conditions. In *ICML*, pages 2225–2234, 2018.

[307] Yaohui Wang, Piotr Bilinski, Francois Bremond, and Antitza Dantcheva. G3an: Disentangling appearance and motion for video generation. In *CVPR*, June 2020.

[308] Yingwei Li, Yi Li, and Nuno Vasconcelos. Resound: Towards action recognition without representation bias. In *ECCV*, 2018.

[309] De-An Huang, Vignesh Ramanathan, Dhruv Mahajan, Lorenzo Torresani, Manohar Paluri, Li Fei-Fei, and Juan Carlos Niebles. What makes a video a video: Analyzing temporal information in video understanding models and datasets. In *CVPR*, 2018.

[310] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. Temporal segment networks: Towards good practices for deep action recognition. In *ECCV*, 2016.

[311] Jinwoo Choi, Chen Gao, Joseph CE Messou, and Jia-Bin Huang. Why can't i dance in the mall? learning to mitigate scene bias in action recognition. In *NeurIPS*, 2019.

[312] Xin Liu, Silvia L Pintea, Fatemeh Karimi Nejadasl, Olaf Booij, and Jan C Van Gemert. No frame left behind: Full video action recognition. In *CVPR*, 2021.

[313] Laura Sevilla-Lara, Shengxin Zha, Zhicheng Yan, Vedanuj Goswami, Matt Feiszli, and Lorenzo Torresani. Only time can tell: Discovering temporal data for temporal modeling. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 535–544, January 2021.

[314] Gedas Bertasius, Heng Wang, and Lorenzo Torresani. Is space-time attention all you need for video understanding? In *ICML*, July 2021.

[315] Limin Wang, Bingkun Huang, Zhiyu Zhao, Zhan Tong, Yinan He, Yi Wang, Yali Wang, and Yu Qiao. Videomae v2: Scaling video masked autoencoders with dual masking. In *CVPR*, pages 14549–14560, June 2023.

[316] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.

[317] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *ICLR*, 2019.

[318] Raghav Goyal, Samira Ebrahimi Kahou, Vincent Michalski, Joanna Materzynska, Susanne Westphal, Heuna Kim, Valentin Haenel, Ingo Fruend, Peter Yianilos, Moritz Mueller-Freitag, Florian Hoppe, Christian Thurau, Ingo Bax, and Roland Memisevic. The "something something" video database for learning and evaluating visual common sense. In *ICCV*, Oct 2017.

[319] Andreas Rössler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies, and Matthias Nießner. FaceForensics++: Learning to detect manipulated facial images. In *ICCV*, 2019.

[320] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, pages 16000–16009, June 2022.

[321] Kunchang Li, Yali Wang, Yinan He, Yizhuo Li, Yi Wang, Limin Wang, and Yu Qiao. Uniformerv2: Spatiotemporal learning by arming image vits with video uniformer, 2022.

[322] Zhan Tong, Yibing Song, Jue Wang, and Limin Wang. Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training. *NeurIPS*, 35:10078–10093, 2022.

[323] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *ICCV*, 2021.

[324] Narek Tumanyan, Michal Geyer, Shai Bagon, and Tali Dekel. Plug-and-play diffusion features for text-driven image-to-image translation. In *CVPR*, 2023.

[325] Gaurav Parmar, Taesung Park, Srinivasa Narasimhan, and Jun-Yan Zhu. One-step image translation with text-to-image models. *arXiv preprint arXiv:2403.12036*, 2024.

[326] Ajay Jain, Amber Xie, and Pieter Abbeel. Vectorfusion: Text-to-svg by abstracting pixel-based diffusion models. In *CVPR*, pages 1911–1920, June 2023.

[327] Grace Luo, Lisa Dunlap, Dong Huk Park, Aleksander Holynski, and Trevor Darrell. Diffusion hyperfeatures: Searching through time and space for semantic correspondence. In *NeurIPS*, 2024.

[328] Luming Tang, Menglin Jia, Qianqian Wang, Cheng Perng Phoo, and Bharath Hariharan. Emergent correspondence from image diffusion. In *NeurIPS*, 2024.

[329] Alexander C Li, Mihir Prabhudesai, Shivam Duggal, Ellis Brown, and Deepak Pathak. Your diffusion model is secretly a zero-shot classifier. In *ICCV*, 2023.

[330] Jiarui Xu, Sifei Liu, Arash Vahdat, Wonmin Byeon, Xiaolong Wang, and Shalini De Mello. Open-vocabulary panoptic segmentation with text-to-image diffusion models. In *CVPR*, 2023.

[331] Haochen Wang, Xiaodan Du, Jiahao Li, Raymond A. Yeh, and Greg Shakhnarovich. Score jacobian chaining: Lifting pretrained 2d diffusion models for 3d generation. In *CVPR*, pages 12619–12629, June 2023.

[332] The ACCESS Project Colorado State University. tutorial: Rich text format (rtf) from microsoft word - the access project, 2012.

[333] Ian H Witten, David Bainbridge, and David M Nichols. *How to build a digital library*. Morgan Kaufmann, 2009.

[334] Arnaud Sahuguet and Fabien Azavant. Wysiwyg web wrapper factory (w4f), 1999.

[335] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. Peritext: A crdt for collaborative rich text editing. *Proceedings of the ACM on Human-Computer Interaction (PACMHCI)*, 2022.

[336] Claudia-Lavinia Ignat, Luc André, and Gérald Oster. Enhancing rich content wikis with real-time collaboration. *Concurrency and Computation: Practice and Experience*, 33(8):e4110, 2021.

[337] Maneesh Agrawala. Unpredictable black boxes are terrible interfaces, March 2023. https://magrawala.substack.com/p/unpredictable-black-boxes-are-terrible.

[338] Andrej Karpathy and Li Fei-Fei. Deep visual-semantic alignments for generating image descriptions. In *CVPR*, 2015.

[339] Justin Johnson, Andrej Karpathy, and Li Fei-Fei. Densecap: Fully convolutional localization networks for dense captioning. In *CVPR*, 2016.

[340] Rinon Gal, Yuval Alaluf, Yuval Atzmon, Or Patashnik, Amit Haim Bermano, Gal Chechik, and Daniel Cohen-or. An image is worth one word: Personalizing text-to-image generation using textual inversion. In *ICLR*, 2023.

[341] Xi Chen, Lianghua Huang, Yu Liu, Yujun Shen, Deli Zhao, and Hengshuang Zhao. Anydoor: Zero-shot object-level image customization. *arXiv preprint arXiv:2307.09481*, 2023.

[342] Dongxu Li, Junnan Li, and Steven CH Hoi. Blip-diffusion: Pre-trained subject representation for controllable text-to-image generation and editing. *arXiv preprint arXiv:2305.14720*, 2023.

[343] Xuhui Jia, Yang Zhao, Kelvin CK Chan, Yandong Li, Han Zhang, Boqing Gong, Tingbo Hou, Huisheng Wang, and Yu-Chuan Su. Taming encoder for zero fine-tuning image customization with text-to-image diffusion models. *arXiv preprint arXiv:2304.02642*, 2023.

[344] Wenhu Chen, Hexiang Hu, Yandong Li, Nataniel Rui, Xuhui Jia, Ming-Wei Chang, and William W Cohen. Subject-driven text-to-image generation via apprenticeship learning. *arXiv preprint arXiv:2304.00186*, 2023.

[345] Rinon Gal, Moab Arar, Yuval Atzmon, Amit H Bermano, Gal Chechik, and Daniel Cohen-Or. Encoder-based domain tuning for fast personalization of text-to-image models. *ACM Transactions on Graphics (TOG)*, 42(4):1–13, 2023.

[346] Jing Shi, Wei Xiong, Zhe Lin, and Hyun Joon Jung. Instantbooth: Personalized text-to-image generation without test-time finetuning. *arXiv preprint arXiv:2304.03411*, 2023.

[347] Raphael Tang, Akshat Pandey, Zhiying Jiang, Gefei Yang, K. V. S. Manoj Kumar, Jimmy Lin, and Ferhan Ture. What the daam: Interpreting stable diffusion using cross attention. *ArXiv*, abs/2210.04885, 2022.

[348] Jianbo Shi and J. Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000.

[349] Ulrike Von Luxburg. A tutorial on spectral clustering. *Statistics and computing*, 17:395–416, 2007.

[350] Yuxin Wen, Neel Jain, John Kirchenbauer, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Hard prompts made easy: Gradient-based discrete optimization for prompt tuning and discovery. In *NeurIPS*, 2023.

[351] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[352] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. In *NeurIPS*, 2021.

[353] OpenAI. Gpt-4 technical report, 2023.

[354] Chenliang Li, Haiyang Xu, Junfeng Tian, Wei Wang, Ming Yan, Bin Bi, Jiabo Ye, He Chen, Guohai Xu, Zheng Cao, et al. mplug: Effective and efficient vision-language learning by cross-modal skip-connections. In *EMNLP*, 2022.

[355] Inbar Huberman-Spiegelglas, Vladimir Kulikov, and Tomer Michaeli. An edit friendly ddpm noise space: Inversion and manipulations. *arXiv preprint arXiv:2304.06140*, 2023.

[356] Chen Henry Wu and Fernando De la Torre. Unifying diffusion models' latent space, with applications to cyclediffusion and guidance. *arXiv preprint arXiv:2210.05559*, 2022.

[357] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollar, and Ross Girshick. Segment anything. In *ICCV*, 2023.

[358] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023.

[359] Tianhe Ren, Shilong Liu, Ailing Zeng, Jing Lin, Kunchang Li, He Cao, Jiayu Chen, Xinyu Huang, Yukang Chen, Feng Yan, Zhaoyang Zeng, Hao Zhang, Feng Li, Jie Yang, Hongyang Li, Qing Jiang, and Lei Zhang. Grounded sam: Assembling open-world models for diverse visual tasks, 2024.

[360] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[361] Kyungmin Lee, Kihyuk Sohn, and Jinwoo Shin. Dreamflow: High-quality text-to-3d generation by approximating probability flow. In *ICLR*, 2024.

[362] E. Schrödinger. Sur la théorie relativiste de l'électron et l'interprétation de la mécanique quantique. *Annales de l'institut Henri Poincaré*, 2(4):269–310, 1932.

[363] Yongxin Chen, Tryphon T. Georgiou, and Michele Pavon. On the relation between optimal transport and schrödinger bridges: A stochastic control viewpoint. *Journal of Optimization Theory and Applications*, 169:671–691, 2014.

[364] Yongxin Chen and Tryphon Georgiou. Stochastic bridges of linear systems. *IEEE Transactions on Automatic Control*, 61(2):526–531, 2016.

[365] Christian Léonard. A survey of the schrödinger problem and some of its connections with optimal transport, 2013.

[366] Xuan Su, Jiaming Song, Chenlin Meng, and Stefano Ermon. Dual diffusion implicit bridges for image-to-image translation. In *ICLR*, 2022.

[367] Yixun Liang, Xin Yang, Jiantao Lin, Haodong Li, Xiaogang Xu, and Yingcong Chen. Luciddreamer: Towards high-fidelity text-to-3d generation via interval score matching. In *CVPR*, 2023.

[368] Artem Lukoianov, Haitz Sáez de Ocáriz Borde, Kristjan Greenewald, Vitor Campagnolo Guizilini, Timur Bagautdinov, Vincent Sitzmann, and Justin Solomon. Score distillation via reparametrized ddim. *arXiv preprint arXiv:2405.15891*, 2024.

[369] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*. PMLR.

[370] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, 2022.

[371] Gefei Wang, Yuling Jiao, Qian Xu, Yang Wang, and Can Yang. Deep generative learning via schrödinger bridge. In *ICML*, 2021.

[372] Valentin De Bortoli, James Thornton, Jeremy Heng, and Arnaud Doucet. Diffusion schrödinger bridge with applications to score-based generative modeling. In *NeurIPS*, 2021.

[373] Guan-Horng Liu, Arash Vahdat, De-An Huang, Evangelos A Theodorou, Weili Nie, and Anima Anandkumar. I2sb: image-to-image schrödinger bridge. In *ICML*, 2023.

[374] Tianrong Chen, Guan-Horng Liu, and Evangelos A Theodorou. Likelihood training of schrödinger bridge using forward-backward sdes theory. In *ICLR*, 2022.

[375] Christina Tsalicoglou, Fabian Manhardt, Alessio Tonioni, Michael Niemeyer, and Federico Tombari. Textmesh: Generation of realistic 3d meshes from text prompts. *arXiv preprint arXiv:2304.12439*, 2023.

[376] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, 2014.

[377] Yuan-Chen Guo, Ying-Tian Liu, Ruizhi Shao, Christian Laforte, Vikram Voleti, Guan Luo, Chia-Hao Chen, Zi-Xin Zou, Chen Wang, Yan-Pei Cao, and Song-Hai Zhang. threestudio: A unified framework for 3d content generation. `https://github.com/threestudio-project/threestudio`, 2023.

[378] Wenqi Dong, Bangbang Yang, Lin Ma, Xiao Liu, Liyuan Cui, Hujun Bao, Yuewen Ma, and Zhaopeng Cui. Coin3d: Controllable and interactive 3d assets generation with proxy-guided conditioning. *arXiv preprint arXiv:2405.08054*, 2024.

[379] Assaf Shocher, Amil V Dravid, Yossi Gandelsman, Inbar Mosseri, Michael Rubinstein, and Alexei A Efros. Idempotent generative network. In *ICLR*, 2024.